MICROCONTROLLERS

(Common to EC/TC/EE/IT/BM/ML)

Sub Code: 10ES42 IA Marks: 25
Hrs/ Week: 04 Exam Hours: 03
Total Hrs. 52 Exam Marks: 100

PART-A

UNIT 1:

Microprocessors and microcontrollers. Introduction, Microprocessors and Microcontrollers, RISC & CISC CPU Architectures, Harvard & Von- Neumann CPU architecture, Computer software.

The 8051 Architecture: Introduction, Architecture of 8051, Pin diagram of 8051, Memory organization, External Memory interfacing, Stacks. 6 Hrs

UNIT 2:

Addressing Modes: Introduction, Instruction syntax, Data types, Subroutines, Addressing modes: Immediate addressing , Register addressing, Direct addressing, Indirect addressing, relative addressing, Absolute addressing, Long addressing, Indexed addressing, Bit inherent addressing, bit direct addressing. Instruction set: Instruction timings, 8051 instructions: Data transfer instructions, Arithmetic instructions, Logical instructions, Branch instructions, Subroutine instructions, Bit manipulation instruction. 6 Hrs

UNIT 3:

8051 programming: Assembler directives, Assembly language programs and Time delay calculations.

6 Hrs

UNIT 4:

8051 Interfacing and Applications: Basics of I/O concepts, I/O Port Operation, Interfacing 8051 to LCD, Keyboard, parallel and serial ADC, DAC, Stepper motor interfacing and DC motor interfacing and programming 7 Hrs

PART-B

UNIT 5:

8051 Interrupts and Timers/counters: Basics of interrupts, 8051 interrupt structure, Timers and Counters, 8051 timers/counters, programming 8051 timers in assembly and C. 6 Hrs

UNIT 6:

8051 Serial Communication: Data communication, Basics of Serial Data Communication, 8051 Serial Communication, connections to RS-232, Serial communication Programming in assembly and C.

8255A Programmable Peripheral Interface:, Architecture of 8255A, I/O addressing,, I/O devices interfacing with 8051 using 8255A. 6 Hrs

Course Aim – The MSP430 microcontroller is ideally suited for development of low-power embedded systems that must run on batteries for many years. There are also applications where MSP430 microcontroller must operate on energy harvested from the environment. This is possible due to the ultra-low power operation of MSP430 and the fact that it provides a complete system solution including a RISC CPU, flash memory, on-chip data converters and on-chip peripherals.

UNIT 7:

Motivation for MSP430microcontrollers – Low Power embedded systems, On-chip peripherals (analog and digital), low-power RF capabilities. Target applications (Single-chip, low cost, low power, high performance system design). 2 Hrs

MSP430 RISC CPU architecture, Compiler-friendly features, Instruction set, Clock system, Memory subsystem. Key differentiating factors between different MSP430 families. 2 Hrs

Introduction to Code Composer Studio (CCS v4). Understanding how to use CCS for Assembly, C, Assembly+C projects for MSP430 microcontrollers. Interrupt programming. 3 Hrs

Digital I/O – I/O ports programming using C and assembly, Understanding the muxing scheme of the MSP430 pins. 2 Hrs

UNIT 8:

On-chip peripherals. Watchdog Timer, Comparator, Op-Amp, Basic Timer, Real Time Clock (RTC), ADC, DAC, SD16, LCD, DMA. 2 Hrs

Using the Low-power features of MSP430. Clock system, low-power modes, Clock request feature, Low-power programming and Interrupt. 2 Hrs

Interfacing LED, LCD, External memory. Seven segment LED modules interfacing. Example – Real-time clock. 2 Hrs

Case Studies of applications of MSP430 - Data acquisition system, Wired Sensor network, Wireless sensor network with Chipcon RF interfaces. 3 Hrs

TEXT BOOKS:

1. “The 8051 Microcontroller and Embedded Systems – using assembly and C ”-, Muhammad Ali Mazidi and Janice Gillespie Mazidi and Rollin D. McKinlay; PHI, 2006 / Pearson, 2006

2. “MSP430 Microcontroller Basics”, John Davies, Elsevier, 2010 (Indian edition available)

REFERENCE BOOKS:

1. "The 8051 Microcontroller Architecture, Programming & Applications", 2e Kenneth J.Ayala , Penram International, 1996 / Thomson Learning 2005.

2. "The 8051 Microcontroller", V.Udayashankar and MalikarjunaSwamy, TMH, 2009

3. MSP430 Teaching CD-ROM, Texas Instruments, 2008 (can be requested http://www.uniti.in)

4. Microcontrollers: Architecture, Programming, Interfacing and System Design",Raj Kamal,

"Pearson Education, 2005

INDEX SHEET

Chapter	TOPIC	PAGE NO.
1	Microprocessors and microcontroller: Introduction,	7-29
	Microprocessors and Microcontrollers, RISC & CISC CPU Architectures, Harvard & Von- Neumann CPU architecture, Computer software.	
	The 8051 Architecture: Introduction, Architecture of 8051, Pin diagram of 8051, Memory organization, External Memory interfacing, Stacks.	
2	Addressing Modes: Introduction, Instruction syntax, Data types, Subroutines,	30-58
	Addressing modes: Immediate addressing, Register addressing, Direct addressing, Indirect addressing, relative addressing, Absolute addressing, Long addressing, Indexed addressing, Bit inherent addressing, and bit direct addressing.	
	Instruction set: Instruction timings, 8051 instructions: Data transfer instructions, Arithmetic instructions, Logical instructions, Branch instructions, Subroutine instructions, Bit manipulation instruction	
3	8051 programming: Assembler directives, Assembly language programs and Time delay calculations.	59-61
4	8051 Interfacing and Applications: Basics of I/O concepts, I/O Port Operation, Interfacing 8051 to LCD, Keyboard, parallel and serial ADC, DAC, Stepper motor interfacing and DC motor interfacing and programming	62-90
5	8051 Interrupts and Timers/counters: Basics of interrupts, 8051 interrupt structure, Timers and Counters, 8051 timers/counters, programming 8051 timers in assembly and C.	91-100
6	8051 Serial Communication: Data communication, Basics of Serial Data Communication, 8051 Serial Communication, connections to RS-232, Serial communication Programming in assembly and C. 8255A Programmable Peripheral Interface:, Architecture of 8255A, I/O addressing,, I/O devices interfacing with 8051 using 8255A.	100-109
7	Motivation for MSP430microcontrollers – Low Power embedded systems, On-chip peripherals (analog and digital), low-power RF capabilities. Target applications (Single-chip, low cost, low power, high performance system design).	110-117
	MSP430 RISC CPU architecture, Compiler-friendly features, Instruction set, Clock system, Memory subsystem. Key differentiating factors between different MSP430 families.	
	Introduction to Code Composer Studio (CCS v4). Understanding how to use CCS for Assembly, C, Assembly+C projects for MSP430 microcontrollers. Interrupt programming.	
	Digital I/O – I/O ports programming using C and assembly,	

	Understanding the muxing scheme of the MSP430 pins.	
8	On-chip peripherals. Watchdog Timer, Comparator, Op-Amp, Basic Timer, Real Time Clock (RTC), ADC, DAC, SD16, LCD, DMA. Using the Low-power features of MSP430. Clock system, low-power modes, Clock request feature, Low-power programming and Interrupt. Interfacing LED, LCD, External memory. Seven segment LED modules interfacing. Example – Real-time clock. Case Studies of applications of MSP430 - Data acquisition system, Wired Sensor network, Wireless sensor network with Chipcon RF interfaces.	118-121

UNIT 1:

Microprocessors and microcontroller. Introduction, Microprocessors and Microcontrollers, RISC & CISC CPU Architectures, Harvard & Von- Neumann CPU architecture, Computer software.

The 8051 Architecture: Introduction, Architecture of 8051, Pin diagram of 8051, Memory organization, External Memory interfacing, Stacks. 6 Hrs

Computer: A computer is a multipurpose programmable machine that reads binary instructions from its memory , accepts binary data as input ,processes the data according to those instructions and provides results as output. It is a programmable device made up of both *hardware* and *software*. The various components of the computer are called *hardware*. A set of instructions written for the computer to solve a specific task is called program and collection of programs is called *software* .

The computer hardware consists of four main components. The central processing unit which acts as computer's brain. Input unit through which program and data can be entered to computer, output unit on which the results of the computations can be displayed. Memory in which data and programs are stored.

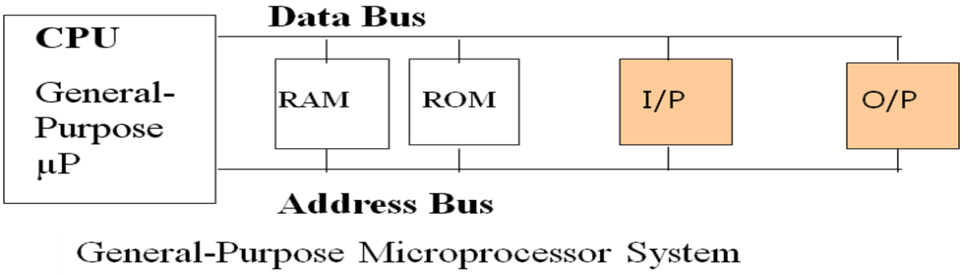


Fig 1. Block diagram of a microcomputer

A computer that is designed using a microprocessor as its CPU , is known as a microcomputer.

Microprocessor or 'Computer on Chip' first became a commercial reality in 1971 with the introduction of the 4 bit 4004 by Intel. A byproduct of Microprocessor development was Microcontroller. The same fabrication technology and programming concept that make the general purpose microprocessor also yielded the Microcontroller.

Microprocessors:

A microprocessor is a general purpose digital computer central processing unit (CPU). Although known as a 'Computer on Chip' the Microprocessor in no sense a complete digital computer. Block diagram of a Microprocessor CPU which contains ALU; Program counter (PC), a stack pointer (SP) ,some working registers , a clock timing circuit and interrupt circuit s is shown in the following figure

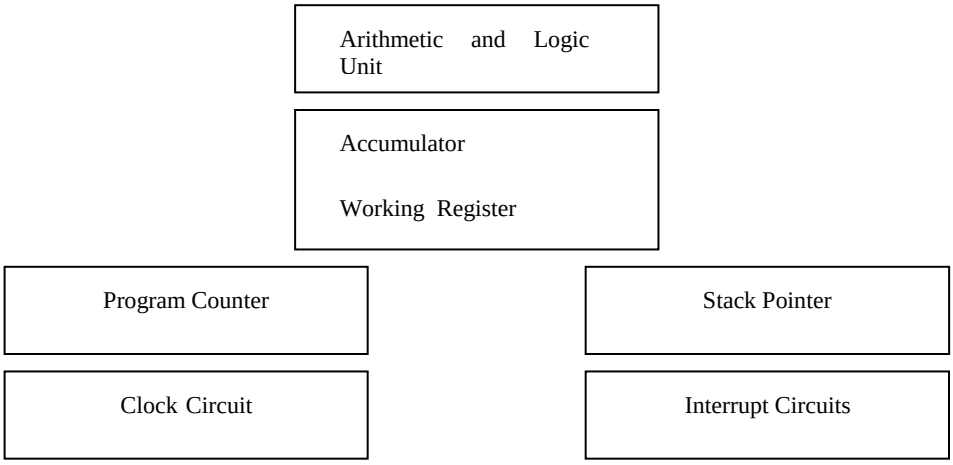


Fig.2.Block Diagram of a Microprocessor

To make a computer microcomputer one must add memory usually RAM and ROM, memory decoders , an oscillator and a number of Input ,Output devices such as serial and parallel ports. In addition special purpose devices such as interrupt handler and counters may be added to relieve the CPU from time consuming counting or timing cores. When the Microcomputer is equipped with mass storage devices , I/O peripherals such

as a key board and a display CRT it yields a small computer that can be applied to a range of general purpose applications.

The hardware design of a microprocessor is arranged such that a very small or very large system can be configured around the CPU as the application demands as shown in Fig1. The prime use of the Microprocessor is to read data , perform extensive calculations on that data, and store those calculations in a mass storage device or display the results for human use. The programs used by microprocessor are stored in the mass storage device and loaded into RAM as user directs. A few microprocessor program are stored in ROM . The ROM based programs are primarily small fixed programs that operate peripherals and other fixed devices that are connected to the system.

Microcontroller: A Microcontroller is a programmable digital processor with necessary peripherals. Both microcontrollers and microprocessors are complex sequential digital circuits meant to carry out job according to the program / instructions. Sometimes analog input/output interface makes a part of microcontroller circuit as mixed mode(both analog and digital) in nature.

A microcontroller can be compared to a Swiss knife with multiple functions incorporated in the same Integrated Circuits. Block diagram of a typical Microcontroller which is a true computer on a chip is shown below. The design incorporates all the features found in microprocessor CPU : ALU,PC, SP and registers. It also has other features needed to make a complete computer: ROM, RAM, Parallel I/O, serial I/O, Counters and clock circuits. Like the microprocessor , a microcontroller is a general purpose device, but one that is meant to read data, perform limited calculations on that data and control its environment based on those calculations. The prime use of microcontroller is to control the operation of a machine using a fixed program that is stored in ROM and that does not change over the lifetime of the system.

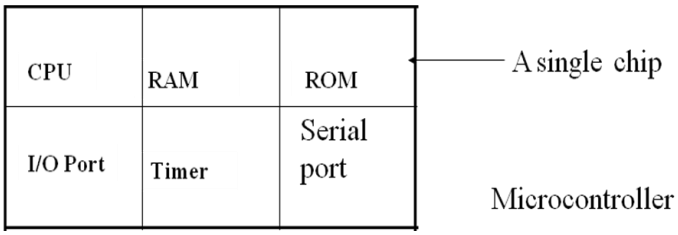


Fig3. Block diagram of a single chip computer

Complex Instruction Set Computer (CISC):

Memory in those days was expensive. Bigger programs required more storage which included more money . There was a need to reduce the number of instructions per program . This was achieved by having multiple operations within single instruction. Multiple operations lead to many different kinds of instructions .Access to memory in turn makes the instruction length variable and fetch-decode execute time unpredictable – making it more complex. Thus hardware was made to understand the complexity of instruction set. The computer having such instruction set was named as Complex Instruction Set Computer (CISC). Intel 8051 is an example for CISC architecture.

Reduced Instruction Set Computer (RISC):

In applications which require more of input , output related operations having few simple instructions that are of the same length allows memory access only with explicit load and store instructions. Hence each instruction performs less work but instruction execution time among different instructions is consistent.

This would lead to instruction execution by hardware including multiple number of registers inside CPU. The computer using such instructions is called Reduced Instruction Set Computer (RISC). PIC microcontroller manufactured by Microchip Company is an example for RISC architecture.

Vonneumann (Princeton) and Harvard Architecture :

Intel's 8051 employs Harvard architecture. A microcontroller has some embedded peripherals and Input/Output (I/O) devices. The data transfer to these devices takes place through I/O registers.

In a microprocessor, input /output (I/O) devices are externally interfaced and are mapped either to memory address (memory mapped I/O) or a separate I/O address space (I/O mapped I/O). There are two possible architectures one is Princeton (Von Neumann) and another is Harvard .I/O Registers space in Princeton architecture have only one memory interface for program memory (ROM) and data memory (RAM). One option is to map the I/O Register as a part of data memory or variable RAM area (memory mapped I/O). Alternatively a separate I/O register space can be assigned (I/O Mapped I/O) . Both the arrangements are shown in Fig.4.

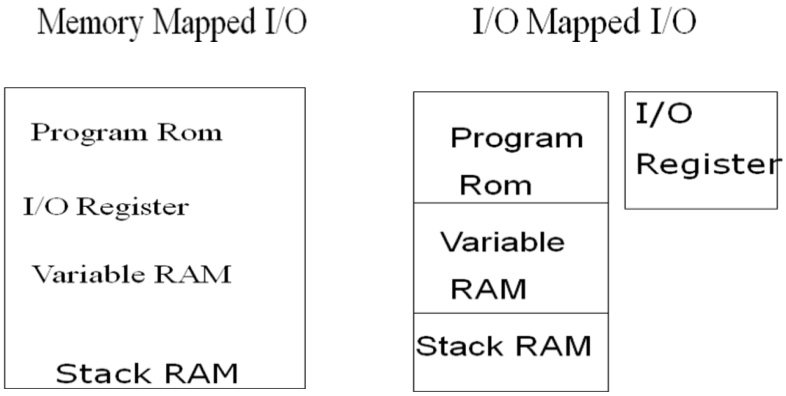


Fig 4. Input/Output Registers in Princeton Architecture

As shown in Fig 4. Program memory and Data memory are together in both the arrangements. The Princeton or Von neumann architecture one bus is used to carry the address and data with an appropriate multiplexing technique ,which in turn reduces the cost. But Harvard architecture which 8051 employs has separate Data memory and separate Code or Program memory . The Fig. 5 and Fig .6 show the need for separate address and data bus for each Program and Data memory in Harvard architecture. Since there are separate bus for access the operation of fetching the code and data can happen simultaneously which increases the speed of operation of execution inside CPU.

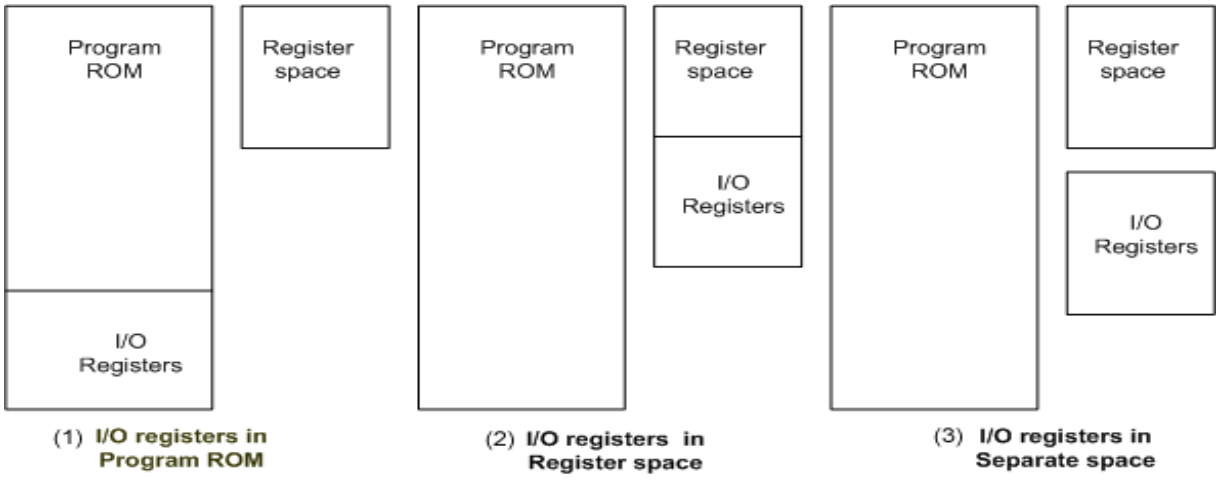


Fig. 5.Organization of I/O registers in Harvard Architecture

In Fig. 5**,** the first option is difficult to implement as there is no means to write to program ROM area. It is also complicated to have a separate I/O space as shown in (3). Hence the second option where I/O registers are placed in the register space is widely used in Harvard architecture.

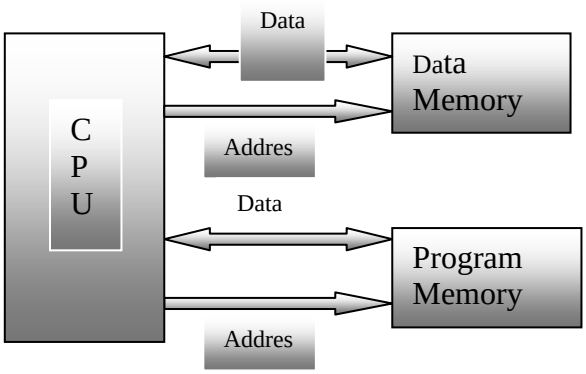


Fig6. Harvard Architecture

Computer Software: A set of instructions written in a specific sequence for computer to solve a specific task is called a program, and software is collection of programs. The program stored in the computer memory in the form of 0s and 1sand it is called as machine level instructions. Since it would be difficult to remember machine codes in the form of binary numbers an intermediate level of language for programming, between higher and machine level was developed and is known as assembly level language . Assembly language programs are written using assembly instructions known as mnemonics.

For example in CLR A, instruction CLR means clear and A means accumulator. The program mnemonics are converted to machine codes in the form of binary by a software called *Assembler*.

The Assembly language programming requires a detailed knowledge of the architecture with which the program is executed. In order to overcome the drawback of assembly language programming Higher level language like C,C++ are introduced where an interpreter or a compiler takes care of translating a higher level source code into machine codes.

Development/Classification of microcontrollers : Microcontrollers have gone through a silent evolution (invisible). The evolution can be rightly termed as silent as the impact or application of a microcontroller is not well known to a common user, although microcontroller technology has undergone significant change since early 1970's. Development of some popular microcontrollers is given as follows.

Intel 4004	4 bit (2300 PMOS trans, 108 kHz)	1971
Intel 8048	8 bit	1976
Intel 8031	8 bit (ROM-less)	.
Intel 8051	8 bit (Mask ROM)	1980
Microchip PIC16C64	8 bit	1985
Motorola 68HC11	8 bit (on chip ADC)	.
Intel 80C196	16 bit	1982
Atmel AT89C51	8 bit (Flash memory)	.
Microchip PIC 16F877	8 bit (Flash memory + ADC)	.

We use more number of microcontrollers compared to microprocessors. Microprocessors are primarily used for computational purpose, whereas microcontrollers find wide application in devices needing real time processing and control. Application of microcontrollers are numerous. Starting from domestic applications such as in washing machines, TVs, air conditioners, microcontrollers are used in automobiles, process control industries , cell phones, electrical drives, robotics and in space applications.

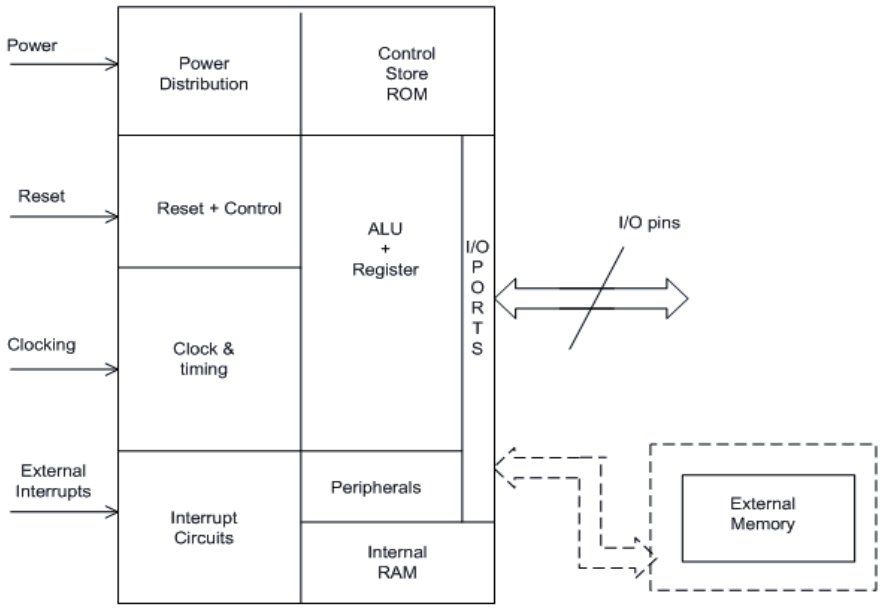


Fig. 7. Internal Structure of a typical Microcontroller

The one we are studying is a 8 bit Embedded Microcontroller introduced by Intel, 8051.

8051 ARCHITECTURE:

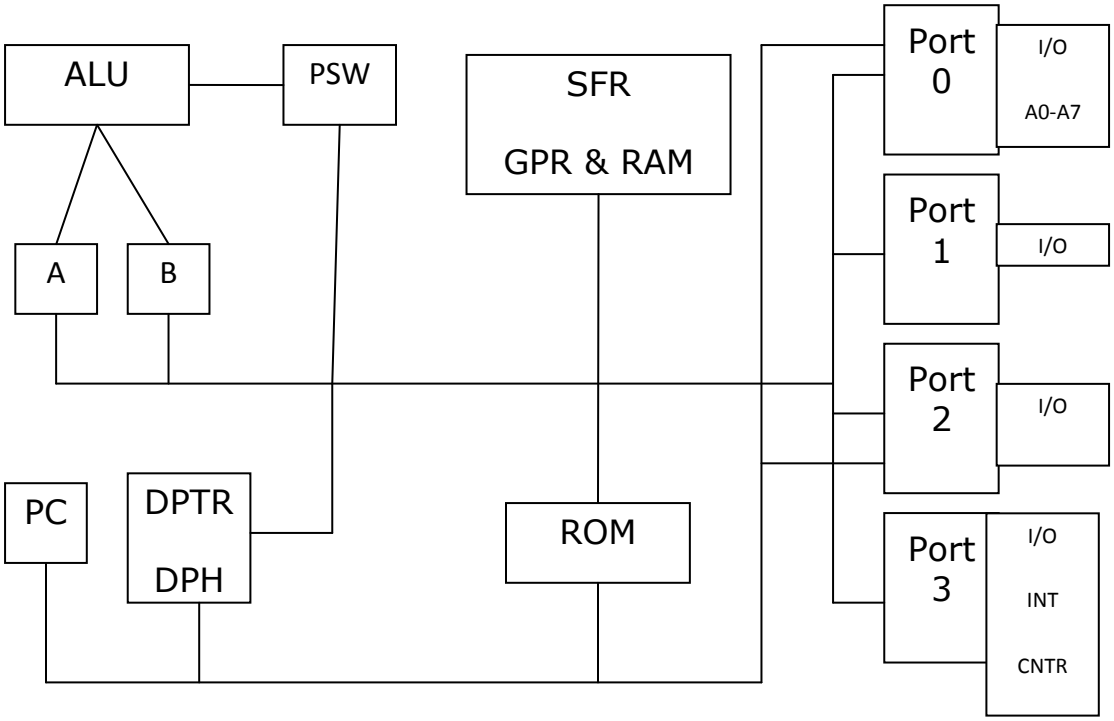


Fig 8. Block diagram of 8051 Microcontroller

Salient Features

- Eight bit CPU with registers A (Accumulator) and B
- Sixteen bit Program counter (PC) and a data pointer (DPTR)
- 8 Bit Program Status Word (PSW)
- 8 Bit Stack Pointer
- 4K Code Memory
- Internal Memory of 128 Bytes
- 32 I/O Pins arranged as 4 , 8 Bit ports
- Two 16 Bit Timer/Counter :T0, T1
- Full Duplex serial data receiver/transmitter
- Control Registers : TCON,TMOD,SCON,PCON,IP and IE
- Two External and Internal Interrupt sources
- Oscillator and clock circuits
-

The programming model of 8051 shows the 8051 as the collection of 8 and 16 bit registers and 8 bit memory locations. These registers and memory locations can be made to operate using software instructions that are incorporated as part of the program instructions. The pin configuration of 8051 is shown in Fig.9.

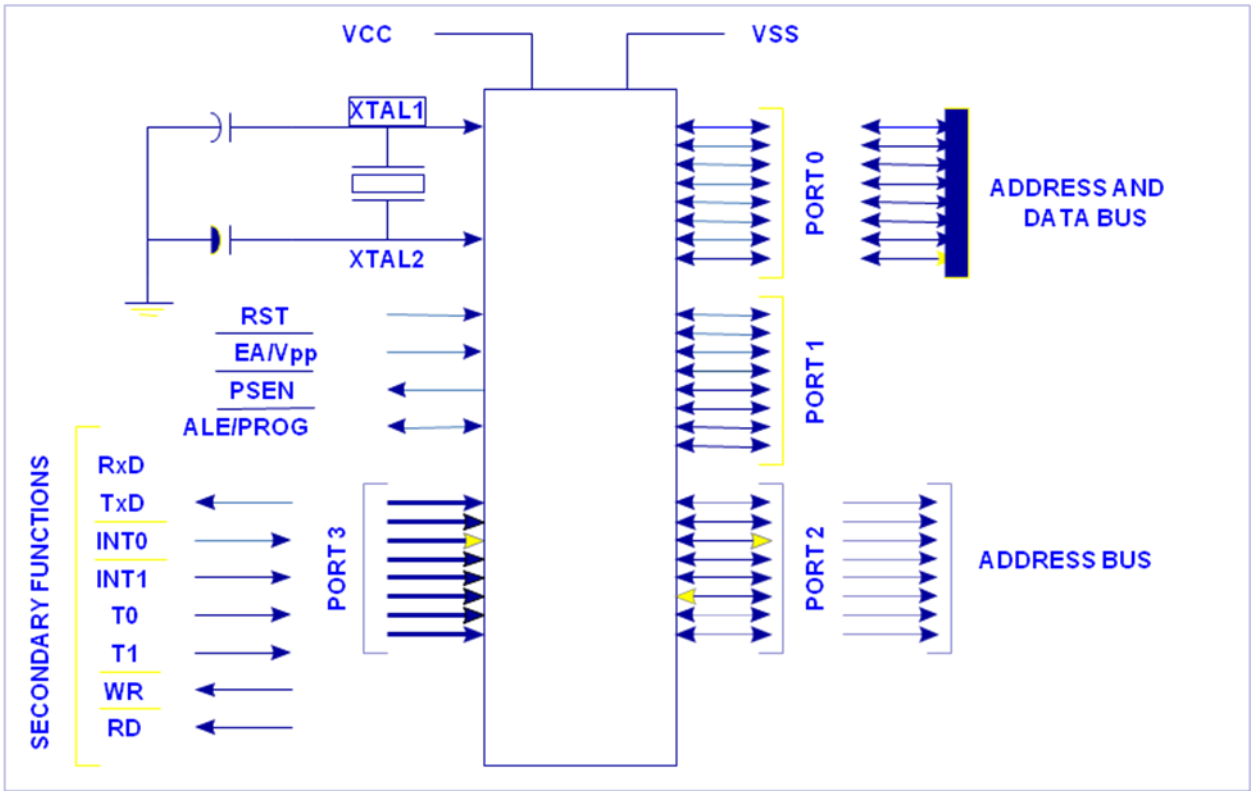


Fig.9 Pin configuration of 8051

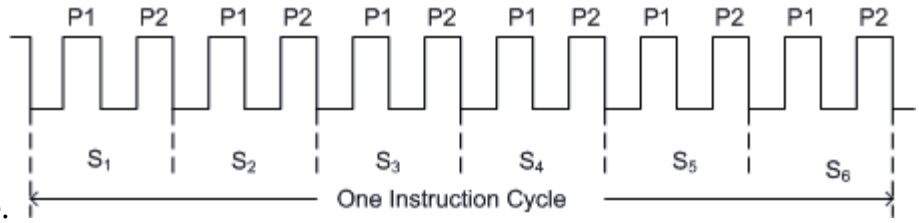
8051 Clock and Instruction Cycle:

The heart of 8051 is the circuitry that generates the clock pulses by which all internal operations are synchronised. Pins XTAL1 and XTAL2 are provided for connecting resonator to form an oscillator. The crystal frequency is the basic internal frequency of the microcontroller. 8051 is designed to operate between 1MHz to 16MHz and generally operates with a crystal frequency 11.04962 MHz.

The oscillator formed by the crystal , capacitor and an on-chip inverter generates a pulse train at the frequency of the crystal. The clock frequency *f* establishes the smallest interval to accomplish any simple instruction. The time taken to complete any instruction is called as machine cycle or instruction cycle. In 8051 one instruction cycle consists of 6 states or 12 clock cycles, instruction cycle is also referred as Machine

Microcontroller Chips :
Broad Classification of different microcontroller chips could be as follows:

- Embedded (Self -Contained) 8 - bit Microcontroller
- 16 to 32 Microcontrollers
- Digital Signal Processors



cycle.

Fig. 10 Instruction cycle of 8051(Instruction cycle has six states (S_1 - S_6). Each state has two pulses (P1 and P2))

Processor Architectures:

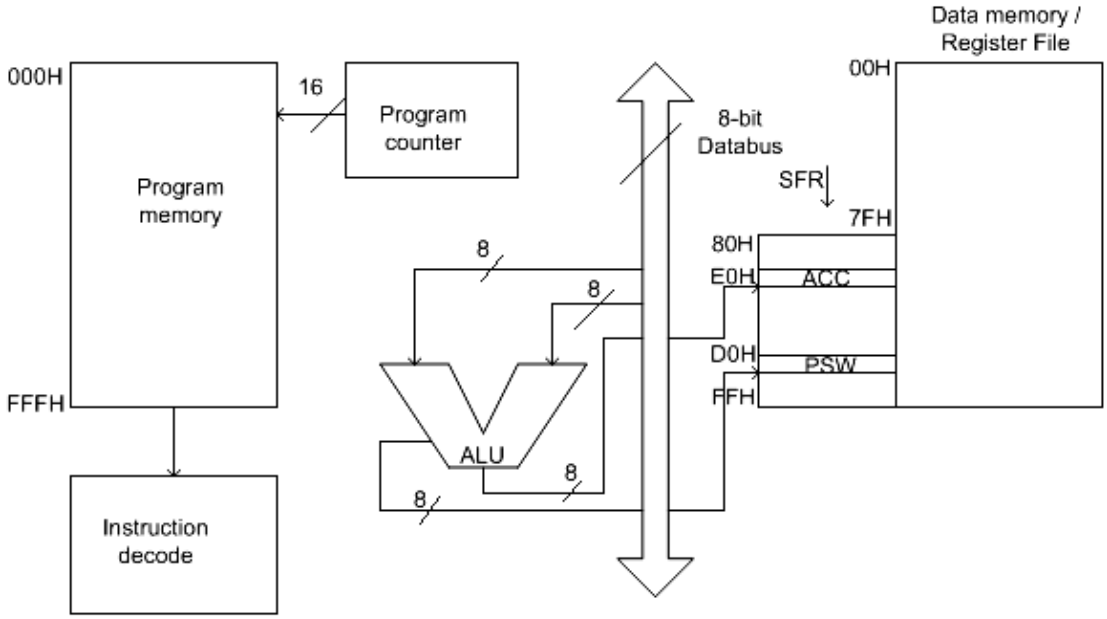


Fig 11.Basic 8051 Architecture

Internal Memory:

A functioning computer memory for program code bytes , commonly in ROM, and RAM memory for variable data that can be altered as the program runs.. Additional memory can be added externally using suitable circuits.

Unlike microcontrollers with Von- Neumann architectures, which can use a single memory address for either program code or data, but not for both, the 8051 has Harvard architecture which uses the same address in different memories for code and data The internal circuitry accesses the current memory based on the nature of operation in the program.

Internal RAM: The 128 bytes internal RAM is organized into 3 distinct areas.

1. 32 bytes from address 00h to 1fh that make up 32 working registers organized as 4 memory banks of 8 registers each. The 4 register banks are numbered 0 to 3 and are made up of 8 registers named R0 to R7. Each register can be addressed by name or by its RAM addresses. Thus R0 of bank3 is R0 (if bank3 is selected) or address 18h (where bank3 is selected). Bits RS0 and RS1 in the PSW determine which bank

of registers is currently in use at any time when program is running. Register banks not selected can be used as general purpose RAM. Bank0 is selected by default on reset..

2. A bit addressable area of 16 bytes occupies RAM byte addresses 20h to 2fh, forming total of 128 bits. An addressable bit may be specified by its bit address of 00h to 7fh or 8 bits may form any byte address from 20h to 2fh.For example bit address 4fh is also bit 7 of byte address 29h. Addressable bits are useful when the program need only remember a binary event.
3. A general purpose RAM area above the bit area from 30h to 7f h, addressable as byte.

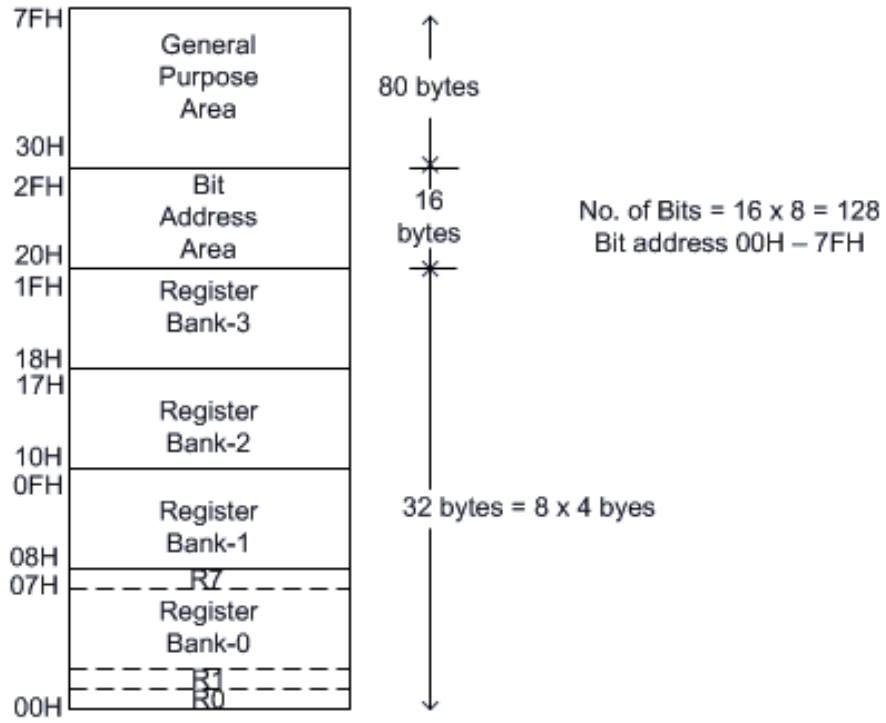


Fig.12. Internal RAM structure

The Stack and Stack pointer:

The stack refers to an area of internal RAM that is used in conjunction with certain opcodes to store and retrieve data quickly. The 8 bit Stack Pointer (SP) register is used by the 8051 to hold internal RAM address that is called the top of the stack. The address in SP register is the location in internal RAM where the last byte of the data was stored by stack operation.

When data is to be placed on the stack , the SP increments before storing data on the stack so that the stack grows up as data is stored. Whenever data is retrieved from the stack, the byte is read from the stack and then the SP decrements to point to the next available byte of stored data.

Operation of the Stack and Stack Pointer: Operation of the stack is shown in the above figure. The SP is set to 07 when the 8051 is reset and can be changed to any internal RAM address by the programmer. The stack is limited in height to the size of internal RAM. The stack can overwrite valuable data in register banks, bit addressable RAM and scratched pad RAM areas.It is programmer's responsibility to make it sure

that the stack does not grow beyond predefined bounds. The stack is normally placed high in the internal RAM by an appropriate choice of the number placed in SP register, to avoid conflict with registers or RAM.

Special Function Registers (SFRs):

The 8051 operations that do not use the internal RAM addresses from 00h to 7fh are done by a group of specific internal registers each called a specific function register (SFR) which may be addressed much like internal RAM using addresses from 80h to ffh.

Some SFRs are also bit addressable as is the case for the bit area of RAM. This feature allows the programmer the programmer to change only what needs to be altered leaving the remaining bits in that SFR unchanged. Not all of the addresses from 80h to ffh are used for SFRs . Only the addressed ones can be used in programming SFRs and equivalent internal RAM addresses are shown in Fig.10.
SFR Map: The set of Special Function Registers (SFRs) contain important registers such as Accumulator, Register B, I/O Port latch registers, Stack pointer, Data Pointer, Processor Status Word (PSW) and various control registers. Some of these registers are bit addressable (they are marked with a * in the Fig. 13 below). The detailed map of various registers is shown in the following figure.

The PC is not part of the SFR 0e0h or 8ch. and has no internal RAM address. SFRs are named in certain opcodes by their function names as A, TH0 and can also be referred by their addresses such as

Address

F8H								
F0H	B*							
E8H								
E0H	ACC*							
D8H								
D0H	PSW*							
C8H	(T2CON)*		(RCAP2L)	(RCAP2H)	(TL2)	(TH2)		
C0H								
B8H	IP*							
B0H	P3*							
A8H	IE*							
A0H	P2*							
98H	SCON*	SBUF						
90H	P1*							
88H	TCON*	TMOD	TL0	TL1	TH0	TH1		
80H	P0*	SP	DPL	DPH				PCON

Fig.13 Special Function Registers and the addresses

Internal ROM

8051 is organized so that data memory and program code memory can be two entirely different physical memory entities. Each has the same address ranges. The internal program ROM occupies code address space 000h to 0fffh. The PC is normally used to address program code bytes from address 0000h to ffffh. Program addresses higher than offfh which exceed the internal ROM capacity will cause the 8051 to automatically fetch code bytes from external memory, addresses 00h to ffffh by connecting the external access pin (EA) to ground.

I/O Port pins, Ports and Circuits: One major feature of a microcontroller is versatility built into the I/O circuits that connect the 8051 to the outside world. Out of 40 pins 24 pins may each be used for one of two entirely different functions yielding a total pin configuration of 64.But the port pins have been multiplexed to perform different functions to make 8051 as 40 Pin IC.

The port pin circuitry is as shown below.

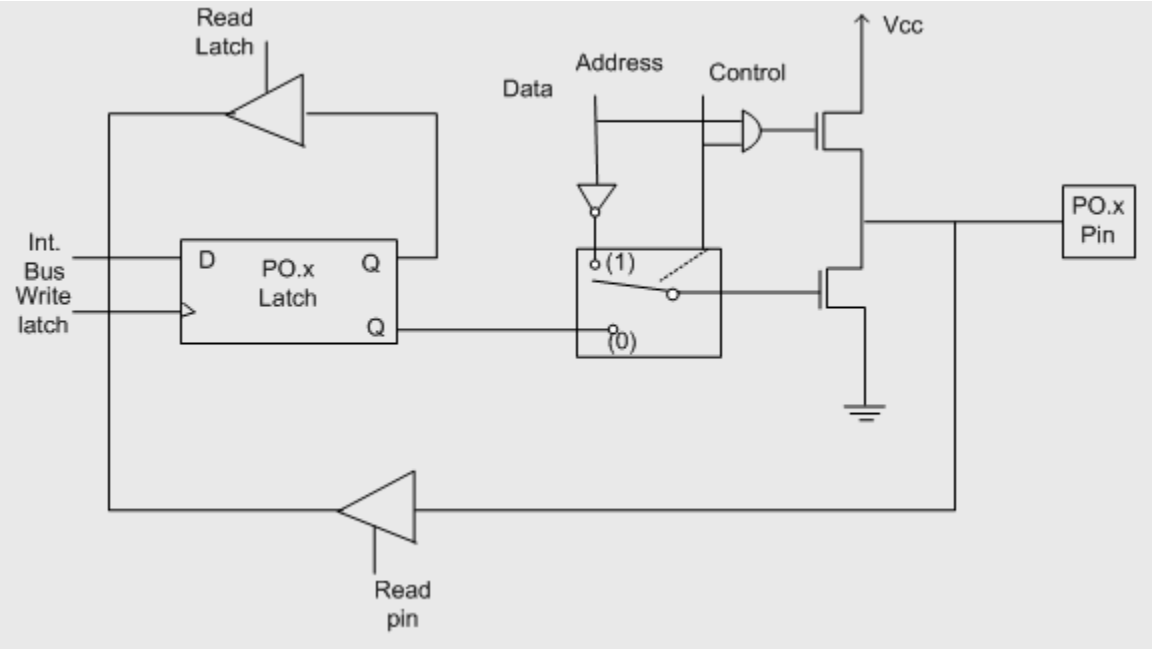


Fig. 14 Port -0

Port -0 has 8 pins (P0.0-P0.7).The structure of a Port-0 pin is shown in fig 13.Port-0 can be configured as a normal bidirectional I/O port or it can be used for address/data interfacing for accessing external memory. When control is '1', the port is used for address/data interfacing. When the control is '0', the port can be used as a normal bidirectional I/O port.

Let us assume that control is '0'. When the port is used as an input port, '1' is written to the latch. In this situation both the output MOSFETs are 'off'. Hence the output pin floats. This high impedance pin can be pulled up or low by an external source. When the port is used as an output port, a '1' written to the latch again turns 'off' both the output MOSFETs and causes the output pin to float. An external pull-up is required to output a '1'. But when '0' is written to the latch, the pin is pulled down by the lower MOSFET. Hence the output becomes zero.

When the control is '1', address/data bus controls the output driver MOSFETs. If the address/data bus (internal) is '0', the upper MOSFET is 'off' and the lower MOSFET is 'on'. The output becomes '0'. If the address/data bus is '1', the upper transistor is 'on' and the lower transistor is 'off'. Hence the output is '1'. Hence for normal address/data interfacing (for external memory access) no pull-up resistors are required. Port-0 latch is written to with 1's when used for external memory access.

Port-1 Pin Structure:

Port-1 has 8 pins (P1.1-P1.7) .The structure of a port-1 pin is shown in fig 15

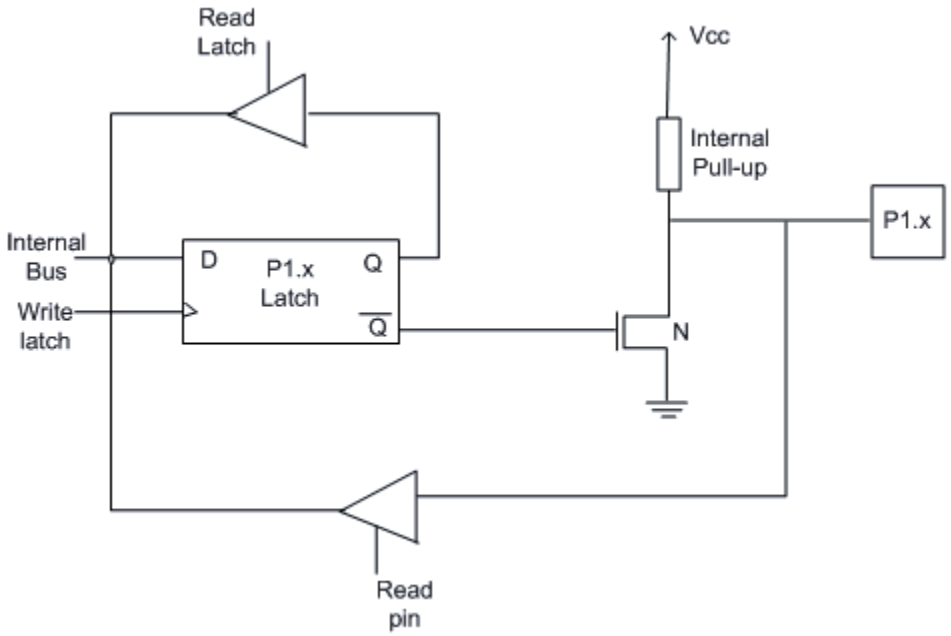


Fig 15. Port 1 Structure

Port-1 does not have any alternate function i.e. it is dedicated solely for I/O interfacing. When used as output port, the pin is pulled up or down through internal pull-up. To use port-1 as input port, '1' has to be written to the latch. In this input mode when '1' is written to the pin by the external device then it reads fine. But when '0' is written to the pin by the external device then the external source must sink current due to internal pull-up. If the external device is not able to sink the current the pin voltage may rise, leading to a possible wrong reading.

Port-2 Pin Structure:

Port-2 has 8-pins (P2.0-P2.7) . The structure of a port-2 pin is shown in fig 14.

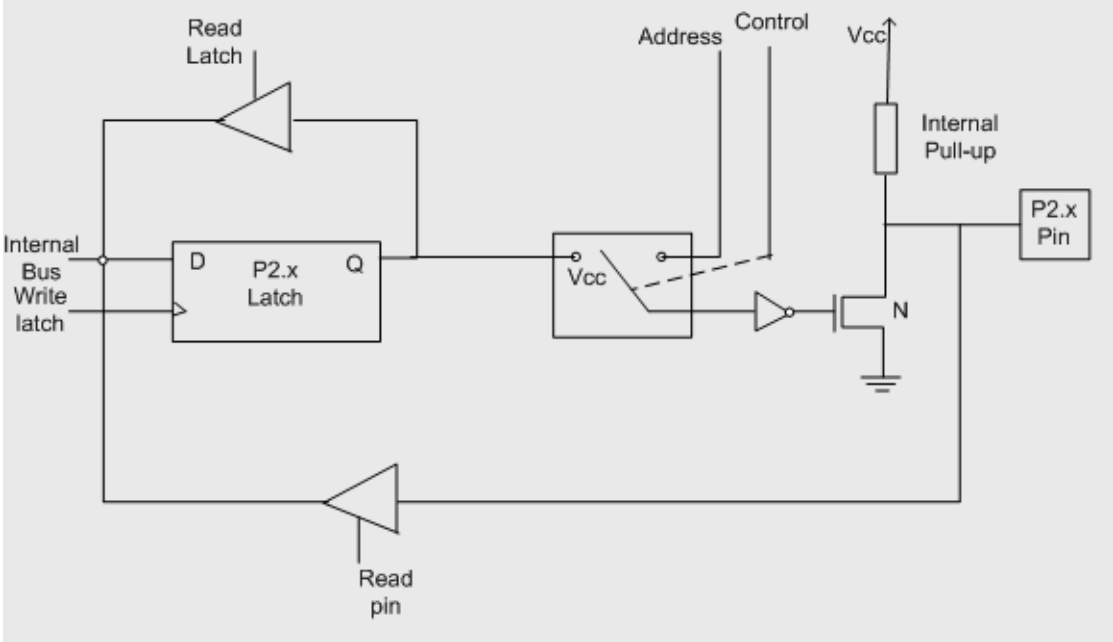


Fig. 16.PORT 2 Pin Structure

Port-2 is used for higher external address byte or a normal input/output port. The I/O operation is similar to Port-1. Port-2 latch remains stable when Port-2 pin are used for external memory access. Here again due to internal pull-up there is limited current driving capability.

Port-3 Pin Structure:

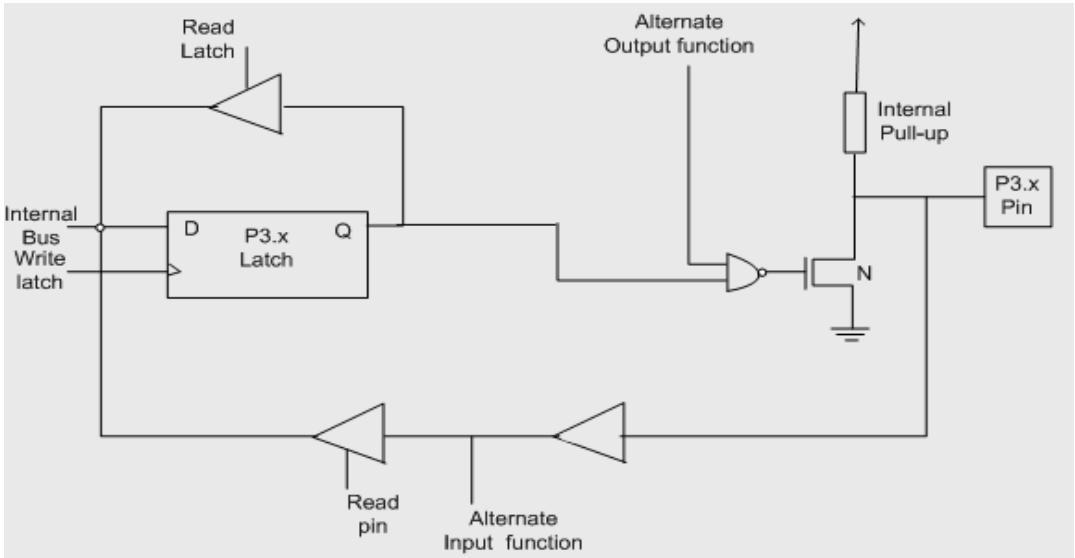


Fig. 17.PORT 3 Pin Structure:

Each pin of Port-3 can be individually programmed for I/O operation or for alternate function. The alternate function can be activated only if the corresponding latch has been written to '1'. To use the port as input port, '1' should be written to the latch. This port also has internal pull-up and limited current driving capability.

Alternate functions of Port-3 pins –

P3.0	RxD
P3.1	TxD
P3.2	$\overline{INT0}$
P3.3	$\overline{INT1}$
P3.4	T0
P3.5	T1
P3.6	$\overline{WR}$
P3.7	$\overline{RD}$

Note:

1. Port 1, 2, 3 each can drive 4 LS TTL inputs.
2. Port-0 can drive 8 LS TTL inputs in address /data mode. For digital output port, it needs external pull-up resistors.
3. Ports-1,2and 3 pins can also be driven by open-collector or open-drain outputs.

Each Port 3 bit can be configured either as a normal I/O or as a special function bit. Reading a port (port-pins) versus reading a latch. There is a subtle difference between reading a latch and reading the output port pin.

The status of the output port pin is sometimes dependant on the connected load. For instance if a port is configured as an output port and a '1' is written to the latch, the output pin should also show '1'. If the output is used to drive the base of a transistor, the transistor turns 'on'. If the port pin is read, the value will be '0' which is corresponding to the base-emitter voltage of the transistor. *Reading a latch:* Usually the instructions that read the latch, read a value, possibly change it, and then rewrite it to the latch. These are called "read-modify-write" instructions. Examples of a few instructions are-

ORL P2, A; P2 <-- P2 or A

MOV P2.1, C; Move carry bit to PX.Y bit

In this the latch value of P2 is read, is modified such that P2.1 is the same as Carry and is then written back to P2 latch.

Reading a Pin: Examples of a few instructions that read port pin, are-

MOV A, P0; Move port-0 pin values to A

MOV A, P1; Move port-1 pin values to A

Connecting External Memory: The following figure shows the connection between an 8051 and external memory

Interfacing External Memory: The system designer is not limited by the amount of internal ROM and RAM available on chip. Two separate external memory spaces are made available by the 16 bit Program Counter PC and DPTR and by different control pins for enabling the external ROM and RAM chips.

Internal control entry accesses the correct physical memory , depending on the machine cycle state and opcode being executed . There are several reasons for adding external memory, particularly Program Memory, when applying the 8051 in a system. When project is in the prototype stage, having a masked internal ROM for each program "try" is prohibitive. To help the programmer the manufacturers make available an EPROM version, the 8751, which has 4K of on-chip EPROM that may be programmed and erased as needed as the program is developed

If external program/data memory are to be interfaced, they are interfaced in the following way.

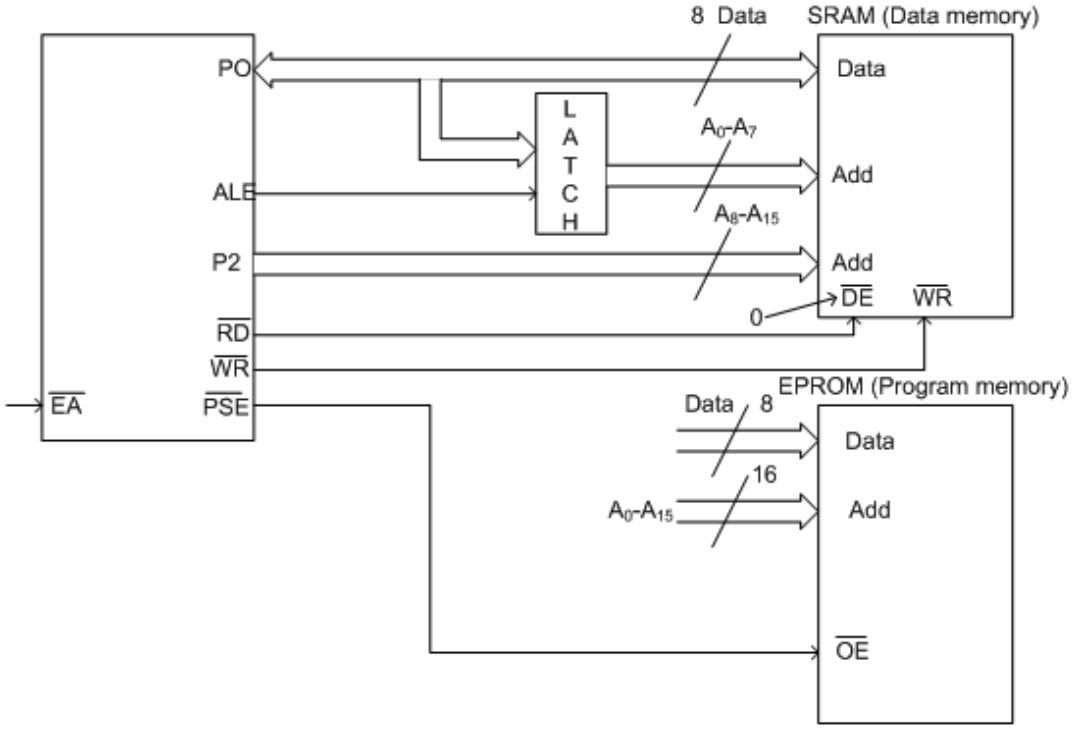


Fig.18.Diagram for Interfacing of External Memory

External program memory is fetched if either of the following two conditions are satisfied. External program memory is fetched if either of the following two conditions are satisfied.

1. $\overline{EA}$ Enable Address) is low. The microcontroller by default starts searching for program from external program memory.
2. PC is higher than FFFH for 8051 or 1FFFH for 8052.
3. $\overline{PSEN}$ tells the outside world whether the external memory fetched is program memory or data memory. $\overline{EA}$ is user configurable. $\overline{PSEN}$ is processor controlled.

Accessing external memory: Access to external program memory uses the signal $\overline{PSEN}$ (Program store enable) as the read strobe. Access to external data memory uses $\overline{RD}$ or $\overline{WR}$ (alternate function of P3.7 and P3.6).

For external program memory, always 16 bit address is used. For example –Access to external data memory can be either 8-bit address or 16-bit address - *8-bit address*- MOVX A, @Rp where Rp is either R0 or R1
MOVX @Rp, A
16 bit address- MOVX A,@DPTR

MOV X @DPTR, A.The external memory access in 8051 can be shown by a schematic diagram as given in fig 19.

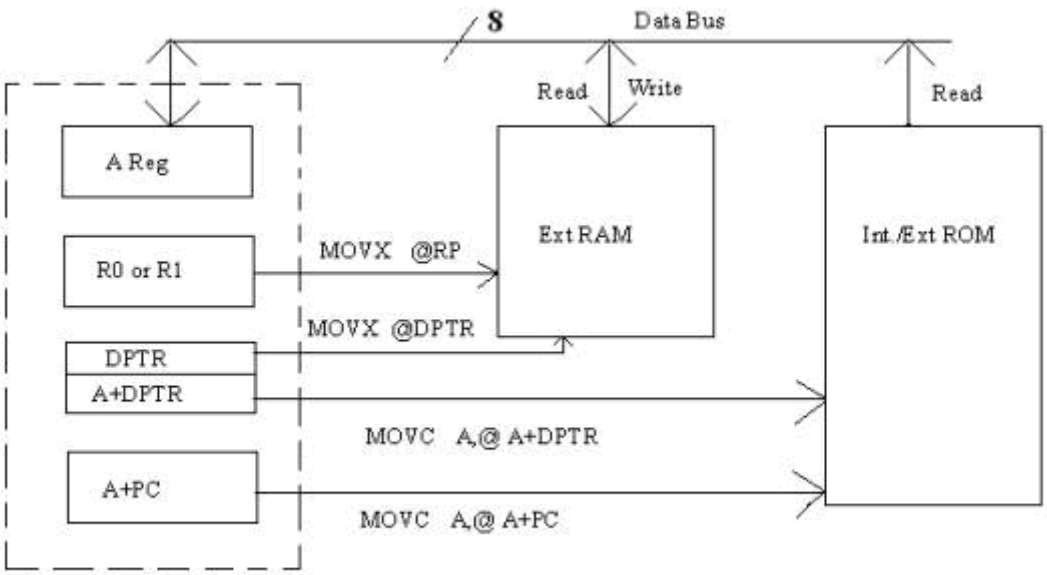


Fig 19. Schematic diagram of external memory access

If an 8-bit external address is used for data memory (i.e. MOVX @Rp) then the content of Port-2 SFR remains at Port-2 pins throughout the external memory cycle. This facilitates memory paging as the upper 8 bit address remains fixed.

During any access to external memory, the CPU writes FFH to Port-0 latch (SFR). If the user writes to Port-0 during an external memory fetch, the incoming byte is corrupted.

External program memory is accessed under the following condition.

1. Whenever $\overline{EA}$ is low, or whenever PC contains a number higher than 0FFFH (for 8051) or 1FFF (for 8052).

Some typical use of code/program memory access: External program memory can be not only used to store the code, but also for lookup table of various functions required for a particular application. Mathematical

functions such as Sine, Square root, Exponential, etc. can be stored in the program memory (Internal or external) and these functions can be accessed using MOVC instruction.

Timers / Counters :

8051 has two 16-bit programmable UP timers/counters. They can be configured to operate either as timers or as event counters. The names of the two counters are T0 and T1 respectively. The timer content is available in four 8-bit special function registers, viz, TL0,TH0,TL1 and TH1 respectively.

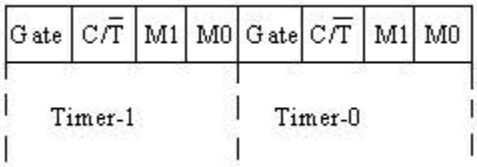
In the "timer" function mode, the counter is incremented in every machine cycle. Thus, one can think of it as counting machine cycles. Hence the clock rate is 1/12 th of the oscillator frequency.

In the "counter" function mode, the register is incremented in response to a 1 to 0 transition at its corresponding external input pin (T0 or T1). It requires 2 machine cycles to detect a high to low transition. Hence maximum count rate is 1/24 th of oscillator frequency.

The operation of the timers/counters is controlled by two special function registers, TMOD and TCON respectively.

Timer Mode control (TMOD) Special Function Register:

TMOD register is not bit addressable.
TMOD Address: 89 H



Various bits of TMOD are described as follows –**Gate:** This is an OR Gate enabled bit which controls the effect of $\overline{INT1/0}$ on START/STOP of Timer. It is set to one ('1') by the program to enable the interrupt to start/stop the timer. If TR1/0 in TCON is set and signal on $\overline{INT1/0}$ pin is high then the timer starts counting using either internal clock (timer mode) or external pulses (counter mode).

C/$\overline{T}$: It is used for the selection of Counter/Timer mode.
Mode Select Bits:

M1	M0	Mode
0	0	Mode 0
0	1	Mode 1
1	0	Mode 2
1	1	Mode 3

M1 and M0 are mode select bits.

Timer/ Counter control logic:

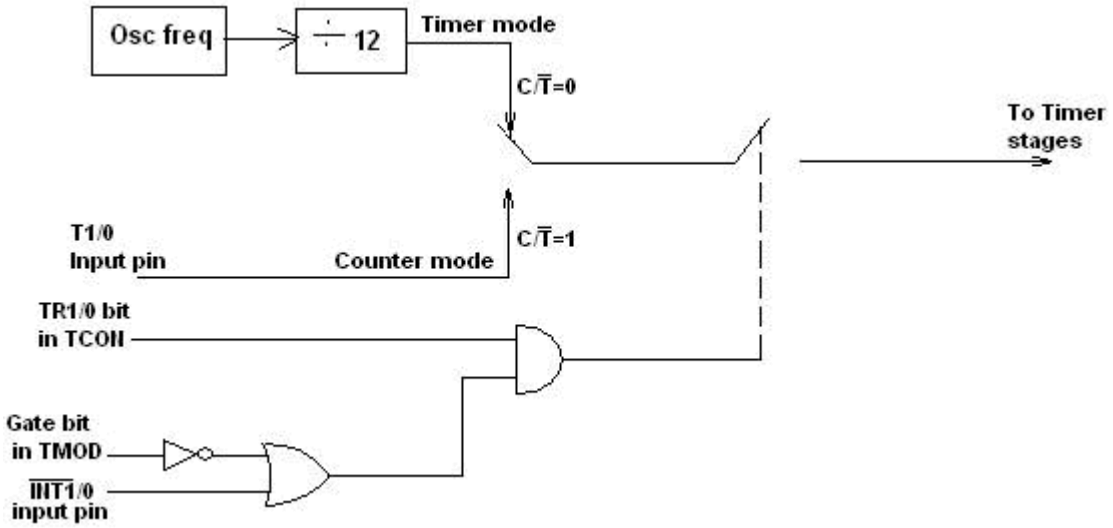


Fig .20. Timer/Counter Control Logic

Timer control (TCON) Special function register:

TCON is bit addressable. The address of TCON is 88H. It is partly related to Timer and partly to interrupt.

7	6	5	4	3	2	1	0
TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
← Timer →				← Interrupt →			

Fig. 20. TCON Register

The various bits of TCON are as follows. TF1 : Timer1 overflow flag. It is set when timer rolls from all 1s to 0s. It is cleared when processor vectors to execute ISR located at address 001BH.

TR1:Timer1 run control bit. Set to1tostartthe timer / counter.
TF0:Timer0overflowflag.(SimilartoTF1)TR0:Timer0 run control bit.
IE1 : Interrupt1 edge flag. Set by hardware when an external interrupt edge is detected. It is
interrupt is processed.
IE0:Interrupt0edgeflag.(SimilartoIE1)
IT1 : Interrupt1 type control bit. Set/ cleared by software to specify falling edge / low level triggered external interrupt.
IT0 : Interrupt0 type control bit. (Similar to IT1)
As mentioned earlier, Timers can operate in four different modes. They are as follows

Timer Mode-0:

In this mode, the timer is used as a 13-bit UP counter as follows.

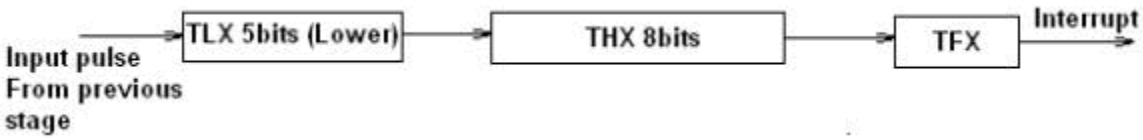


Fig. 21. Operation of Timer on Mode-0

The lower 5 bits of TLX and 8 bits of THX are used for the 13 bit count.Upper 3 bits of TLX are ignored. When the counter rolls over from all 0's to all 1's, TFX flag is set and an interrupt is generated.

The input pulse is obtained from the previous stage. If TR1/0 bit is 1 and Gate bit is 0, the counter continues counting up. If TR1/0 bit is 1 and Gate bit is 1, then the operation of the counter is controlled by $\overline{INTX}$input. This mode is useful to measure the width of a given pulse fed to $\overline{INTX}$ input.

Timer Mode-1:

This mode is similar to mode-0 except for the fact that the Timer operates in 16-bit mode.

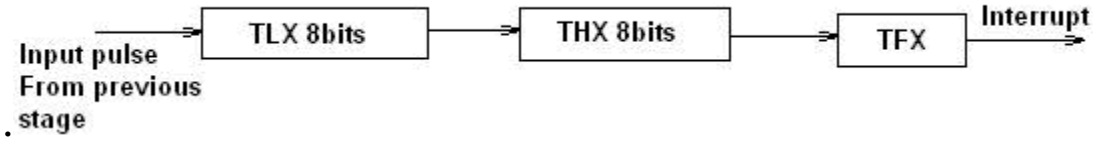


Fig .22of Timer in Mode 1

Timer Mode-2: (Auto-Reload Mode)

This is a 8 bit counter/timer operation. Counting is performed in TLX while THX stores a constant value. In this mode when the timer overflows i.e. TLX becomes FFH, it is fed with the value stored in THX. For example if we load THX with 50H then the timer in mode 2 will count from 50H to FFH. After that 50H is again reloaded. This mode is useful in applications like fixed time sampling.

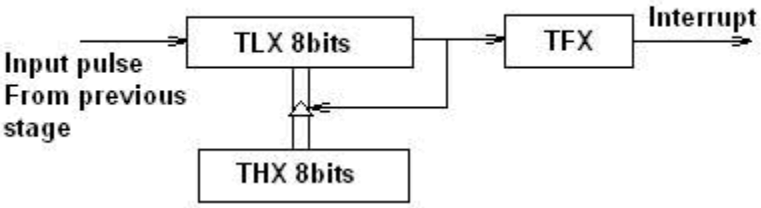


Fig .23. Operation of Timer in Mode 2

Timer Mode-3:

Timer 1 in mode-3 simply holds its count. The effect is same as setting TR1=0. Timer0 in mode-3 establishes TL0 and TH0 as two separate counters.

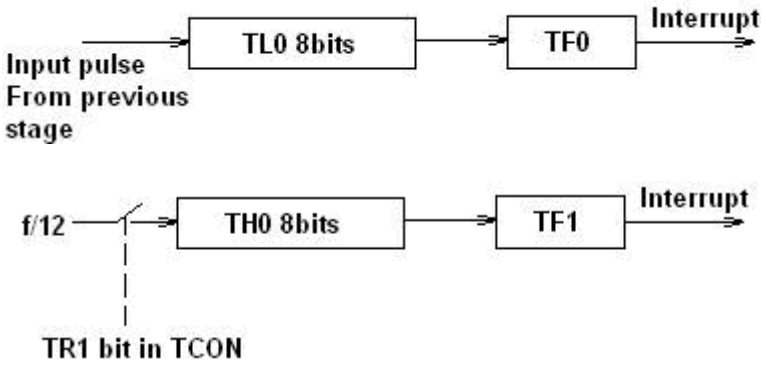


Fig. 24. Operation of Timer in Mode 3

Control bits TR1 and TF1 are used by Timer-0 (higher 8 bits) (TH0) in Mode-3 while TR0 and TF0 are available to Timer-0 lower 8 bits(TL0).

Serial Interface

The serial port of 8051 is full duplex, i.e., it can transmit and receive simultaneously. The register SBUF is used to hold the data. The special function register SBUF is physically two registers. One is, write-only and is used to hold data to be transmitted out of the 8051 via TXD. The other is, read-only and holds the received data from external sources via RXD. Both mutually exclusive registers have the same address 099H.

Serial Port Control Register (SCON)

Register SCON controls serial data communication.
Address: 098H (Bit addressable)

SM0	SM1	SM2	REN	TB8	RB8	TI	RI

Mode select bits

SM0	SM1	Mode
0	0	Mode 0
0	1	Mode 1
1	0	Mode 2
1	1	Mode 3

SM2:multi processor communication bit
REN: Receive enable bit
TB8: Transmitted bit 8 (Normally we have 0-7 bits transmitted/received)
RB8: Received bit 8
TI: Transmit interrupt flag
RI: Receive interrupt flag

Power Mode control Register

Register PCON controls processor powerdown, sleep modes and serial data bandrate. Only one bit of PCON is used with respect to serial communication. The seventh bit (b7)(SMOD) is used to generate the baud rate of serial communication.

Address: 87H

b7							b0
SMOD	—	—	—	GF1	GF0	PD	IDL

SMOD: Serial baud rate modify bit
GF1: General purpose user flag bit 1
GF0: General purpose user flag bit 0
PD: Power down bit
IDL: Idle mode bit

Data Transmission :Transmission of serial data begins at any time when data is written to SBUF. Pin P3.1 (Alternate function bit TXD) is used to transmit data to the serial data network. TI is set to 1 when data has been transmitted. This signifies that SBUF is empty so that another byte can be sent

Data Reception: Reception of serial data begins if the receive enable bit is set to 1 for all modes. Pin P3.0 (Alternate function bit RXD) is used to receive data from the serial data network. Receive interrupt flag, RI, is set after the data has been received in all modes. The data gets stored in SBUF register from where it can be read

<u>Serial Data Transmission Modes:</u>

Mode-0**:** In this mode, the serial port works like a shift register and the data transmission works synchronously with a clock frequency of f_{osc} /12. Serial data is received and transmitted through RXD. 8 bits are transmitted/ received aty a time. Pin TXD outputs the shift clock pulses of frequency f_{osc} /12, which is connected to the external circuitry for synchronization. The shift frequency or baud rate is always 1/12 of the oscillator frequency

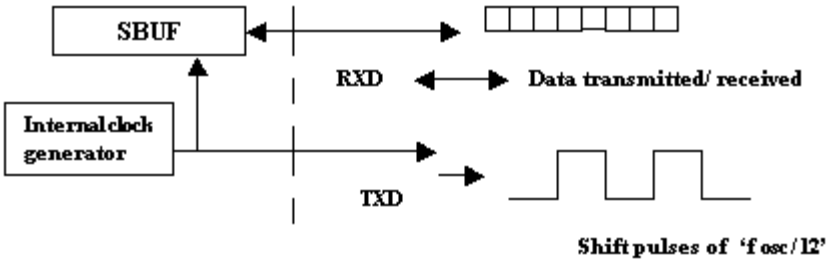


Fig .25. Data transmission/reception in Mode-0

Mode-1 (standard UART mode) :

In mode-1, the serial port functions as a standard Universal Asynchronous Receiver Transmitter (UART) mode. 10 bits are transmitted through TXD or received through RXD. The 10 bits consist of one start bit (which is

usually '0'), 8 data bits (LSB is sent first/received first), and a stop bit (which is usually '1'). Once received, the stop bit goes into RB8 in the special function register SCON. The baud rate is variable.

The following figure shows the way the bits are transmitted/ received.

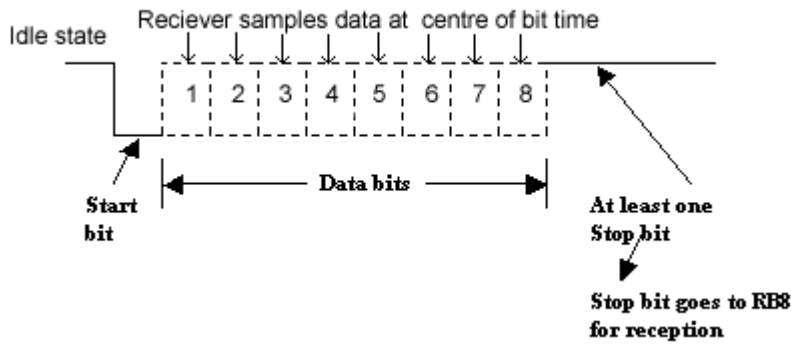


Fig .26. Data transmission format in UART mode

Bit time= $1/f_{baud}$

In receiving mode, data bits are shifted into the receiver at the programmed baud rate. The data word (8-bits) will be loaded to SBUF if the following conditions are true.

1. RI must be zero. (i.e., the previously received byte has been cleared from SBUF)

Mode bit SM2 = 0 or stop bit = 1.

After the data is received and the data byte has been loaded into SBUF, RI becomes one.

Mode-1 baud rate generation:
Timer-1 is used to generate baud rate for mode-1 serial communication by using overflow flag of the timer to determine the baud frequency. Timer-1 is used in timer mode-2 as an auto-reload 8-bit timer. The data rate is generated by timer-1 using the following formula.

$$f_{baud} = \frac{2^{SMOD}}{32} X \frac{fosc}{12 X [256-(TH1)]}$$

Where, SMOD is the 7th bit of PCON register
f_{osc} is the crystal oscillator frequency of the microcontroller
It can be noted that f_{osc}/ (12 X [256- (TH1)]) is the timer overflow frequency in timer mode-2, which is the auto-reload mode.
If timer-1 is not run in mode-2, then the baud rate is,

$$f_{baud} = \frac{2^{SMOD}}{32} X \text{ (timer-1 overflow frequency)}$$

Timer-1 can be run using the internal clock, fosc/12 (timer mode) or from any external source via pin T1 (P3.5) (Counter mode).
Example: If standard baud rate is desired, then 11.0592 MHz crystal could be selected. To get a standard 9600 baud rate, the setting of TH1 is calculated as follows.
Assuming SMOD to be '0'

$$9600 = \frac{2^0}{32} \times \frac{11.0592 \times 10^6}{12 \times (256\text{-}TH1)}$$

Or,

$$256\text{-}TH1 = \frac{1}{32} \times \frac{11.0592 \times 10^6}{12 \times 9600} = 3$$

Or,

$TH1 = 256 - 3 = 253 = FDH$

In mode-1, if SM2 is set to 1, no receive interrupt (RI) is generated unless a valid stop bit is received.

Interrupts:

8051 provides 5 vectored interrupts. They are-

1. $\overline{INT0}$
2. TF0
3. $\overline{INT1}$
4. TF1
5. RI/TI

Out of these, $\overline{INT0}$ and $\overline{INT1}$ are external interrupts whereas Timer and Serial port interrupts are generated internally. The external interrupts could be negative edge triggered or low level triggered. All these interrupt, when activated, set the corresponding interrupt flags. Except for serial interrupt, the interrupt flags are cleared when the processor branches to the Interrupt Service Routine (ISR). The external interrupt flags are cleared on branching to Interrupt Service Routine (ISR), provided the interrupt is negative edge triggered. For low level triggered external interrupt as well as for serial interrupt, the corresponding flags have to be cleared by software by the programmer.

UNIT 2:

Addressing Modes: Introduction, Instruction syntax, Data types, Subroutines, Addressing modes: Immediate addressing , Register addressing, Direct addressing, Indirect addressing, relative addressing, Absolute addressing, Long addressing, Indexed addressing, Bit inherent addressing, bit direct addressing. Instruction set: Instruction timings, 8051 instructions: Data transfer instructions, Arithmetic instructions, Logical instructions, Branch instructions, Subroutine instructions, Bit manipulation instruction.

Instruction set of 8051

1. **Data transfer instructions**

a. MOV <dest-byte>,<src-byte>-

Function: Move byte variable

Description: The byte variable indicated by the second operand is copied into the location specified by the first operand. The source byte is not affected. No other register or flag is affected.

1. mov direct , A
2. mov A, @R_i
3. mov A, R_n
4. mov direct, direct
5. mov A, #data

EX: MOV 30h, A

MOV A,@R0 ; moves the content of memory pointed to by Ro into A

MOV A, R_1; ;moves the content of Register R_1 to Accumulator A

MOV 20h,30h;moves the content of memory location 30h to 20h

MOV A,#45h;moves 45h to Accumulator A

MOV <dest-bit>,<src-bit>

Function: Move bit data

Description: MOV <dest-bit>,<src-bit> copies the Boolean variable indicated by the second operand into the location specified by the first operand. One of the operands must be the carry flag; the other may be any directly addressable bit. No other register or flag is affected.

Example: MOV P1.3,C; moves the carry bit to 3rd bit of port1

C. MOV DPTR,#data16

Function: Load Data Pointer with a 16-bit constant

Description: MOV DPTR,#data16 loads the Data Pointer with the 16-bit constant indicated. The 16-bit constant is loaded into the second and third bytes of the instruction. The second byte (DPH) is the high-order byte, while the third byte

(DPL) holds the lower-order byte. No flags are affected.

This is the only instruction which moves 16 bits of data at once.

Example: The instruction, MOV DPTR, # 4567H

loads the value 4567H into the Data Pointer. DPH holds 45H, and DPL holds 67H.

d. MOVC A,@A+ <base-reg>

Function: Move Code byte

Description: The MOVC instructions load the Accumulator with a code byte or constant from program memory. The address of the byte fetched is the sum of the original unsigned 8-bit Accumulator contents and the contents of a 16-bit base register, which may be either the Data Pointer or the PC. In the latter case, the PC is incremented to the address of the following instruction before being added with the Accumulator; otherwise the base register is not altered. Sixteen-bit addition is performed so a carry-out from the low-order eight bits may propagate through higher-order bits. No flags are affected.

e. MOVC A,@A+PC

(PC) ← (PC) + 1

(A) ← ((A) + (PC))

f. MOVX <dest-byte>,<src-byte>

Function: Move External

Description: The MOVX instructions transfer data between the Accumulator and a byte of external data memory, which is why "X" is appended to MOV. There are two types of instructions, differing in whether they provide an 8-bit or 16-bit indirect address to the external data RAM.

In the first type, the contents of R0 or R1 in the current register bank provide an 8-bit address multiplexed with data on P0. Eight bits are sufficient for external I/O expansion decoding or for a relatively small RAM array. For somewhat larger arrays, any output port pins can be used to output higher-order address bits. These pins are controlled by an output instruction preceding the MOVX.

In the second type of MOVX instruction, the Data Pointer generates a 16-bit address. P2 outputs the high-order eight address bits (the contents of DPH), while P0 multiplexes the low-order eight bits (DPL) with data. The P2 Special Function Register retains its previous contents, while the P2 output buffers emit the contents of DPH.

This form of MOVX is faster and more efficient when accessing very large data arrays (up to 64K bytes), since no additional instructions are needed to set up the output ports.

It is possible to use both MOVX types in some situations. A large RAM array with its high-order address lines driven by P2 can be addressed via the Data Pointer, or with code to output high-order address bits to P2, followed by a MOVX instruction using R0 or R1.

Example: An external 256 byte RAM using multiplexed address/data lines is connected to the 8051 Port 0. Port 3 provides control lines for the external RAM. Ports 1 and 2 are used for normal I/O. Registers 0 and 1 contain 12H and 34H. Location 34H of the external RAM holds the value 56H. The instruction sequence,

MOVX A,@R1

MOVX @R0,A

copies the value 56H into both the Accumulator and external RAM location 12H.

MOVX A,@DPTR

(A) ← ((DPTR))

PUSH direct

Function: Push onto stack

Description: The Stack Pointer is incremented by one. The contents of the indicated variable is then copied into the internal RAM location addressed by the Stack Pointer. No flags are affected.

Example: On entering an interrupt routine, the Stack Pointer contains 09H. The Data Pointer holds the value 0123H. The

following instruction sequence,

PUSH DPL

PUSH DPH

leaves the Stack Pointer set to 0BH and stores 23H and 01H in internal RAM locations 0AH and 0BH,

respectively.

POP direct

Function: Pop from stack.

Description: The contents of the internal RAM location addressed by the Stack Pointer is read, and the Stack Pointer is decremented by one. The value read is then transferred to the directly addressed byte indicated. No flags are affected.

Example: The Stack Pointer originally contains the value 32H, and internal RAM locations 30H through 32H contain the values 20H, 23H, and 01H, respectively. The following instruction sequence,

POP DPH

POP DPL

leaves the Stack Pointer equal to the value 30H and sets the Data Pointer to 0123H.

2. Arithmetic Group of Instructions

a. **ADD A,<src-byte>**

Function: Add

Description: ADD adds the byte variable indicated to the Accumulator, leaving the result in the Accumulator. The carry and auxiliary-carry flags are set, respectively, if there is a carry-out from bit 7 or bit 3, and cleared otherwise. When adding unsigned integers, the carry flag indicates an overflow occurred.

OV is set if there is a carry-out of bit 6 but not out of bit 7, or a carry-out of bit 7 but not bit 6; otherwise, OV is cleared. When adding signed integers, OV indicates a negative number produced as the sum of two positive operands, or a positive sum from two negative operands.

Four source operand addressing modes are allowed: register, direct, register-indirect, or immediate.

Example: The Accumulator holds 0C3H (1100001lB), and register 0 holds 0AAH (10101010B). The following instruction,

ADD A,R0

leaves 6DH (01101101B) in the Accumulator with the AC flag cleared and both the carry flag and OV set to 1.

ADD A, direct

(A) ← □ □ □ □ □ □ □ □ □ □ (A) + (direct)

ADD A, @Ri

(A) ← (A) + data

ADDC A, <src-byte>

Function: Add with Carry

Description: ADDC simultaneously adds the byte variable indicated, the carry flag and the Accumulator contents, leaving the result in the Accumulator. The carry and auxiliary-carry flags are set respectively, if there is a carry-out from bit 7 or bit 3, and cleared otherwise. When adding unsigned integers, the carry flag indicates an overflow occurred.

OV is set if there is a carry-out of bit 6 but not out of bit 7, or a carry-out of bit 7 but not out of bit 6; otherwise OV is cleared. When adding signed integers, OV indicates a negative number produced as the sum of two positive operands or a positive sum from two negative operands.

Four source operand addressing modes are allowed: register, direct, register-indirect, or immediate.

Example: The Accumulator holds 0C3H (11000011B) and register 0 holds 0AAH (10101010B) with the carry flag set. The following instruction,

ADDC A,R0

leaves 6EH (01101110B) in the Accumulator with AC cleared and both the Carry flag and OV set to 1.

ADDC A,Rn Operation: ADDC

(A)←□ (A) + (C) + (Rn)

ADDC A, direct Operation: ADDC

(A) ←(A) + (C) + (direct)

ADDC A, @Ri Operation: ADDC

(A) □←□ □ □ □ □ □ (A) + (C) + ((Ri))

ADDC A, #data Operation: ADDC (A) ← (A) + (C) + #data

SUBB A,<src-byte>

Function: Subtract with borrow

Description: SUBB subtracts the indicated variable and the carry flag together from the Accumulator, leaving the result in the Accumulator. SUBB sets the carry (borrow) flag if a borrow is needed for bit 7 and clears C otherwise. (If C was set *before* executing a SUBB instruction, this indicates that a borrow was needed for the previous step in a

multiple-precision subtraction, so the carry is subtracted from the Accumulator along with the source operand.)

AC is set if a borrow is needed for bit 3 and cleared otherwise. OV is set if a borrow is needed into bit 6, but not into bit 7, or into bit 7, but not bit 6.

When subtracting signed integers, OV indicates a negative number produced when a negative value is subtracted from a positive value, or a positive result when a positive number is subtracted from a negative number.

The source operand allows four addressing modes: register, direct, register-indirect, or immediate.

Example: The Accumulator holds 0C9H (11001001B), register 2 holds 54H (01010100B), and the carry flag is set. The instruction,

SUBB A,R2

will leave the value 74H (01110100B) in the accumulator, with the carry flag and AC cleared but OV set.

Instructions	**OpCode**	**Bytes**	**Flags**
SUBB A,*#data*	0x94	2	C, AC, OV
SUBB A,*iram addr*	0x95	2	C, AC, OV
SUBB A,@R0	0x96	1	C, AC, OV

SUBB A,@R1	0x97	1	C, AC, OV
SUBB A,R0	0x98	1	C, AC, OV
SUBB A,R1	0x99	1	C, AC, OV
SUBB A,R2	0x9A	1	C, AC, OV
SUBB A,R3	0x9B	1	C, AC, OV
SUBB A,R4	0x9C	1	C, AC, OV
SUBB A,R5	0x9D	1	C, AC, OV
SUBB A,R6	0x9E	1	C, AC, OV
SUBB A,R7	0x9F	1	C, AC, OV

SUBB A,Rn

Operation: SUBB

(A) ← (A) - (C) - (Rn)

SUBB A, direct

Operation: SUBB

(A) ← (A) - (C) - (direct)

SUBB A,@Ri

Operation: SUBB

(A) ← (A) - (C) - ((Ri))

SWAP A

Function: Swap nibbles within the Accumulator

Description: SWAP A interchanges the low- and high-order nibbles (four-bit fields) of the Accumulator (bits 3 through 0 and bits 7 through 4). The operation can also be thought of as a 4-bit rotate instruction. No flags are affected.

Example: The Accumulator holds the value 0C5H (11000101B). The instruction,

SWAP A

leaves the Accumulator holding the value 5CH (01011100B

Operation: SWAP

(A3-0) D (A7-4)

XCH A,<byte>

Function: Exchange Accumulator with byte variable

Description: XCH loads the Accumulator with the contents of the indicated variable, at the same time writing the original Accumulator contents to the indicated variable. The source/destination operand can use register, direct, or register-indirect addressing.

Example: R0 contains the address 20H. The Accumulator holds the value 3FH (0011111lB). Internal RAM location 20H holds the value 75H (01110101B).

The following instruction,

XCH A,@R0

leaves RAM location 20H holding the values 3FH (00111111B) and 75H (01110101B) in the accumulator.

XCHD A,@Ri

Function: Exchange Digit

Description: XCHD exchanges the low-order nibble of the Accumulator (bits 3 through 0), generally representing a hexadecimal or BCD digit, with that of the internal RAM location indirectly addressed by the specified register.

The high-order nibbles (bits 7-4) of each register are not affected. No flags are affected.

Example: R0 contains the address 20H. The Accumulator holds the value 36H (00110110B). Internal RAM location 20H holds the value 75H (01110101B).

The following instruction,

XCHD A,@R0

leaves RAM location 20H holding the value 76H (01110110B) and 35H (00110101B) in the Accumulator.

CPL A

Function: Complement Accumulator

Description: CPLA logically complements each bit of the Accumulator (one’s complement). Bits which previously contained a 1 are changed to a 0 and vice-versa. No flags are affected.

Example: The Accumulator contains 5CH (01011100B).

The following instruction,

CPL A

leaves the Accumulator set to 0A3H (10100011B).

CPL bit

Function: Complement bit

Description: CPL bit complements the bit variable specified. A bit that had been a 1 is changed to 0 and vice-versa. No other flags are affected. CLR can operate on the carry or any directly addressable bit.

Example: Port 1 has previously been written with 5BH (01011101B). The following instruction sequence, CPL P1.1CPL

P1.2 leaves the port set to 5BH (01011011B).

DA A

Function: Decimal-adjust Accumulator for Addition

Description: DA A adjusts the eight-bit value in the Accumulator resulting from the earlier addition of two variables (each in packed-BCD format), producing two four-bit digits. Any ADD or ADDC instruction may have been used to perform the addition.

If Accumulator bits 3 through 0 are greater than nine or if the AC flag is one, six is added to the Accumulator producing the proper BCD digit in the low-order nibble. This internal addition sets the carry flag if a carry-out of the low-order four-bit field propagates through all high-order bits, but it does not clear the carry flag otherwise.

If the carry flag is now set, or if the four high-order bits now exceed nine, these high-order bits are incremented by six, producing the proper BCD digit in the high-order nibble. Again, this sets the carry flag if there is a carry-out of the high-order bits, but does not clear the carry. The carry flag thus indicates if the sum of the original two BCD variables is greater than 100, allowing multiple precision decimal addition. OV is not affected.

DEC byte

Function: Decrement

Description: DEC byte decrements the variable indicated by 1. An original value of 00H underflows to 0FFH. No flags are affected.

Example: Register 0 contains 7FH (01111111B). Internal RAM locations 7EH and 7FH contain 00H and 40H, respectively.

The following instruction sequence,

DEC @R0

DEC R0

DEC @R0

leaves register 0 set to 7EH and internal RAM locations 7EH and 7FH set to 0FFH and 3FH.

DEC A

DEC Rn

DEC direct

DEC @Ri

DIV AB

Function: Divide

Description: DIV AB divides the unsigned eight-bit integer in the Accumulator by the unsigned eight-bit integer in register B.

The Accumulator receives the integer part of the quotient; register B receives the integer remainder. The carry and OV flags are cleared.

Exception: if B had originally contained 00H, the values returned in the Accumulator and B-register are undefined and the overflow flag are set. The carry flag is cleared in any case.

Example: The Accumulator contains 251 (0FBH or 11111011B) and B contains 18 (12H or 00010010B). The following instruction,

DIV AB

leaves 13 in the Accumulator (0DH or 00001101B) and the value 17 (11H or 00010001B) in B, since

251 = (13 x 18) + 17. Carry and OV are both cleared.

INC <byte>

Function: Increment

Description: INC increments the indicated variable by 1. An original value of 0FFH overflows to 00H. No flags are affected.

Example: Register 0 contains 7EH (011111110B). Internal RAM locations 7EH and 7FH contain 0FFH and 40H,

respectively. The following instruction sequence,

INC @R0

INC R0

INC @R0

leaves register 0 set to 7FH and internal RAM locations 7EH and 7FH holding 00H and 41H, respectively.

INC A

Operation: INC

(A) ← □ □ □ □ (A) + 1

INC DPTR

Function: Increment Data Pointer

Description: INC DPTR increments the 16-bit data pointer by 1. A 16-bit increment (modulo 216) is performed, and an overflow of the low-order byte of the data pointer (DPL) from 0FFH to 00H increments the high-order byte (DPH).

No flags are affected.

This is the only 16-bit register which can be incremented.

Example: Registers DPH and DPL contain 12H and 0FEH, respectively. The following instruction sequence,

INC DPTR

INC DPTR

INC DPTR

changes DPH and DPL to 13H and 01H.

MUL AB

Function: Multiply

Description: MUL AB multiplies the unsigned 8-bit integers in the Accumulator and register B. The low-order byte of the 16-bit product is left in the Accumulator, and the high-order byte in B. If the product is greater than 255 (0FFH), the overflow flag is set; otherwise it is cleared. The carry flag is always cleared.

Example: Originally the Accumulator holds the value 80 (50H). Register B holds the value 160 (0A0H). The instruction,

MUL AB

will give the product 12,800 (3200H), so B is changed to 32H (00110010B) and the Accumulator is cleared. The overflow flag is set, carry is cleared.

NOP

Function: No Operation

Description: Execution continues at the following instruction. Other than the PC, no registers or flags are affected.

Logical instructions

ANL <dest-byte>,<src-byte>

Function: Logical-AND for byte variables

Description: ANL performs the bitwise logical-AND operation between the variables indicated and stores the results in the destination variable. No flags are affected.

The two operands allow six addressing mode combinations. When the destination is the Accumulator, the source can use register, direct, register-indirect, or immediate addressing; when the destination is a direct address, the source can be the Accumulator or immediate data.

.**Example:** If the Accumulator holds 0C3H (1100001lB), and register 0 holds 55H (01010101B), then the following instruction,

ANL A,R0

leaves 41H (01000001B) in the Accumulator.

When the destination is a directly addressed byte, this instruction clears combinations of bits in any RAM location or hardware register. The mask byte determining the pattern of bits to be cleared would either be a constant contained in the instruction or a value computed in the Accumulator at run-time. The following instruction,

ANL P1,#01110011B

clears bits 7, 3, and 2 of output port 1.

Instructions	OpCode	Bytes	Flags
ANL *iram addr*,A	0x52	2	None
ANL *iram addr*,#*data*	0x53	3	None
ANL A,#*data*	0x54	2	None
ANL A,*iram addr*	0x55	2	None
ANL A,@R0	0x56	1	None

ANL A,@R1	0x57	1	None
ANL A,R0	0x58	1	None
ANL A,R1	0x59	1	None
ANL A,R2	0x5A	1	None
ANL A,R3	0x5B	1	None
ANL A,R4	0x5C	1	None
ANL A,R5	0x5D	1	None
ANL A,R6	0x5E	1	None
ANL A,R7	0x5F	1	None
ANL C,*bit addr*	0x82	2	C
ANL C,/*bit addr*	0xB0	2	C

ANL A,Rn

Operation: ANL

(A)←(A) ^□ (Rn)

ANL A,@Ri

Operation: ANL

(A)←□ □ □ □ □ (A) ^ ((Ri))

ANL direct,#data

Operation: ANL

(direct) ←(direct) □ ^□ #data

ORL <dest-byte> <src-byte>

Function: Logical-OR for byte variables

Description: ORL performs the bitwise logical-OR operation between the indicated variables, storing the results in the destination byte. No flags are affected.

Example: If the Accumulator holds 0C3H (11000011B) and R0 holds 55H (01010101B) then the following instruction,

ORL A,R0

leaves the Accumulator holding the value 0D7H (1101011lB).

The instruction,

ORL P1,#00110010B

sets bits 5, 4, and 1 of output Port 1.

ORL A, Rn ; or the content of Accumulator and Register Rn and store the result in Accumulator

ORL A, direct ; or the content of Accumulator and the memory and store the result in Accumulator

ORL A, @Ri ; or the content of accumulator and the memory location whose address is specified in Ri

ORL C,<src-bit>

Function: Logical-OR for bit variables

Description: Set the carry flag if the Boolean value is a logical 1; leave the carry in its current state otherwise. A slash (/) preceding the operand in the assembly language indicates that the logical complement of the addressed bit is used as the source value, but the source bit itself is not affected. No other flags are affected.

Example:

ORL C, ACC.7 ;OR CARRY WITH THE ACC. BIT 7

ORL C, /OV ;OR CARRY WITH THE INVERSE OF OV.

<u>SETB</u>

Operation: SETB
Function: Set Bit
Syntax: SETB *bit addr*
Description: Sets the specified bit.

XRL <dest-byte>,<src-byte>

Function: Logical Exclusive-OR for byte variables

Description: XRL performs the bitwise logical Exclusive-OR operation between the indicated variables, storing the results in the destination. No flags are affected.

The two operands allow six addressing mode combinations. When the destination is the Accumulator, the source can use register, direct, register-indirect, or immediate addressing; when the destination is a direct address, the source can be the Accumulator or immediate data.

Example: If the Accumulator holds 0C3H (1100001lB) and register 0 holds 0AAH (10101010B) then the instruction,

XRL A,R0

leaves the Accumulator holding the value 69H (01101001B).

Instructions	OpCode	Bytes	Flags
XRL *iram addr*,A	0x62	2	None
XRL *iram addr,#data*	0x63	3	None
XRL A,*#data*	0x64	2	None
XRL A,*iram addr*	0x65	2	None
XRL A,@R0	0x66	1	None
XRL A,@R1	0x67	1	None
XRL A,R0	0x68	1	None
XRL A,R1	0x69	1	None
XRL A,R2	0x6A	1	None
XRL A,R3	0x6B	1	None
XRL A,R4	0x6C	1	None
XRL A,R5	0x6D	1	None
XRL A,R6	0x6E	1	None
XRL A,R7	0x6F	1	None

<u>Rotate Instructions</u>

RL A

Function: Rotate Accumulator Left

Description: The eight bits in the Accumulator are rotated one bit to the left. Bit 7 is rotated into the bit 0 position. No flags are affected.

Example: The Accumulator holds the value 0C5H (11000101B). The following instruction,

RL A

leaves the Accumulator holding the value 8BH (10001011B) with the carry unaffected.

RLC A

Function: Rotate Accumulator Left through the Carry flag

Description: The eight bits in the Accumulator and the carry flag are together rotated one bit to the left. Bit 7 moves into the carry flag; the original state of the carry flag moves into the bit 0 position. No other flags are affected.

Example: The Accumulator holds the value 0C5H(11000101B), and the carry is zero. The following instruction,

RLC A

leaves the Accumulator holding the value 8BH (10001010B) with the carry set.

RRC A

Function: Rotate Accumulator Right through Carry flag

Description: The eight bits in the Accumulator and the carry flag are together rotated one bit to the right. Bit 0 moves into the carry flag; the original value of the carry flag moves into the bit 7 position. No other flags are affected.

Example: The Accumulator holds the value 0C5H (11000101B), the carry is zero. The following instruction,

RRC A

leaves the Accumulator holding the value 62 (01100010B) with the carry set.

3. Branch instructions

Unconditional Branch Instructions

Operation: **AJMP**
Function: Absolute Jump Within 2K Block
Syntax: AJMP *code address*

Instructions	OpCode	Bytes	Flags

AJMP *page0*	0x01	2	None
AJMP *page1*	0x21	2	None
AJMP *page2*	0x41	2	None
AJMP *page3*	0x61	2	None
AJMP *page4*	0x81	2	None
AJMP *page5*	0xA1	2	None
AJMP *page6*	0xC1	2	None
AJMP *page7*	0xE1	2	None

Description: AJMP unconditionally jumps to the indicated *code address*. The new value for the Program Counter is calculated by replacing the least-significant-byte of the Program Counter with the second byte of the AJMP instruction, and replacing bits 0-2 of the most-significant-byte of the Program Counter with 3 bits that indicate the page *of the byte following the AJMP instruction*. Bits 3-7 of the most-significant-byte of the Program Counter remain unchanged.

Since only 11 bits of the Program Counter are affected by AJMP, jumps may only be made to code located within the same 2k block as the first byte that follows AJMP.

Operation: LJMP
Function: Long Jump
Syntax: LJMP *code address.*

Description: LJMP jumps unconditionally to the specified *code address*.

Operation: SJMP
Function: Short Jump
Syntax: SJMP *reladdr*

Description: SJMP jumps unconditionally to the address specified *reladdr*. *Reladdr* must be within -128 or +127 bytes of the instruction that follows the SJMP instruction

<u>Conditional Branch Instructions</u>

Operation: JNC
Function: Jump if Carry Not Set
Syntax: JNC *reladdr*

Description: JNC branches to the address indicated by *reladdr* if the carry bit is not set. If the carry bit is set program execution continues with the instruction following the JNB instruction.

Operation: JC

Function: Jump if Carry Set
Syntax: JC *reladdr*

Description: JC will branch to the address indicated by *reladdr* if the Carry Bit is set. If the Carry Bit is not set program execution continues with the instruction following the JC instruction.

Operation: JNB
Function: Jump if Bit Not Set
Syntax: JNB *bit addr,reladdr*

Description: JNB will branch to the address indicated by *reladdress* if the indicated bit is not set. If the bit is set program execution continues with the instruction following the JNB instruction.

Operation: JB
Function: Jump if Bit Set
Syntax: JB *bit addr, reladdr*

Description: JB branches to the address indicated by *reladdr* if the bit indicated by *bit addr* is set. If the bit is not set program execution continues with the instruction following the JB instruction.

Operation: JNZ
Function: Jump if Accumulator Not Zero
Syntax: JNZ *reladdr*

Description: JNZ will branch to the address indicated by *reladdr* if the Accumulator contains any value except 0. If the value of the Accumulator is zero program execution continues with the instruction following the JNZ instruction.

Operation: JZ
Function: Jump if Accumulator Zero
Syntax: JNZ *reladdr*

Description: JZ branches to the address indicated by *reladdr* if the Accumulator contains the value 0. If the value of the Accumulator is non-zero program execution continues with the instruction following the JNZ instruction.

Operation: **DJNZ**
Function: Decrement and Jump if Not Zero
Syntax: DJNZ *register, reladdr*

Instructions	OpCode	Bytes	Flags
DJNZ *iram addr,reladdr*	0xD5	3	None

DJNZ R0,*reladdr*	0xD8	2	None
DJNZ R1,*reladdr*	0xD9	2	None
DJNZ R2,*reladdr*	0xDA	2	None
DJNZ R3,*reladdr*	0xDB	2	None
DJNZ R4,*reladdr*	0xDC	2	None
DJNZ R5,*reladdr*	0xDD	2	None
DJNZ R6,*reladdr*	0xDE	2	None
DJNZ R7,*reladdr*	0xDF	2	None

Description: DJNZ decrements the value of *register* by 1. If the initial value of *register* is 0, decrementing the value will cause it to reset to 255 (0xFF Hex). If the new value of *register* is not 0 the program will branch to the address indicated by *relative addr*. If the new value of *register* is 0 program flow continues with the instruction following the DJNZ instruction.

Operation: **CJNE**
Function: Compare and Jump If Not Equal
Syntax: CJNE *operand1*,*operand2*,*reladdr*

Instructions	OpCode	Bytes	Flags
CJNE A,*#data, reladdr*	0xB4	3	C
CJNE A,*iram addr*,*reladdr*	0xB5	3	C
CJNE @R0,*#data*,*reladdr*	0xB6	3	C
CJNE @R1,*#data*,*reladdr*	0xB7	3	C
CJNE R0,*#data*,*reladdr*	0xB8	3	C
CJNE R1,*#data*,*reladdr*	0xB9	3	C
CJNE R2,*#data*,*reladdr*	0xBA	3	C
CJNE R3,*#data*,*reladdr*	0xBB	3	C
CJNE R4,*#data*,*reladdr*	0xBC	3	C
CJNE R5,*#data*,*reladdr*	0xBD	3	C
CJNE R6,*#data*,*reladdr*	0xBE	3	C
CJNE R7,*#data*,*reladdr*	0xBF	3	C

Description: CJNE compares the value of *operand1* and *operand2* and branches to the indicated relative address if *operand1* and *operand2* are not equal. If the two operands are equal program flow continues with the instruction following the CJNE instruction.

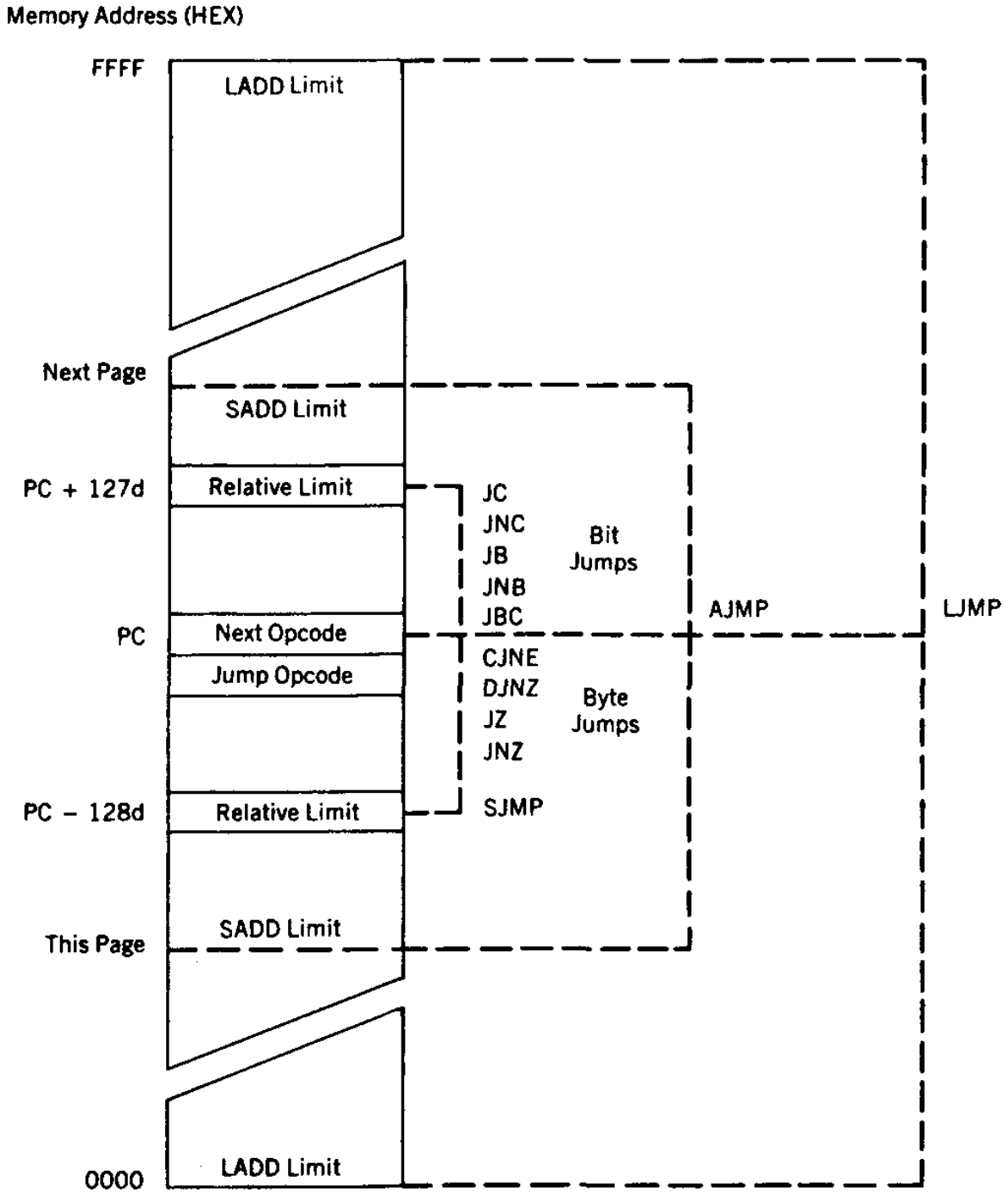
The **Carry bit (C)** is set if *operand1* is less than *operand2*, otherwise it is cleared.

The Jump and Call Program Range

A jump or call instruction can replace the contents of the program counter with a new program address number that causes program execution to begin at the code located at the new address. The difference, in bytes, of this new address from the address in the program where the jump or call is located is called the *range* of the jump or call. For example, if a jump instruction is located at program address 0100h, and the jump causes the program counter to become 0120h, then the range of the jump is 20h bytes.

Jump or call instructions may have one of three ranges: a *relative* range of +127d, −128d bytes from the instruction *following* the jump or call instruction; an *absolute* range on the same 2K byte page as the instruction *following* the jump or call; or a *long* range of any address from 0000h to FFFFh, anywhere in program memory. Figure 6.1 shows the relative range of all the jump instructions.

Jump Instruction Ranges



Relative Range

Jumps that replace the program counter contents with a new address that is greater than the address of the instruction *following* the jump by 127d or less than the address of the instruction following the jump by 128d are called *relative* jumps. They are so named because the address that is placed in the program counter is relative to the address where the jump occurs. If the absolute address of the jump instruction changes, then the jump address changes also but remains the same distance away from the jump instruction. The address following the jump is used to calculate the relative jump because of the action of the PC. The PC is incremented to point to the *next* instruction *before* the current instruction is executed. Thus, the PC is set to the following address before the jump instruction is executed, or in the vernacular: "before the jump is taken."

Relative jumping has two advantages. First, only one byte of data need be specified, either in positive format for jumps ahead in the program or in 2's complement negative format for jumps behind. The jump address displacement byte can then be added to the PC to get the absolute address. Specifying only one byte saves program bytes and speeds up program execution. Second, the program that is written using relative jumps can be located anywhere in the program address space without re-assembling the code to generate absolute addresses.

The disadvantage of using relative addressing is the requirement that all addresses jumped be within a range of +127d, −128d bytes of the jump instruction. This range is not a serious problem. Most jumps form program loops over short code ranges that are within the relative address capability. Jumps are the only branch instructions that can use the relative range.

If jumps beyond the relative range are needed, then a relative jump can be done to another relative jump until the desired address is reached. This need is better handled, however, by the jumps that are covered in the next sections.

Short Absolute Range

Absolute range makes use of the concept of dividing memory into logical divisions called "pages." Program memory may be regarded as one continuous stretch of addresses from 0000h to FFFFh. Or, it may be divided into a series of pages of any convenient binary size, such as 256 bytes, 2K bytes, 4K bytes, and so on.

The 8051 program memory is arranged as 2K byte pages, giving a total of 32d (20h) pages. The hexadecimal address of each page is shown in the following table:

PAGE	ADDRESS(HEX)	PAGE	ADDRESS(HEX)	PAGE	ADDRESS(HEX)
00	0000–07FF	0B	5800–5FFF	16	B000–B7FF
01	0800–0FFF	0C	6000–67FF	17	B800–BFFF
02	1000–17FF	0D	6800–6FFF	18	C000–C7FF
03	1800–1FFF	0E	7000–77FF	19	C800–CFFF
04	2000–27FF	0F	7800–7FFF	1A	D000–D7FF
05	2800–2FFF	10	8000–87FF	1B	D800–DFFF
06	3000–37FF	11	8800–8FFF	1C	E000–E7FF
07	3800–3FFF	12	9000–97FF	1D	E800–EFFF
08	4000–47FF	13	9800–9FFF	1E	F000–F7FF
09	4800–4FFF	14	A000–A7FF	1F	F800–FFFF
0A	5000–57FF	15	A800–AFFF		

Inspection of the page numbers shows that the upper five bits of the program counter hold the page *number*, and the lower eleven bits hold the *address* within each page. An absolute address is formed by taking the page number of the instruction *following* the

branch and attaching the absolute page range address of eleven bits to it to form the 16-bit address.

Branches on page *boundaries* occur when the jump or call instruction finishes at X7FFh or XFFFh. The next instruction starts at X800h or X000h, which places the jump or call address on the same page as the *next* instruction after the jump or call. The page change presents no problem when branching ahead but could be troublesome if the branch is *backwards* in the program. The assembler should flag such problems as errors, so adjustments can be made by the programmer to use a different type of range.

Absolute range addressing has the same advantages as relative addressing; fewer bytes are needed and the code is relocatable as long as the relocated code begins at the start of a page. Absolute addressing has the advantage of allowing jumps or calls over longer programming distances than does relative addressing.

Long Absolute Range

Addresses that can access the entire program space from 0000h to FFFFh use long range addressing. Long-range addresses require more bytes of code to specify and are relocatable only at the beginning of 64K byte pages. Since we are limited to a nominal ROM address range of 64K bytes, the program must be re-assembled every time a long-range address changes and these branches are not generally relocatable.

Long-range addressing has the advantage of using the entire program address space available to the 8051. It is most likely to be used in large programs.

Bit Jumps

Bit jumps all operate according to the status of the carry flag in the PSW or the status of any bit-addressable location. All bit jumps are relative to the program counter.

Jump instructions that test for bit conditions are shown in the following table:

Mnemonic	Operation
JC radd	Jump relative if the carry flag is set to 1
JNC radd	Jump relative if the carry flag is reset to 0
JB b,radd	Jump relative if addressable bit is set to 1
JNB b,radd	Jump relative if addressable bit is reset to 0
JBC b,radd	Jump relative if addressable bit is set, and clear the addressable bit to 0

Note that no flags are affected unless the bit in JBC is a flag bit in the PSW. When the bit used in a JBC instruction is a port bit, the SFR latch for that port is read, tested, and altered.

Byte Jumps

Byte jumps—jump instructions that test bytes of data—behave as bit jumps. If the condition that is tested is *true*, the jump is taken; if the condition is *false*,the instruction after the jump is executed. All byte jumps are relative to the program counter.

The following table lists examples of byte jumps:

Mnemonic	Operation
CJNE A,add,radd	Compare the contents of the A register with the contents of the direct address; if they are *not* equal, then jump to the relative address; set the carry flag to 1 if A is less than the contents of the direct address; otherwise, set the carry flag to 0
CJNE A,#n,radd	Compare the contents of the A register with the immediate number n; if they are *not* equal, then jump to the relative address; set the carry flag to 1 if A is less than the number; otherwise, set the carry flag to 0
CJNE Rn,#n,radd	Compare the contents of register Rn with the immediate number n; if they are *not* equal, then jump to the relative address; set the carry flag to 1 if Rn is less than the number; otherwise, set the carry flag to 0
CJNE @Rp,#n,radd	Compare the contents of the address contained in register Rp to the number n; if they are *not* equal, then jump to the relative address; set the carry flag to 1 if the contents of the address in Rp are less than the number; otherwise, set the carry flag to 0
DJNZ Rn,radd	Decrement register Rn by 1 and jump to the relative address if the result is *not* zero; no flags are affected
DJNZ add,radd	Decrement the direct address by 1 and jump to the relative address if the result is *not* 0; no flags are affected unless the direct address is the PSW
JZ radd	Jump to the relative address if A is 0; the flags and the A register are not changed
JNZ radd	Jump to the relative address if A is *not* 0; the flags and the A register are not changed

Note that if the direct address used in a DJNZ is a port, the port SFR is decremented and tested for 0.

Unconditional Jumps

Unconditional jumps do not test any bit or byte to determine whether the jump should be taken. The jump is *always* taken. All jump ranges are found in this group of jumps, and these are the only jumps that can jump to any location in memory.

The following table shows examples of unconditional jumps:

Mnemonic	Operation
JMP @A+DPTR	Jump to the address formed by adding A to the DPTR; this is an unconditional jump and will always be done; the address can be anywhere in program memory; A, the DPTR, and the flags are unchanged
AJMP sadd	Jump to absolute short range address sadd; this is an unconditional jump and is always taken; no flags are affected
LJMP ladd	Jump to absolute long range address ladd; this is an unconditional jump and is always taken; no flags are affected
SJMP radd	Jump to relative address radd; this is an unconditional jump and is always taken; no flags are affected
NOP	Do nothing and go to the next instruction; NOP (no operation) is used to waste time in a software timing loop; or to leave room in a program for later additions; no flags are affected

Calls and Subroutines

The life of a microcontroller would be very tranquil if all programs could run with no thought as to what is going on in the real world outside. However, a microcontroller is specifically intended to interact with the real world and to react, very quickly, to events that require program attention to correct or control.

A program that does not have to deal unexpectedly with the world outside of the microcontroller could be written using jumps to alter program flow as external conditions require. This sort of program can determine external conditions by moving data from the port pins to a location and jumping on the conditions of the port pin data. This technique is called "polling" and requires that the program does not have to respond to external conditions quickly. (Quickly means in microseconds; slowly means in milliseconds.)

Another method of changing program execution is using "interrupt" signals on certain external pins or internal registers to automatically cause a branch to a smaller program that deals with the specific situation. When the event that caused the interruption has been dealt with, the program resumes at the point in the program where the interruption took place. Interrupt action can also be generated using software instructions named *calls*.

Call instructions may be included explicitly in the program as mnemonics or implicitly included using hardware interrupts. In both cases, the call is used to execute a smaller, stand-alone program, which is termed a *routine* or, more often, a *subroutine*.

Subroutines

A *subroutine* is a program that may be used many times in the execution of a larger program. The subroutine could be written into the body of the main program everywhere it is needed, resulting in the fastest possible code execution. Using a subroutine in this manner has several serious drawbacks.

Common practice when writing a large program is to divide the total task among many programmers in order to speed completion. The entire program can be broken into smaller parts and each programmer given a part to write and debug. The main program can then call each of the parts, or subroutines, that have been developed and tested by each individual of the team.

Even if the program is written by one individual, it is more efficient to write an oft-used routine once and then call it many times as needed. Also, when writing a program, the programmer does the main part first. Calls to subroutines, which will be written later, enable the larger task to be defined before the programmer becomes bogged down in the details of the application.

Finally, it is quite common to buy "libraries" of common subroutines that can be called by a main program. Again, buying libraries leads to faster program development.

Calls and the Stack

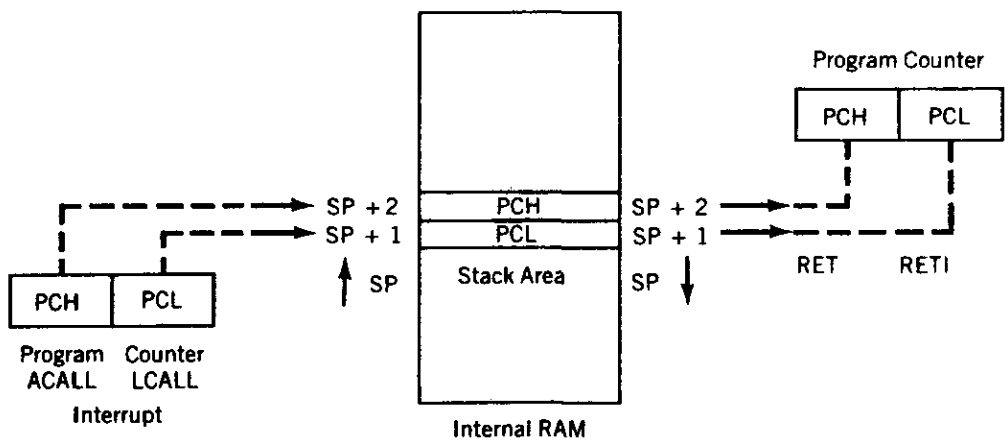
A call, whether hardware or software initiated, causes a jump to the address where the called subroutine is located. At the end of the subroutine the program resumes operation at the opcode address immediately *following* the call. As calls can be located anywhere in the program address space and used many times, there must be an automatic means of storing the address of the instruction following the call so that program execution can continue after the subroutine has executed.

The *stack* area of internal RAM is used to automatically store the address, called the return address, of the instruction found immediately after the call. The stack pointer register holds the address of the *last* space used on the stack. It stores the return address above this space, adjusting itself upward as the return address is stored. The terms "stack" and "stack pointer" are often used interchangeably to designate the *top* of the stack area in RAM that is pointed to by the stack pointer.

Figure 6.2 diagrams the following sequence of events:

1. A call opcode occurs in the program software, or an interrupt is generated in the hardware circuitry.
2. The return address of the next instruction after the call instruction or interrupt is found in the program counter.
3. The return address bytes are pushed on the stack, *low* byte *first*.
4. The stack pointer is incremented for each push on the stack.
5. The subroutine address is placed in the program counter.
6. The subroutine is executed.
7. A RET (return) opcode is encountered at the end of the subroutine.

FIGURE 6.2 Storing and Retrieving the Return Address



8. Two pop operations restore the return address to the PC from the stack area in internal RAM.
9. The stack pointer is decremented for each address byte pop.

All of these steps are automatically handled by the 8051 hardware. It is the *responsibility* of the programmer to ensure that the subroutine ends in a RET instruction *and* that the stack does not grow up into data areas that are used by the program.

Calls and Returns

Calls use short- or long-range addressing; returns have no addressing mode specified but are always long range. The following table shows examples of call opcodes:

Mnemonic	Operation
ACALL sadd	Call the subroutine located on the same page as the address of the opcode immediately following the ACALL instruction; push the address of the instruction immediately after the call on the stack
LCALL ladd	Call the subroutine located anywhere in program memory space; push the address of the instruction immediately following the call on the stack
RET	Pop two bytes from the stack into the program counter

Note that no flags are affected unless the stack pointer has been allowed to erroneously reach the address of the PSW special-function register.

Interrupts and Returns

As mentioned previously, an *interrupt* is a hardware-generated call. Just as a call opcode can be located within a program to automatically access a subroutine, certain pins on the 8051 can cause a call when external electrical signals on them go to a low state. Internal operations of the timers and the serial port can also cause an interrupt call to take place.

The subroutines called by an interrupt are located at fixed hardware addresses discussed in Chapter 2. The following table shows the interrupt subroutine addresses.

INTERRUPT	ADDRESS (HEX) CALLED
IE0	0003
TF0	000B
IE1	0013
TF1	001B
SERIAL	0023

When an interrupt call takes place, hardware interrupt disable flip-flops are set to prevent another interrupt of the same priority level from taking place until an interrupt return instruction has been executed in the interrupt subroutine. The action of the interrupt routine is shown in the table below.

Mnemonic	Operation
RETI	Pop two bytes from the stack into the program counter and reset the interrupt enable flip-flops

Note that the only difference between the RET and RETI instructions is the enabling of the interrupt logic when RETI is used. RET is used at the ends of subroutines called by an opcode. RETI is used by subroutines called by an interrupt.

UNIT 3:

8051 programming: Assembler directives, Assembly language programs and Time delay calculations.
Introduction:

8051 micro controller has one data type. It is 8-bit and size of each register is also 8-bit. It is job of programmer to break down data larger than 8 bits (00 to FFH, 0 to 255 in decimal) to be processed by CPU.
Data byte (DB) directive: The DB directive is most widely used data directive in assembler It is used to define 8-bit data. When DB is used to define data, the number can be in decimal, binary, hex or ASCII formats. The assembler will convert the number into hex. The assembler will assign the ASCII code for the numbers or characters automatically. The DB directive is only directive that can be used to define ASCII strings larger than two characters Therefore, it should be used for all ASCII data definitions.
The most widely used Assembler directives are ORG Directive
EQU Directive & END Directive

Delay calculations:

Delay	Calculations	Code	Remarks
50mS Xtal freq =11.0592MHz	50mS/1.085uS = 46082.9		
	46082.9/255 = 180.7		Its an Odd Value, so round off the result i.e. 180.7 to the nearest even number i.e 182
	46082.9 / x =182		Find value of x
	46082.9 / 253 = 182		We'll load 253 in the inner most loop
	182 / 2 = 91		Always do this, so its result will be our outer loop
		Delay:Mov R0, #91 Here1:Mov R1, #253 Here:DJNZ R1, here DJNZ R0, here1 RET	
500mS Xtal freq =11.0592MHz	500mS/1.085uS = 460829.5		
	460829.5 / 255 = 1807.1		Odd number does not account as its not less than 255, innermost loop = 255
	1807.1 / 255 = 7.086		Round off to nearest higher even i.e 8
	1807.1 / x = 8		
	1807.1 / 226 = 8		Inner loop = 226
	8 / 2 = 4		Outer loop = 4
		Delay:Mov R0,#4 Here2:Mov R1, #226 Here1:Mov R2, #255 Here:DJNZ R2, Here DJNZ R1, Here1 DJNZ R0,Here2 End	

1Second Xtal freq= 20MHz	1 / 0.6uS = 1.6 x 10e6		
	1.6 x 10e6 / 255 = 6536		Innermost loop= 255
	6536 / 255 = 25.6		Not Even so make it even i.e. 26
	6536 / x = 26		
	6536 / 251 = 26		Inner loop 251
	26/2 = 13		Outer loop = 13
		Delay:Mov R0, # 13 Here2: Mov R1, # 251 Here1: Mov R2, #255 Here: DJNZ R2, Here DJNZ R1, Here1 DJNZ R0, Here2 Ret	

UNIT 4:

8051 Interfacing and Applications: Basics of I/O concepts, I/O Port Operation, Interfacing 8051 to LCD, Keyboard, parallel and serial ADC, DAC, Stepper motor interfacing and DC motor interfacing and programming.

Objectives:

At the end of this chapter, we will be able to:

- List the different devices that can be interfaced with 8051
- Understand the working principle.
- Develop the following applications using assembly and C
  - Stepper motor interface
  - DC motor interfacing and PWM
  - Digital-to-Analog interfacing
  - Analog-to-Digital interfacing
  - LCD interface
  - Keyboard interface

This chapter basically gives an insight into the study of different interfacings listed above. Further we will also study and understand their operation that is the working principle. We will further discuss on how to develop these interfaces using assembly and C.

Stepper Motor Interfacing:

Stepper motor is a widely used device that translates electrical pulses into mechanical movement. Stepper motor is used in applications such as; disk drives, dot matrix printer, robotics etc,. The construction of the motor is as shown in figure 1 below.

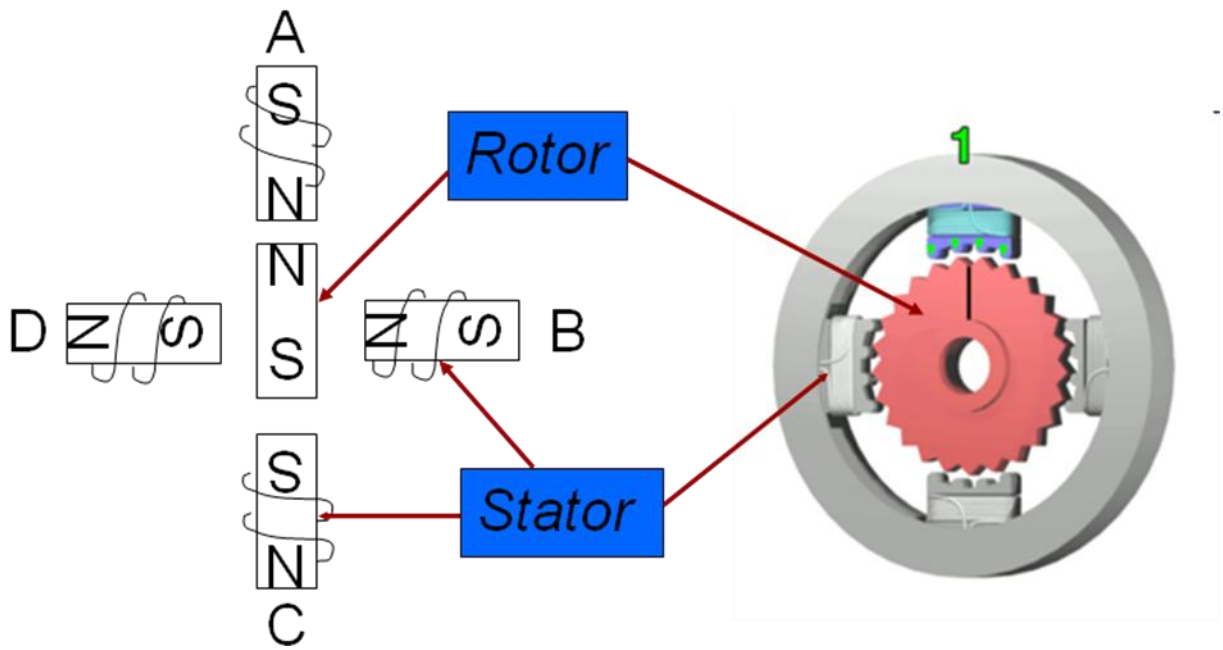


Figure 1: Structure of stepper motor

It has a permanent magnet rotor called the shaft which is surrounded by a stator. Commonly used stepper motors have four stator windings that are paired with a center – tapped common. Such motors are called as four-phase or unipolar stepper motor.

The stator is a magnet over which the electric coil is wound. One end of the coil are connected commonly either to ground or +5V. The other end is provided with a fixed sequence such that the motor rotates in a particular direction. Stepper motor shaft moves in a fixed repeatable increment, which allows one to move it to a precise position. Direction of the rotation is dictated by the stator poles. Stator poles are determined by the current sent through the wire coils.

Step angle:
Step angle is defined as the minimum degree of rotation with a single step.
No of steps per revolution = 360° / step angle
Steps per second = (rpm x steps per revolution) / 60
Example: step angle = 2°

No of steps per revolution = 180

Switching Sequence of Motor:

As discussed earlier the coils need to be energized for the rotation. This can be done by sending a bits sequence to one end of the coil while the other end is commonly connected. The bit sequence sent can make either one phase ON or two phase ON for a full step sequence or it can be a combination of one and two phase ON for half step sequence. Both are tabulated below.

Full Step:

Two Phase ON

Clockwise ↓

Step #	A	B	C	D
1	1	0	0	1
2	1	1	0	0
3	0	1	1	0
4	0	0	1	1

Counter-clockwise ↑

One Phase ON

Clockwise ↓

Step #	A	B	C	D
1	1	0	0	0
2	0	1	0	0
3	0	0	1	0
4	0	0	0	1

Counter-clockwise ↑

Half Step (8 – sequence):

The sequence is tabulated as below:

Clk ↓

Step #	A	B	C	D
1	1	0	0	1
2	1	0	0	0
3	1	1	0	0
4	0	1	0	0
5	0	1	1	0
6	0	0	1	0
7	0	0	1	1
8	0	0	0	1

Anti-Clk ↑

8051 Connection to Stepper Motor: (explanation of the diagram can be done)

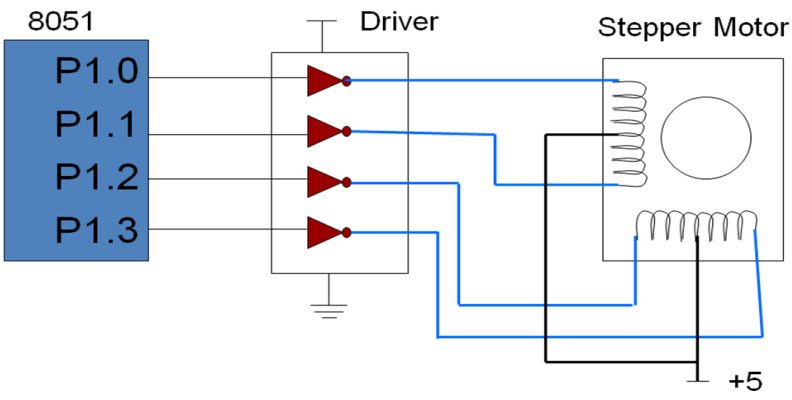


Figure 2: 8051 interfaces to stepper motor

The following example 1 to example 6 shown below will elaborate on the discussion done above:

Example 1: Write an ALP to rotate the stepper motor clockwise / anticlockwise continuously with full step sequence.

Program:

```
        MOV A,#66H
BACK:   MOV P1,A
        RR A
        ACALL DELAY
        SJMP BACK

DELAY:  MOV R1,#100
 UP1:     MOV R2,#50
 UP:      DJNZ R2,UP
          DJNZ R1,UP1
          RET
```

Note: motor to rotate in anticlockwise use instruction RL A instead of RR A

Example 2: A switch is connected to pin P2.7. Write an ALP to monitor the status of the SW. If SW = 0, motor moves clockwise and if SW = 1, motor moves anticlockwise.

Program:

```
        ORG 0000H
        SETB P2.7
        MOV A, #66H
        MOV P1,A
TURN: JNB P2.7, CW
        RL A
        ACALL DELAY
        MOV P1,A
        SJMP TURN
CW:     RR A
        ACALL DELAY
        MOV P1,A
        SJMP TURN
DELAY: as previous example
```

Example 3: Write an ALP to rotate a motor 90° clockwise. Step angle of motor is 2°.

Solution:

Step angle = 2°
Steps per revolution = 180
No of rotor teeth = 45
For 90° rotation the no of steps is 45
Program:

```
        ORG 0000H
        MOV A, #66H
        MOV R0, #45
BACK: RR A
        MOV P1, A
        ACALL DELAY
        DJNZ R0, BACK
        END
```

Example 4: Rotate the stepper motor continuously clockwise using half-step 8-step sequence. Say the sequence is in ROM locations.

Program:

```
ORG 0000H
START: MOV R0, #08
MOV DPTR, #HALFSTEP
RPT:      CLR A
MOVC A, @A+DPTR
MOV P1, A
ACALL DELAY
INC DPTR
DJNZ R0, RPT
SJMP START
ORG 0200H
HALFSTEP DB 09, 08, 0CH, 04, 06, 02, 03, 01
END
```

Programming Stepper Motor with 8051 C

The following examples 5 and 6 will show the programming of stepper motor using 8051 C.

Example 5: Problem definition is same as example 1.
Program:

```
#include <reg51.h>
void main ()
{
        while (1)
         {
            P1=0x66;
            MSDELAY (200);
            P1=0x33;
            MSDELAY (200);
            P1=0x99;
            MSDELAY (200);
            P1=0xCC;
            MSDELAY (200);
         }
                                                    }
void MSDELAY (unsigned char value)
{
         unsigned int x,y;
         for(x=0;x<1275;x++)
         for(y=0;y<value;y++);
}
```

Example 6: Problem definition is same as example 2.
Program:

```
#include <reg51.h>
sbit SW=P2^7;
void main ()
{
        SW=1;
         while (1)
         {
           if(SW==0){
            P1=0x66;
            MSDELAY (100);
            P1=0x33;
            MSDELAY (100);
            P1=0x99;
            MSDELAY (100);
            P1=0xCC;
            MSDELAY (100);
         }
         else {
            P1=0x66;
            MSDELAY (100);
            P1=0xCC;
            MSDELAY (100);
            P1=0x99;
            MSDELAY (100);
            P1=0x33;
            MSDELAY (100);
}
void MSDELAY (unsigned char value)
{
         unsigned int x,y;
         for(x=0;x<1275;x++)
         for(y=0;y<value;y++);
}
```

DC Motor Interfacing with 8051:

The DC motor is another widely used device that translates electrical pulses into mechanical movement. Motor has 2 leads +ve and – ve , connecting them to a DC voltage supply moves the motor in one direction. On reversing the polarity rotates the motor in the reverse direction. Basic difference between Stepper and DC motor is stepper motor moves in steps while DC motor moves continuously. Another difference is with stepper motor the number of steps can be counted while it is not possible in DC motor. Maximum speed of a DC motor is indicated in rpm. The rpm is either with no load it is few thousands to tens of thousands or with load rpm decreases with increase in load.

Voltage and current rating : Nominal voltage is the voltage for a motor under normal condition. It ranges from 1V to 150V. As voltage increases, rpm goes up. Current rating is the current consumption when the nominal voltage is applied with no load that is 25mA to a few amperes. As load increases, rpm increases, unless voltage or current increases implies torque increases. With fixed voltage, as load increases, power consumption of a DC motor is increased.

Unidirectional Control:

Figure 3 shows the rotation of the DC motor in clockwise and anticlockwise direction.

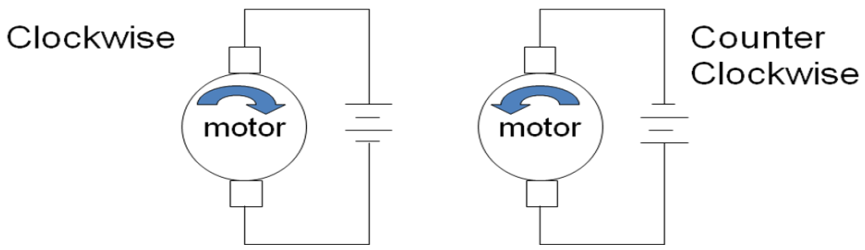
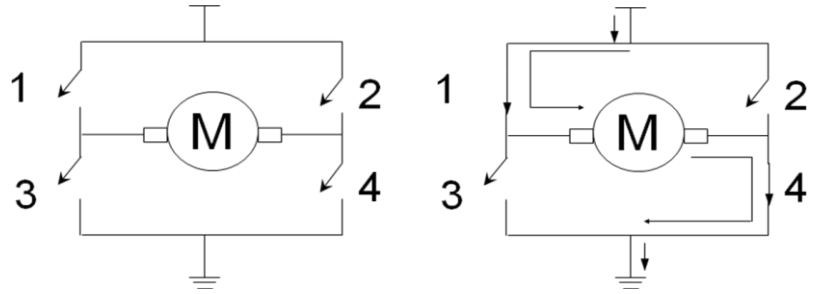
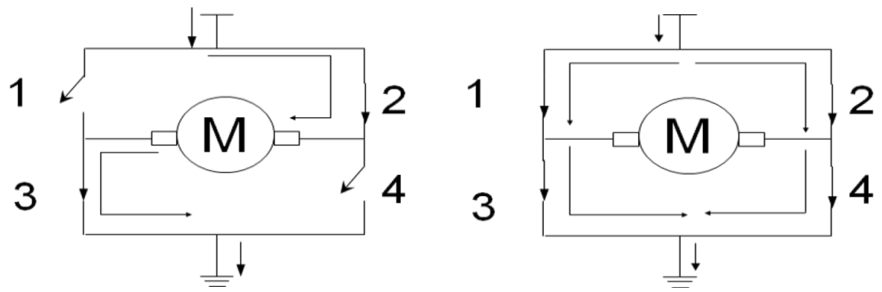


Figure 3: DC motor rotation

Bidirectional Control:



(a) Motor not running (b) Clockwise direction



(c) Counter clockwise direction (d) Invalid state (short circuit)

Figure 4: H-Bridge Motor Configuration

Figure 4 shows the H-Bridge motor configuration. It consists of four switches and based on the closing and opening of these switches the motor either rotates in clockwise or anti-clockwise direction.

As seen in figure 4a, all the switches are open hence the motor is not running. In b, turning of the motor is in one direction when the switches 1 and 4 are closed that is clockwise direction.

Similarly, in c the switches 2 and 3 are closed so the motor rotates in anticlockwise direction, while in figure 4d all the switches are closed which indicates a invalid state or a short circuit.

The interfacing diagram of 8051 to bidirectional motor control can be referred to fig 17-18 from text prescribed.

Example 6: A switch is connected to pin P2.7. Write an ALP to monitor the status of the SW. If SW = 0, DC motor moves clockwise and if SW = 1, DC motor moves anticlockwise.

Program:

```
            ORG 0000H
            CLR P1.0
            CLR P1.1
            CLR P1.2
            CLR P1.3
            SETB P2.7
MONITOR:    JNB P2.7, CLOCK
            SETB P1.0
            CLR P1.1
            CLR P1.2
            SETB P1.3
            SJMP MONITOR
CLOCK:      CLR P1.0
            SETB P1.1
            SETB P1.2
            CLR P1.3
            SJMP MONITOR
            END
```

Pulse Width Modulation (PWM):

The speed of the motor depends on 3 parameters: load, voltage and current. For a given load, we can maintain a steady speed by using PWM. By changing the width of the pulse applied to DC motor, power provided can either be increased or decreased. Though voltage has fixed amplitude, has a variable duty cycle. The wider the pulse, higher the speed obtained. One of the reasons as to why dc motor are referred over ac is, the ability to control the speed of the DC motor using PWM. The speed of the ac motor is dictated by the ac frequency of voltage applied to the motor and is generally fixed. Hence, speed of the AC motors cannot be controlled when load is increased.

Figure 5 below shows the pulse width modulation comparison.

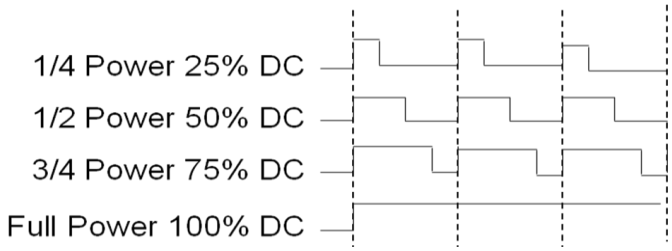


Figure 5: PWM comparison

Example 7: A switch is connected to pin P2.7. Write a C to monitor the status of the SW. If SW = 0, DC motor moves clockwise and if SW = 1, DC motor moves anticlockwise.
Program:

```
# include <reg51.h>
sbit SW =P2^7;
sbit Enable = P1^0;
sbit MTR_1 = P1^1;
sbit MTR_2 = P1^2;
void main ( )
{
        SW=1;
        Enable = 0;
        MTR_1=0;
        MTR_2=0;
while( )
{
        Enable =1;
        if( SW==1)
        {       MTR_1=1;
                MTR_2=0;
        }
        else
        {       MTR_1=0;
                MTR_2=1;
        }}}
```

Example 8: A switch is connected to pin P2.7. Write an C to monitor the status of the SW. If SW = 0, DC motor moves 50% duty cycle pulse and if SW = 1, DC motor moves with 25% duty cycle pulse.

Program:

```
# include <reg51.h>
sbit SW =P2^7;
sbit MTR = P1^0;
void main ( )
{
        SW=1;
        MTR=0;
while( )
{
        if( SW==1)
        {       MTR=1;
                Msdelay(25);
                MTR=0;
                Msdelay(75);
        }
        else
        {       MTR=1;
                Msdelay(50);
                MTR=0;
                Msdelay(50);
        }
}
}
```

The interfacing diagrams for the above examples can be referred to the text.

Digital-to-Analog (DAC) converter:

The DAC is a device widely used to convert digital pulses to analog signals. In this section we will discuss the basics of interfacing a DAC to 8051.

The two method of creating a DAC is binary weighted and R/2R ladder.

The Binary Weighted DAC, which contains one resistor or current source for each bit of the DAC connected to a summing point. These precise voltages or currents sum to the correct output value. This is one of the fastest conversion methods but suffers from poor accuracy because of the high precision required for each individual voltage or current. Such high-precision resistors and current-sources are expensive, so this type of converter is usually limited to 8-bit resolution or less.

The R-2R ladder DAC, which is a binary weighted DAC that uses a repeating cascaded structure of resistor values R and 2R. This improves the precision due to the relative ease of producing equal valued matched

resistors (or current sources). However, wide converters perform slowly due to increasingly large RC-constants for each added R-2R link.

The first criterion for judging a DAC is its resolution, which is a function of the number of binary inputs. The common ones are 8, 10, and 12 bits. The number of data bit inputs decides the resolution of the DAC since the number of analog output levels is equal to 2^n, where n is the number of data bit inputs.

DAC0808:

The digital inputs are converter to current $I_{out,}$ and by connecting a resistor to the I_{out} pin, we can convert the result to voltage. The total current I_{out} is a function of the binary numbers at the D0-D7 inputs of the DAC0808 and the reference current I_{ref} , and is as follows:

$$I_{out} = I_{ref}\left(\frac{D7}{2}+\frac{D6}{4}+\frac{D5}{8}+\frac{D4}{16}+\frac{D3}{32}+\frac{D2}{64}+\frac{D1}{128}+\frac{D0}{256}\right)$$

Usually reference current is 2mA. Ideally we connect the output pin to a resistor, convert this current to voltage, and monitor the output on the scope. But this can cause inaccuracy; hence an opamp is used to convert the output current to voltage. The 8051 connection to DAC0808 is as shown in the figure 6 below.

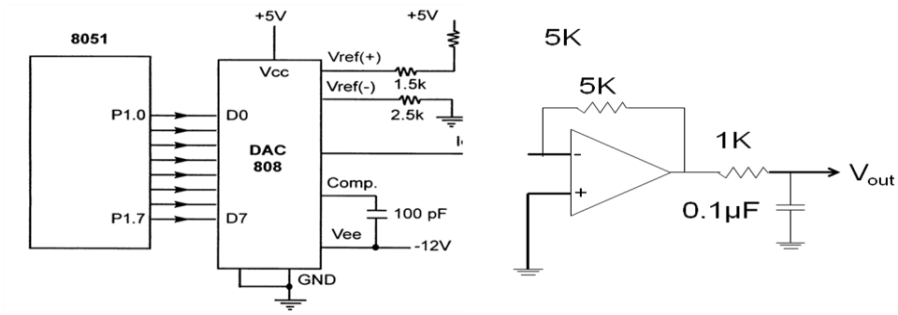


Figure 6: 8051 connections to DAC0808

The following examples 9, 10 and 11 will show the generation of waveforms using DAC0808.

Example 9: Write an ALP to generate a triangular waveform.

Program:

```
        MOV A, #00H
INCR:   MOV P1, A
        INC A
        CJNE A, #255, INCR
DECR:   MOV P1, A
        DEC A
        CJNE A, #00, DECR
        SJMP INCR
        END
```

Example 10: Write an ALP to generate a sine waveform.

$$V_{out} = 5V(1+\sin\theta)$$

Solution: Calculate the decimal values for every 10 degree of the sine wave. These values can be maintained in a table and simply the values can be sent to port P1. The sinewave can be observed on the CRO.

Program:

```
                ORG 0000H
AGAIN:          MOV DPTR, #SINETABLE
                MOV R3, #COUNT
UP:             CLR A
                MOVC A, @A+DPTR
                MOV P1, A
                INC DPTR
                DJNZ R3, UP
                SJMP AGAIN
                ORG 0300H
SINETABLE DB 128, 192, 238, 255, 238, 192, 128, 64, 17, 0, 17, 64, 128
                END
```

Note: to get a better wave regenerate the values of the table per 2 degree.

Example 10: Write a C program to generate a sine waveform.

$$\mathbf{V_{out} = 5V(1+\sin\theta)}$$

Program:

```
#include<reg51.h>
sfr dacdata=P1;
void main( )
{
        unsigned char sinetable[12]={ 128, 192, 238, 255, 238, 192,
                                      128, 64, 17, 0, 17, 64};
        unsigned char x;
        while (1)
        {
                for(x=0;x<12;x++)
                {
                   dacdata = sinetable[x];
                }
        }
}
```

Analog-to-digital converter (ADC) interfacing:

ADCs (analog-to-digital converters) are among the most widely used devices for data acquisition. A physical quantity, like temperature, pressure, humidity, and velocity, etc., is converted to electrical (voltage, current) signals using a device called a transducer, or sensor. We need an analog-to-digital converter to translate the analog signals to digital numbers, so microcontroller can read them.

ADC804 chip:

ADC804 IC is an analog-to-digital converter. It works with +5 volts and has a resolution of 8 bits. Conversion time is another major factor in judging an ADC. Conversion time is defined as the time it takes the ADC to convert the analog input to a digital (binary) number. In ADC804 conversion time varies depending on the clocking signals applied to CLK R and CLK IN pins, but it cannot be faster than 110µs.

Pin Description of ADC804:

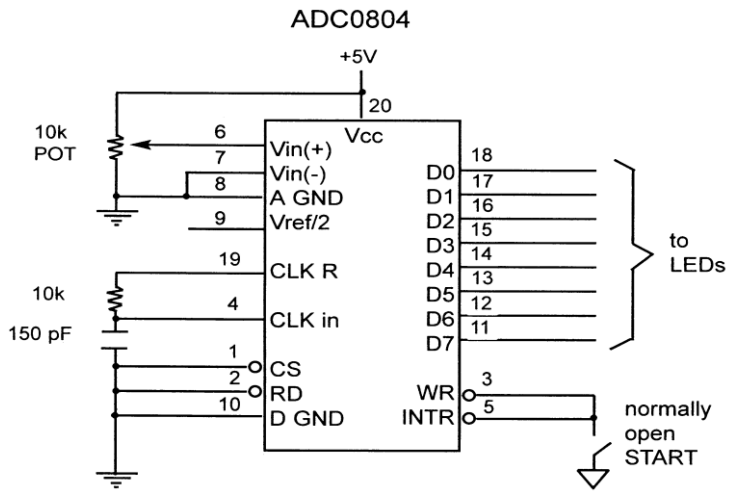


Figure 7: Pin out of ADC0804

- **CLK IN and CLK R:** CLK IN is an input pin connected to an external clock source. To use the internal clock generator (also called self-clocking), CLK IN and CLK R pins are connected to a capacitor and a resistor and the clock frequency is determined by:

$$f = \frac{1}{1.1RC}$$

Typical values are R = 10K ohms and C =150pF. We get f = 606 kHz and the conversion time is 110µs.

- **Vref/2 :** It is used for the reference voltage. If this pin is open (not connected), the analog input voltage is in the range of 0 to 5 volts (the same as the Vcc pin). If the analog input range needs to be 0 to 4 volts, Vref/2 is connected to 2 volts. Step size is the smallest change can be discerned by an ADC

Vref/2 Relation to Vin Range

$V_{ref}/2$(v)	V_{in}(V)	Step Size (mV)
Not connected*	0 to 5	5/256=19.53
2.0	0 to 4	4/255=15.62
1.5	0 to 3	3/256=11.71
1.28	0 to 2.56	2.56/256=10
1.0	0 to 2	2/256=7.81
0.5	0 to 1	1/256=3.90

- **D0-D7:** The digital data output pins. These are tri-state buffered. The converted data is accessed only when CS =0 and RD is forced low. To calculate the output voltage, use the following formula

$$D_{out} = \frac{V_{in}}{step\ size}$$

  - Dout = digital data output (in decimal),
  - Vin = analog voltage, and
  - step size (resolution) is the smallest change
- **Analog ground and digital ground:** Analog ground is connected to the ground of the analog Vin and digital ground is connected to the ground of the Vcc pin. To isolate the analog Vin signal from transient voltages caused by digital switching of the output D0 – D7. This contributes to the accuracy of the digital data output.
- **Vin(+) & Vin(-):** Differential analog inputs where Vin = Vin (+) – Vin (-). Vin (-) is connected to ground and Vin (+) is used as the analog input to be converted.
- **RD:** Is "output enable" a high-to-low RD pulse is used to get the 8-bit converted data out of ADC804.
- **INTR:** It is "end of conversion" When the conversion is finished, it goes low to signal the CPU that the converted data is ready to be picked up.
- **WR:** It is "start conversion" When WR makes a low-to-high transition, ADC804 starts converting the analog input value of Vin to an 8- bit digital number.
- **CS:** It is an active low input used to activate ADC804.

The following steps must be followed for data conversion by the ADC804 chip:

1. Make CS= 0 and send a L-to-H pulse to pin WR to start conversion.
2. Monitor the INTR pin, if high keep polling but if low, conversion is complete, go to next step.
3. Make CS= 0 and send a H-to-L pulse to pin RD to get the data out

Figure 8 shows the read and write timing for ADC804. Figure 9 and 10 shows the self clocking with the RC component for frequency and the external frequency connected to XTAL2 of 8051.

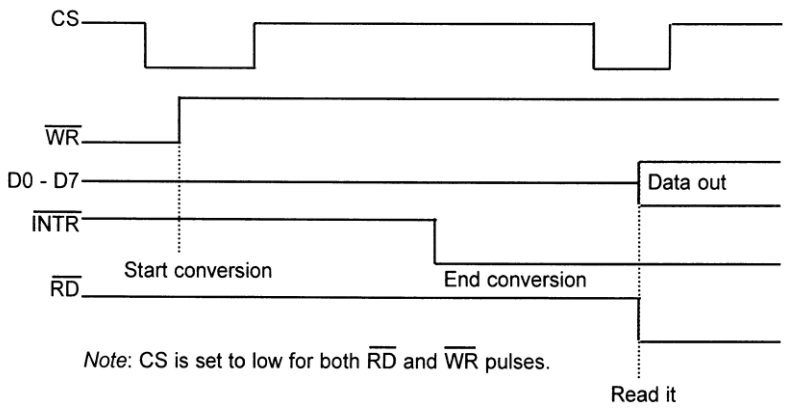


Figure 8: Read and Write timing for ADC0804

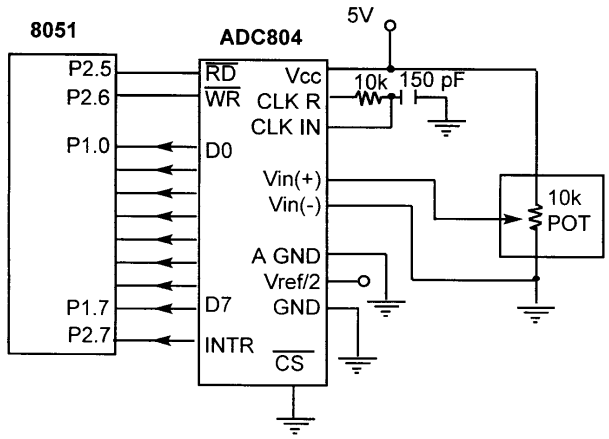


Figure 9: 8051 Connection to ADC0804 with Self-clocking

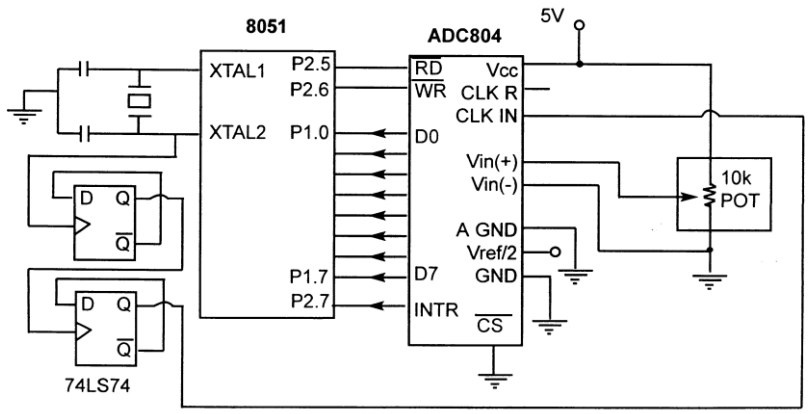


Figure 10: 8051 Connection to ADC0804 with Clock from XTAL2 of 8051

Now let us see how we write assembly as well as C program for the interfacing diagram shown in figure 10.

Programming ADC0804 in assembly

```
                MYDATA EQU P1
                MOV P1, #0FFH
                SETB P2.7
BACK:           CLR P2.6
                SETB P2.6
HERE:           JB P2.7, HERE
                CLR P2.5
                MOV A, MYDATA
                SETB P2.5
                SJMP BACK
```

Programming ADC0804 in C

```
#include<reg51.h>
Sbit RD=P2^5;
Sbit WR=P2^6;
Sbit INTR=P2^7;
Sfr Mydata=P1;
Void main ( )
{
        Unsigned char value;
        Mydata =0xFF;
        INTR=1;
        RD=1;
        WR=1;
        While (1)
        {
          WR=0;
          WR=1;
           While (INTR == 1);
          RD=0;
          Value =Mydata;
          RD=1;
        }
}
```

ADC0808/0809 chip:

ADC808 has 8 analog inputs. It allows us to monitor up to 8 different transducers using only single chip. The chip has 8-bit data output just like the ADC804. The 8 analog input channels are multiplexed and selected according to the values given to the three address pins, A, B, and C. that is; if CBA=000, CH0 is selected; CBA=011, CH3 is selected and so on. The pin details of ADC0808 are as shown in the figure 11 below. (Explanation can be done as is with ADC0804).

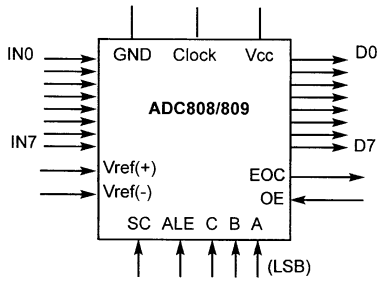


Figure 11: Pin out of ADC0808

Steps to Program ADC0808/0809

1. Select an analog channel by providing bits to A, B, and C addresses.
2. Activate the ALE pin. It needs an L-to-H pulse to latch in the address.
3. Activate SC (start conversion) by an H-to-L pulse to initiate conversion.
4. Monitor EOC (end of conversion) to see whether conversion is finished.
5. Activate OE (output enable) to read data out of the ADC chip. An H-to-L pulse to the OE pin will bring digital data out of the chip.

Let us write an assembly and C program for the interfacing of 8051 to ADC0808 as shown in figure 12

Programming ADC0808/0809 in assembly

```
            MYDATA EQU P1
            ORG 0000H
            MOV MYDATA, #0FFH
            SETB P2.7
            CLR P2.4
            CLR P2.6
            CLR P2.5
BACK:       CLR P2.0
            CLR P2.1
            SETB P2.2
            ACALL DELAY
            SETB P2.4
            ACALL DELAY
            SETB P2.6
            ACALL DELAY
            CLR P2.4
            CLR P2.6
HERE:       JB P2.7, HERE
HERE1:      JNB P2.7, HERE1
            SETB P2.5
            ACALL DELAY
            MOV A, MYDATA
            CLR P2.5
            SJMP BACK
```

Note: replace the assembly instructions with equivalent C statements for programming ADC0808 in C

LCD Interfacing:

LCD is finding widespread use replacing LEDs for the following reasons:

- The declining prices of LCD
- The ability to display numbers, characters, and graphics
- Incorporation of a refreshing controller into the LCD, thereby relieving the CPU of the task of refreshing the LCD
- Ease of programming for characters and graphics

Pin Description:

Pin	Symbol	I/O	Description
1	V_{SS}	--	Ground
2	V_{CC}	--	+5V power supply
3	V_{EE}	--	Power supply to control contrast
4	RS	I	RS=0 to select command register, RS=1 to select data register
5	R/W	I	R/W=0 for write, R/W=1 for read
6	E	I/O	Enable
7	DB0	I/O	The 8-bit data bus
8	DB1	I/O	The 8-bit data bus
9	DB2	I/O	The 8-bit data bus
10	DB3	I/O	The 8-bit data bus
11	DB4	I/O	The 8-bit data bus
12	DB5	I/O	The 8-bit data bus
13	DB6	I/O	The 8-bit data bus
14	DB7	I/O	The 8-bit data bus

- Send displayed information or instruction command codes to the LCD
- Read the contents of the LCD's internal registers

LCD Command Codes:

Code (Hex)	Command to LCD Instruction Register
1	Clear display screen
2	Return home
4	Decrement cursor (shift cursor to left)
6	Increment cursor (shift cursor to right)
5	Shift display right
7	Shift display left
8	Display off, cursor off
A	Display off, cursor on
C	Display on, cursor off
E	Display on, cursor blinking
F	Display on, cursor blinking
10	Shift cursor position to left
14	Shift cursor position to right
18	Shift the entire display to the left
1C	Shift the entire display to the right
80	Force cursor to beginning to 1st line
C0	Force cursor to beginning to 2nd line
38	2 lines and 5x7 matrix

LCD timing diagram for reading and writing is as shown in figure 14 and 15.

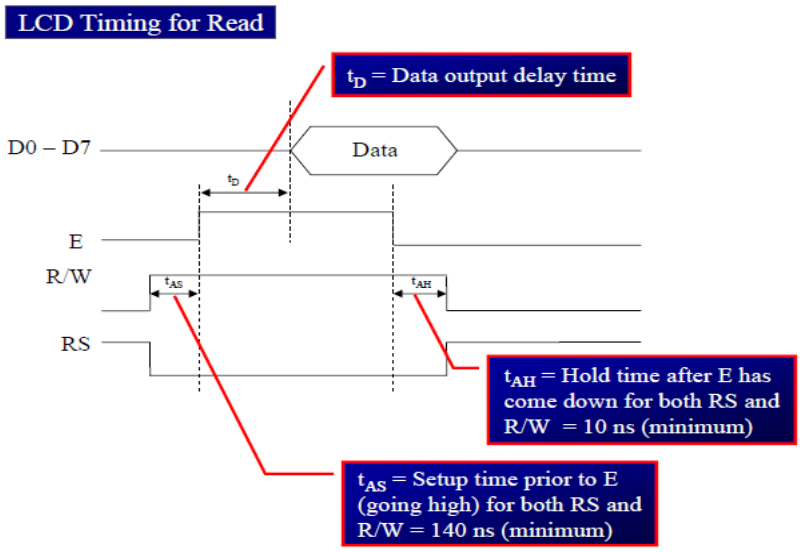


Figure 14: LCD timing for read

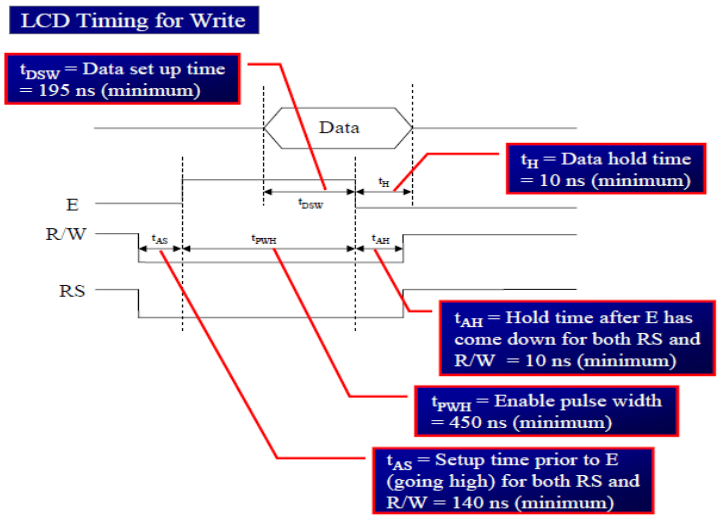


Figure 15: LCD timing for write

Sending Data/ Commands to LCDs with Time Delay:

To send any of the commands to the LCD, make pin RS=0. For data, make RS=1. Then send a high-to-low pulse to the E pin to enable the internal latch of the LCD. This is shown in the code below. The interfacing diagram of LCD to 8051 is as shown in the figure 16.

Example 11: Write an ALP to initialize the LCD and display message "YES". Say the command to be given is :38H (2 lines ,5x7 matrix), 0EH (LCD on, cursor on), 01H (clear LCD), 06H (shift cursor right), 86H (cursor: line 1, pos. 6)

Program:

```
;calls a time delay before sending next data/command ;P1.0-P1.7 are connected to LCD data pins D0-D7
;P2.0 is connected to RS pin of LCD ;P2.1 is connected to  R/W pin of LCD ;P2.2 is connected to E pin
of LCD
ORG 0H
        MOV A,#38H          ;INIT. LCD 2 LINES, 5X7 MATRIX
        ACALL COMNWRT       ;call command subroutine
        ACALL DELAY         ;give LCD some time
        MOV A,#0EH          ;display on, cursor on
        ACALL COMNWRT       ;call command subroutine
        ACALL DELAY          ;give LCD some time
        MOV A,#01           ;clear LCD
        ACALL COMNWRT       ;call command subroutine
        ACALL DELAY         ;give LCD some time
        MOV A,#06H          ;shift cursor right
        ACALL COMNWRT        ;call command subroutine
        ACALL DELAY         ;give LCD some time
        MOV A,#86H          ;cursor at line 1, pos. 6
        ACALL COMNWRT       ;call command subroutine
        ACALL DELAY         ;give LCD some time
```

```
        MOV A,#'Y'              ;display letter Y
        ACALL DATAWRT           ;call display subroutine
        ACALL DELAY             ;give LCD some time
        MOV A,#'E'              ;display letter E
        ACALL DATAWRT           ;call display subroutine
        ACALL DELAY             ;give LCD some time
        MOV A,#'S'              ;display letter S
        ACALL DATAWRT           ;call display subroutine
AGAIN: SJMP AGAIN       ;stay here

COMNWRT:                ;send command to LCD
        MOV P1,A                ;copy reg A to port 1
        CLR P2.0                ;RS=0 for command
        CLR P2.1                ;R/W=0 for write
        SETB P2.2               ;E=1 for high pulse
        ACALL DELAY             ;give LCD some time
        CLR P2.2                ;E=0 for H-to-L pulse
RET
DATAWRT:                ;write data to LCD
        MOV P1,A                ;copy reg A to port 1
        SETB P2.0               ;RS=1 for data
        CLR P2.1                ;R/W=0 for write
        SETB P2.2               ;E=1 for high pulse
        ACALL DELAY             ;give LCD some time
        CLR P2.2                ;E=0 for H-to-L pulse
RET
DELAY:
        MOV R3,#50              ;50 or higher for fast CPUs
HERE2: MOV R4,#255              ;R4 = 255
HERE: DJNZ R4,HERE              ;stay until R4 becomes 0
        DJNZ R3,HERE2
RET
END
```

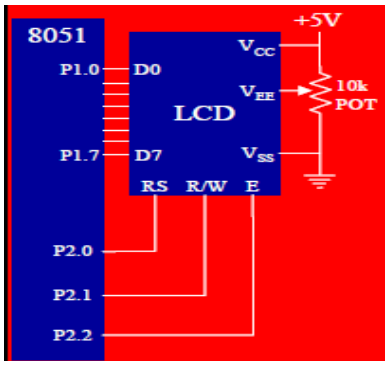


Figure 16: 8051 Connection to LCD

Sending Data/ Commands to LCDs checking the Busy Flag

Example 12: Modify example 11, to check for the busy flag (D7=>P1.7), then send the command and hence display message "NO".

```
;Check busy flag before sending data, command to LCD;p1=data pin ;P2.0 connected to RS
pin ;P2.1 connected to R/W pin ;P2.2 connected to E pin
ORG 0H
        MOV A,#38H              ;init. LCD 2 lines ,5x7 matrix
        ACALL COMMAND           ;issue command
        MOV A,#0EH              ;LCD on, cursor on
        ACALL COMMAND           ;issue command
        MOV A,#01H              ;clear LCD command
        ACALL COMMAND           ;issue command
        MOV A,#06H              ;shift cursor right
        ACALL COMMAND           issue command
        MOV A,#86H              ;cursor: line 1, pos. 6
        ACALL COMMAND           ;command subroutine
        MOV A,#'N'              ;display letter N
        ACALL DATA_DISPLAY
        MOV A,#'O'               ;display letter O
        ACALL DATA_DISPLAY
HERE: SJMP HERE          ;STAY HERE
COMMAND:
        ACALL READY             ;is LCD ready?
        MOV P1,A                 ;issue command code
        CLR P2.0                ;RS=0 for command
        CLR P2.1                ;R/W=0 to write to LCD
        SETB P2.2               ;E=1 for H-to-L pulse
        CLR P2.2                ;E=0,latch in
RET
DATA_DISPLAY:
        ACALL READY             ;is LCD ready?
        MOV P1,A                ;issue data
        SETB P2.0               ;RS=1 for data
        CLR P2.1                ;R/W =0 to write to LCD
        SETB P2.2               ;E=1 for H-to-L pulse
        CLR P2.2                ;E=0,latch in
RET
READY:
        SETB P1.7                ;make P1.7 input port
        CLR P2.0                 ;RS=0 access command reg
        SETB P2.1               ;R/W=1 read command reg ;
 BACK:SETB P2.2                 ;E=1 for H-to-L pulse
        CLR P2.2                ;E=0 H-to-L pulse
        JB P1.7,BACK            ;stay until busy flag=0
RET
END
```

Programming LCD in C

Example 13: Write an 8051 C program to send letters ‘P’, ‘I’, and ‘C’ to the LCD using the busy flag method.

Solution:

```
#include <reg51.h>

sfr ldata = 0x90;                    //P1=LCD data pins

sbit rs = P2^0;

sbit rw = P2^1;

sbit en = P2^2;

sbit busy = P1^7;

void main(){

        lcdcmd(0x38);

        lcdcmd(0x0E);

        lcdcmd(0x01);

        lcdcmd(0x06);

        lcdcmd(0x86);                //line 1, position 6

        lcddata(‘P’);

        lcddata(‘I’);

        lcddata(‘C’);

}

void lcdcmd(unsigned char value){

        lcdready();                  //check the LCD busy flag

        ldata = value;               //put the value on the pins
```

```
void lcdready(){
        busy = 1;                       //make the busy pin at input
        rs = 0;
        rw = 1;
        while(busy==1){                 //wait here for busy flag
        en = 0;                         //strobe the enable pin
        MSDelay(1);
        en = 1;
}
}
void Msdelay(unsigned int itime){
        unsigned int i, j;
        for(i=0;i<itime;i++)
        for(j=0;j<1275;j++);
}
```

Keyboard Interfacing:

Keyboards are organized in a matrix of rows and columns. The CPU accesses both rows and columns through ports. Therefore, with two 8-bit ports, an 8 x 8 matrix of keys can be connected to a microprocessor. When a key is pressed, a row and a column make a contact. Otherwise, there is no connection between rows and columns. A 4x4 matrix connected to two ports. The rows are connected to an output port and the columns are connected to an input port.

Scanning and Identifying the Key:

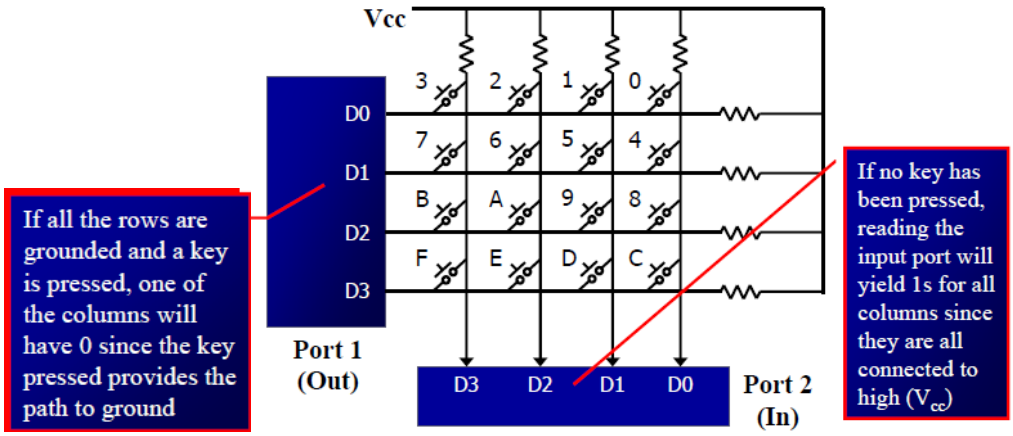


Figure 17: A 4X4 matrix keyboard

It is the function of the microcontroller to scan the keyboard continuously to detect and identify the key pressed

- To detect a pressed key, the microcontroller grounds all rows by providing 0 to the output latch, then it reads the columns
- If the data read from columns is D3 – D0 =1111, no key has been pressed and the process continues till key press is detected
- If one of the column bits has a zero, this means that a key press has occurred For example, if D3 – D0 = 1101, this means that a key in the D1 column has been pressed After detecting a key press, microcontroller will go through the process of identifying the key
- Starting with the top row, the microcontroller grounds it by providing a low to row D0 only. It reads the columns, if the data read is all 1s, no key in that row is activated and the process is moved to the next row
- It grounds the next row, reads the columns, and checks for any zero. This process continues until the row is identified.
- After identification of the row in which the key has been pressed. Find out which column the pressed key belongs to

Algorithm for detection and identification of key activation goes through the following stages:

1. To make sure that the preceding key has been released, 0s are output to all rows at once, and the columns are read and checked repeatedly until all the columns are high

- When all columns are found to be high, the program waits for a short amount of time before it goes to the next stage of waiting for a key to be pressed

2. To see if any key is pressed, the columns are scanned over and over in an infinite loop until one of them has a 0 on it

- Remember that the output latches connected to rows still have their initial zeros (provided in stage 1), making them grounded

- After the key press detection, it waits 20 ms for the bounce and then scans the columns again

  (a) It ensures that the first key press detection was not an erroneous one due a spike noise

  (b) The key press. If after the 20-ms delay the key is still pressed, it goes back into the loop to detect a real key press

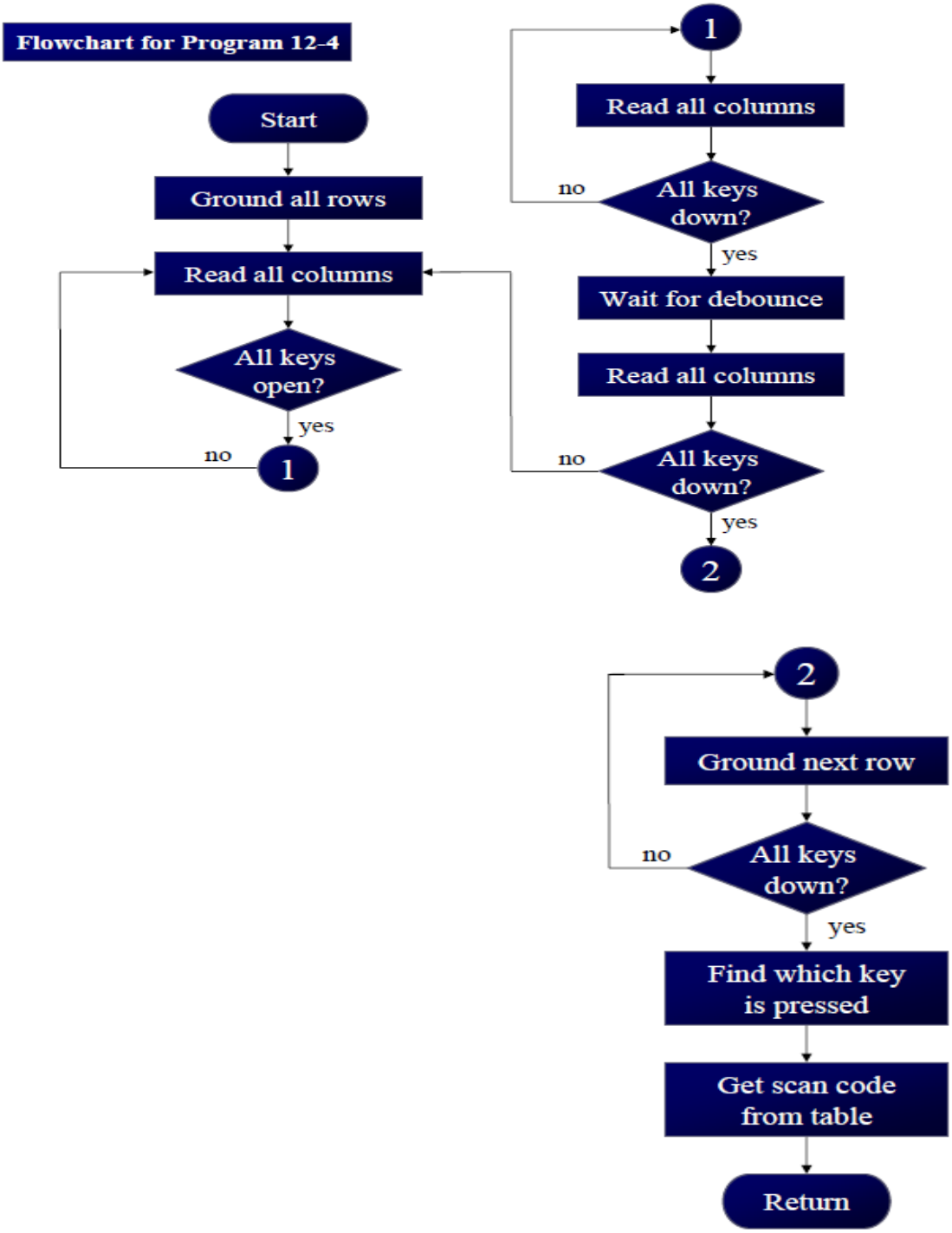
3. To detect which row key press belongs to, it grounds one row at a time, reading the columns each time

- If it finds that all columns are high, this means that the key press cannot belong to that row. Therefore, it grounds the next row and continues until it finds the row the key press belongs to
- Upon finding the row that the key press belongs to, it sets up the starting address for the look-up table holding the scan codes (or ASCII) for that row

4. To identify the key press, it rotates the column bits, one bit at a time, into the carry flag and checks to see if it is low

- Upon finding the zero, it pulls out the ASCII code for that key from the look-up table
- otherwise, it increments the pointer to point to the next element of the look-up table

The flowchart for the above algorithm is as shown below:



Note: The assembly as well as the C program can be written in accordance to the algorithm of the flowchart shown.

Summary

This chapter gives the details of six different devices that can be interfaced to 8051. These are widely used in many applications. Initially, we discussed about the stepper motor, giving the details on the working, sending sequence and hence writing assembly and C program. In continuation to that we also learnt how to interface DC motor, and DC motors with PWM. The chapter also covers the study of devices such as DAC, parallel ADC and serial ADC, LCD and Keyboard along with the interfacing of these devices to 8051. We further, studied how to write assembly and C program for all the above said interfaces which will help in developing applications.

Unit: 5: 8051 Interrupts and Timers/counters: Basics of interrupts, 8051 interrupt structure, Timers and Counters, 8051 timers/counters, programming 8051 timers in assembly and C.

The 8051 has two timers\counters. They can be used either as timers to generate time delay or as counters to count events happening outside the micro computer. Now we shall see how they are programmed.

PROGRAMMING 8051TIMERS:

8051 has two timers, timer 0 & timer 1.this module has two 16bit registers.T0 and T1 registers.These registers can be configured to operate either as timers or event counters. In the 'timer' function. The register is incremented every machine cycle. Thus, one can think it as counting machine cycles. Since a machine cycle consists of 12 oscillator periods, the count rate is 1/12 of the oscillator frequency.
The 16 bit register of T0 / T1 is accessed as low byte and high byte (TH0 / TH1)

TCON and TMOD Function Registers :

THE TIMER MODE CONTROL (TMOD) SPECIAL FUNCTION REGISTER

7	6	5	4	3	2	1	0
Gate	C/$\overline{T}$	M1	M0	Gate	C/$\overline{T}$	M1	M0
[Timer 1]				[Timer 0]			

Bit	Symbol	Function
7/3	Gate	OR gate enable bit which controls RUN/STOP of timer 1/0. Set to 1 by program to enable timer to run if bit TR1/0 in TCON is set and signal on external interrupt $\overline{INT1}$/0 pin is high. Cleared to 0 by program to enable timer to run if bit TR1/0 in TCON is set.
6/2	C/$\overline{T}$	Set to 1 by program to make timer 1/0 act as a counter by counting pulses from external input pins 3.5 (T1) or 3.4 (T0). Cleared to 0 by program to make timer act as a timer by counting internal frequency.
5/1	M1	Timer/counter operating mode select bit 1. Set/cleared by program to select mode.
4/0	M0	Timer/counter operating mode select bit 0. Set/cleared by program to select mode.

M1	M0	Mode
0	0	0
0	1	1
1	0	2
1	1	3

TMOD is not bit addressable

THE TIMER CONTROL (TCON) SPECIAL FUNCTION REGISTER

7	6	5	4	3	2	1	0
TF1	TR1	TFO	TRO	IE1	IT1	IEO	ITO

Bit	Symbol	Function
7	TF1	Timer 1 Overflow flag. Set when timer rolls from all ones to zero. Cleared when processor vectors to execute interrupt service routine located at program address 001Bh.
6	TR1	Timer 1 run control bit. Set to 1 by program to enable timer to count; cleared to 0 by program to halt timer. Does not reset timer.
5	TF0	Timer 0 Overflow flag. Set when timer rolls from all ones to zero. Cleared when processor vectors to execute interrupt service routine located at program address 000Bh.
4	TR0	Timer 0 run control bit. Set to 1 by program to enable timer to count; cleared to 0 by program to halt timer. Does not reset timer.
3	IE1	External interrupt 1 edge flag. Set to 1 when a high to low edge signal is received on port 3 pin 3.3 ($\overline{INT1}$). Cleared when processor vectors to interrupt service routine located at program address 0013h. Not related to timer operations.
2	IT1	External interrupt 1 signal type control bit. Set to 1 by program to enable external interrupt 1 to be triggered by a falling edge signal. Set to 0 by program to enable a low level signal on external interrupt 1 to generate an interrupt.
1	IE0	External interrupt 0 edge flag. Set to 1 when a high to low edge signal is received on port 3 pin 3.2 ($\overline{INT0}$). Cleared when processor vectors to interrupt service routine located at program address 0003h. Not related to timer operations.
0	IT0	External interrupt 0 signal type control bit. Set to 1 by program to enable external interrupt 0 to be triggered by a falling edge signal. Set to 0 by program to enable a low level signal on external interrupt 0 to generate an interrupt.

Bit addressable as TCON.0 to TCON.7

Counters and Timers

Many microcontroller applications require the counting of external events, such as the frequency of a pulse train, or the generation of precise internal time delays between computer actions. Both of these tasks can be accomplished using software techniques, but software loops for counting or timing keep the processor occupied so that other, perhaps more important, functions are not done. To relieve the processor of this burden, two 16-bit *up* counters, named T0 and T1, are provided for the general use of the programmer. Each counter may be programmed to count internal clock pulses, acting as a timer, or programmed to count external pulses as a counter.

The counters are divided into two 8-bit registers called the timer low (TL0, TL1) and high (TH0, TH1) bytes. All counter action is controlled by bit states in the timer mode control register (TMOD), the timer/counter control register (TCON), and certain program instructions.

TMOD is dedicated solely to the two timers and can be considered to be two duplicate 4-bit registers, each of which controls the action of one of the timers. TCON has control bits and flags for the timers in the upper nibble, and control bits and flags for the external interrupts in the lower nibble. Figure 2.10 shows the bit assignments for TMOD and TCON.

Timer Counter Interrupts

The counters have been included on the chip to relieve the processor of timing and counting chores. When the program wishes to count a certain number of internal pulses or external events, a number is placed in one of the counters. The number represents the maximum count *less* the desired count, plus one. The counter increments from the initial number to the maximum and then rolls over to zero on the final pulse and also sets a timer flag. The flag condition may be tested by an instruction to tell the program that the count has been accomplished, or the flag may be used to interrupt the program.

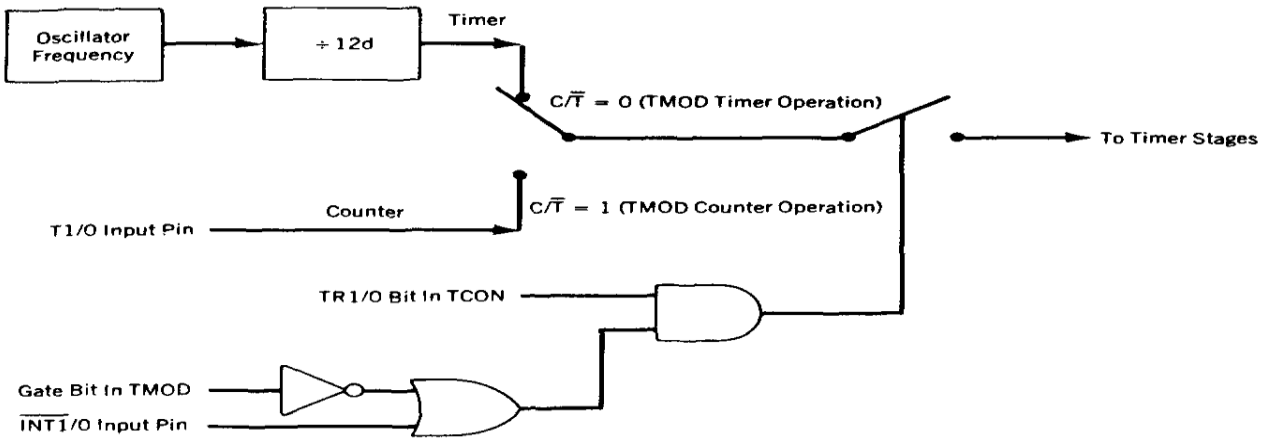


FIGURE 2.11 Timer/Counter Control Logic

Timing

If a counter is programmed to be a timer, it will count the internal clock frequency of the 8051 oscillator divided by 12d. As an example, if the crystal frequency is 6.0 megahertz, then the timer clock will have a frequency of 500 kilohertz.

The resultant timer clock is gated to the timer by means of the circuit shown in Figure 2.11. In order for oscillator clock pulses to reach the timer, the C/$\overline{T}$ bit in the TMOD register must be set to 0 (timer operation). Bit TRX in the TCON register must be set to 1 (timer run), and the *gate* bit in the TMOD register must be 0, or external pin $\overline{INTX}$ must be a 1. In other words, the counter is configured as a timer, then the timer pulses are gated to the counter by the run bit *and* the gate bit *or* the external input bits $\overline{INTX}$.

Timer Modes of Operation

The timers may operate in any one of four modes that are determined by the mode bits, M1 and M0, in the TMOD register. Figure 2.12 shows the four timer modes.

Timer Mode 0

Setting timer X mode bits to 00b in the TMOD register results in using the THX register as an 8-bit counter and TLX as a 5-bit counter; the pulse input is divided by 32d in TL so that TH counts the original oscillator frequency reduced by a total 384d. As an example, the 6 megahertz oscillator frequency would result in a final frequency to TH of 15625 hertz. The timer flag is set whenever THX goes from FFh to 00h, or in .0164 seconds for a 6 megahertz crystal if THX starts at 00h.

Timer Mode 1

Mode 1 is similar to mode 0 except TLX is configured as a full 8-bit counter when the mode bits are set to 01b in TMOD. The timer flag would be set in .1311 seconds using a 6 megahertz crystal.

Timer Mode 2

Setting the mode bits to 10b in TMOD configures the timer to use only the TLX counter as an 8-bit counter. THX is used to hold a value that is loaded into TLX every time TLX overflows from FFh to 00h. The timer flag is also set when TLX overflows.

This mode exhibits an auto-reload feature: TLX will count up from the number in THX, overflow, and be initialized again with the contents of THX. For example, placing 9Ch in THX will result in a delay of exactly .0002 seconds before the overflow flag is set if a 6 megahertz crystal is used.

Timer Mode 3

Timers 0 and 1 may be programmed to be in mode 0, 1, or 2 independently of a similar mode for the other timer. This is not true for mode 3; the timers do not operate independently if mode 3 is chosen for timer 0. Placing timer 1 in mode 3 causes it to stop counting; the control bit TR1 and the timer 1 flag TF1 are then used by timer 0.

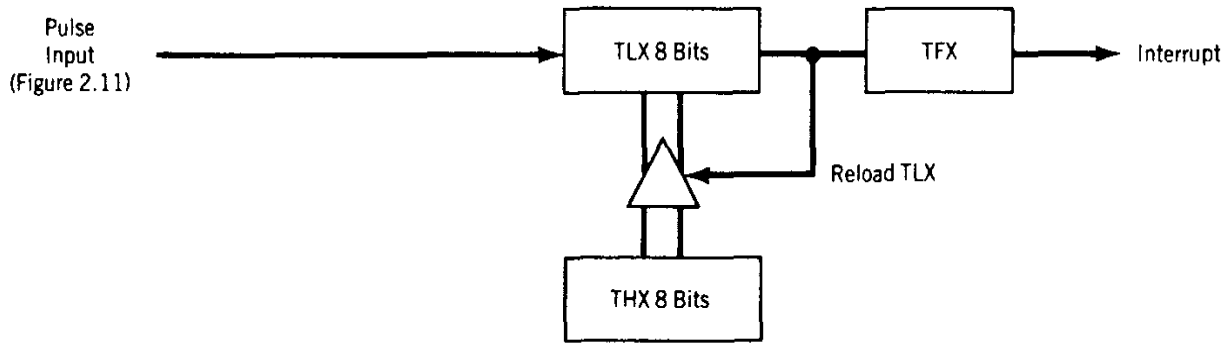
Timer 0 in mode 3 becomes two completely separate 8-bit counters. TL0 is controlled by the gate arrangement of Figure 2.11 and sets timer flag TF0 whenever it overflows from FFh to 00h. TH0 receives the timer clock (the oscillator divided by 12) under the control of TR1 only and sets the TF1 flag when it overflows.

Timer 1 may still be used in modes 0, 1, and 2, while timer 0 is in mode 3 with one important exception: *No interrupts* will be generated by timer 1 while timer 0 is using the TF1 overflow flag. Switching timer 1 to mode 3 will stop it (and hold whatever count is in timer 1). Timer 1 can be used for baud rate generation for the serial port, or any other mode 0, 1, or 2 function that does not depend upon an interrupt (or any other use of the TF1 flag) for proper operation.

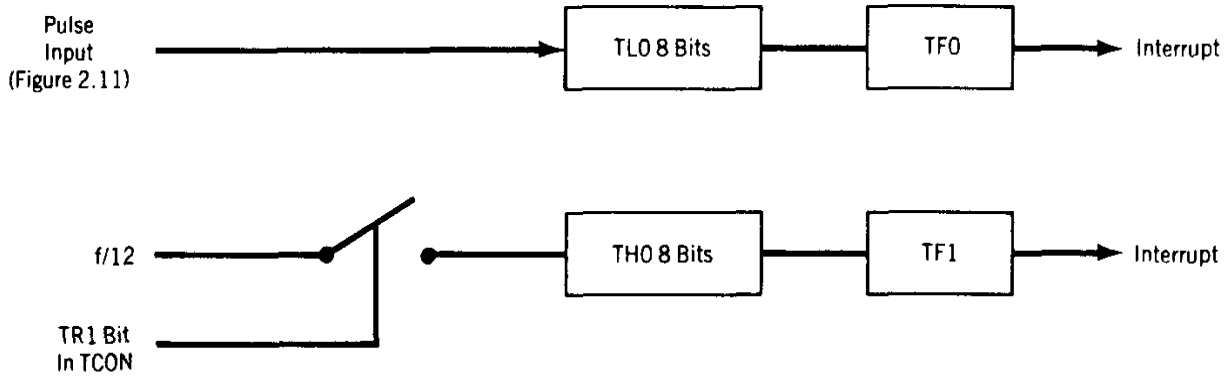
FIGURE 2.12 Timer 1 and Timer 0 Operation Modes

Timer Mode 0 13 - Bit Timer/Counter

Timer Mode 1 16 - Bit Timer/Counter



Timer Mode 2 Auto - Reload of TL from TH



Timer Mode 3 Two 8 - Bit Timers Using Timer 0

Counting

The only difference between counting and timing is the source of the clock pulses to the counters. When used as a timer, the clock pulses are sourced from the oscillator through the divide-by-12d circuit. When used as a counter, pin T0 (P3.4) supplies pulses to counter 0, and pin T1 (P3.5) to counter 1. The C/$\overline{T}$ bit in TMOD must be set to 1 to enable pulses from the TX pin to reach the control circuit shown in Figure 2.11.

The input pulse on TX is sampled during P2 of state 5 every machine cycle. A change on the input from high to low between samples will increment the counter. Each high and low state of the input pulse must thus be held constant for at least one machine cycle to ensure reliable counting. Since this takes 24 pulses, the maximum input frequency that can be accurately counted is the oscillator frequency divided by 24. For our 6 megahertz crystal, the calculation yields a maximum external frequency of 250 kilohertz.

Interrupts

Concept of Interrupt:

A computer has only two ways to determine the conditions that exist in internal and external circuits. One method uses software instructions that jump to subroutines on the states of flags and port pins. The second method responds to hardware signals, called interrupts that force the program to call a subroutine. Most applications of microcontroller involve responding to events quickly enough to control the environment that generates the events termed real-time programming.

Interrupts may be generated by internal chip operation or provided by external sources. Any interrupt can cause the 8051 to perform a hardware call to an interrupt-handling subroutine that is located at a predetermined absolute address in program memory.

The 8051 has five interrupts of which three are internally generated namely:

1. **Timer 0 overflow:** This is indicated by TF0 in TCON, being set
2. **Timer 1 overflow:** This is indicated by TF1 in TCON, being set
3. **Serial port interrupts (RI and TI):** Whenever a data byte is received, an interrupt bit, RI is set to 1 in SCON register. When a data byte is transmitted an interrupt bit TI, is set in SCON. They are ORed together to provide a single interrupt to the processor. These flags must be reset by software instruction to enable the next data communication operation.

Two interrupts are triggered by external signals provided by circuitry that is connected to pins INTO and INT1 (P3.2 and P3.3).

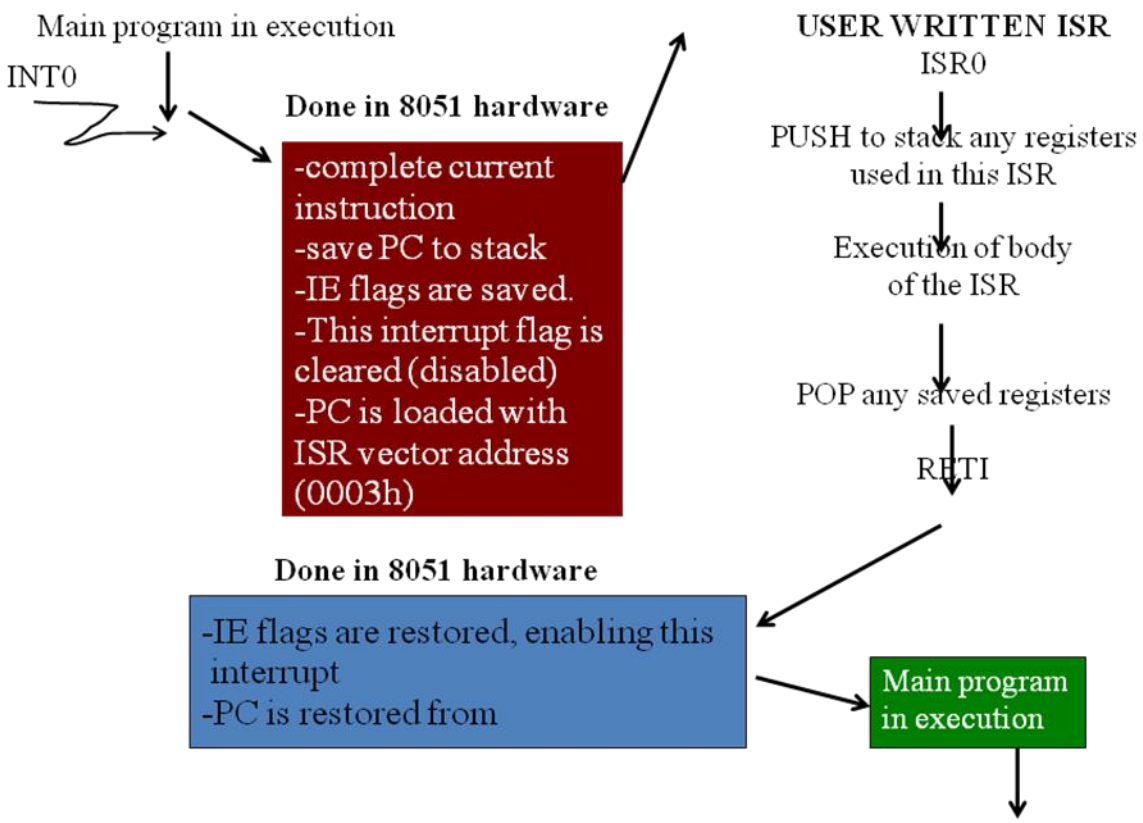
1. **External signal at pin INTO (P3.2):** When a high-to-low edge signal is received on P3.2, the external interrupt 0 edge flag IE0 (TCON.1) is set. This flag is cleared when the processor branches to the subroutine. When the external interrupt signal control bit IT0 (TCON.0) is set to 1 (by program) then interrupt is triggered by falling edge signal. If IT0 is 0, a low-level signal in INTO triggers the interrupt.
2. **External signal at pin INT1 (P3.3):** Flags IE1 (TCON.3) and IT1 (TCON.2) are similar to IE0 and IT0 in function.

Each of these interrupts has an address associated where the routine is to be written called as interrupt service routine addresses. The addresses are listed below:

Interrupt	Address called
IE0	0003
TF0	000B
IE1	0013
TF1	001B
Serial	0023

Sequence of events

The sequence of events that take place on the occurrence of an interrupt is as shown below:



An observation made from the above diagram can be explained by the following steps:

1. The programmer enables interrupt circuit action by setting interrupt enable flag bit to 1. The 8051 has a total of five interrupt sources of each of which may generate an interrupt signal.
2. External or internal circuit action causes one of interrupt signals to be generated.

3. The CPU finishes the current instruction, pushes the PC on the stack, and replaces the original PC contents with the address of the first instruction of the program code for the particular source that caused the interrupt. All the other interrupting source enable bits are temporarily disabled.
4. The interrupt program executes. While executing, the interrupt program must reset internal flag that generated the interrupt signals.
5. At the end of the interrupt program, a RETI instruction resets all the interrupt-enable circuitry and pops the original PC contents from the stack back into the PC. The CPU resumes executing the interrupted program.

Note that if the interrupting signal is not reset before a RETI instruction, the same interrupt will occur again. This process will never stop, and the program will loop forever at the interrupt program location.

The difference between RET and RETI is RET is a return from a function or a subroutine while RETI is return from an interrupt. The RETI instruction is executed at the end of interrupt subroutine. After the execution of the RETI instruction the PC address will be restored from the stack.

Thus the comparison of the call instruction and the interrupt action can be as:

Table 1: Comparison of Call Instruction and Interrupt Action

Call Instruction	**Interrupt Action**
Completely under the control of programmer	Occurs at any time in the program
Placed in the program by the programmer	Enabled by the programmer
Execution is determined by where it is placed in the program	Calls the subroutine at any time and any place the program is executed

More Details on Interrupts:

The interrupts of 8051 can be programmed and serviced by the microcontroller using the SFRs Interrupt Enable (IE) and Interrupt Priority (IP).

Interrupt Enable (IE) SFR:

This is a bit addressable SFR with byte address A8H. The bits and addresses are shown in table 2. The bits are explained below.

Table 2: Interrupt Enable SFR

IE.7	IE.6	IE.5	IE.4	IE.3	IE.2	IE.1	IE.0
EA	-	ET2	ES	ET1	EX1	ET0	EX0
AF	AE	AD	AC	AB	AA	A9	A8

EA: This bit is a global interrupt enable/disable bit. When set to 1, it permits individual interrupts to be enable by their respective enable bits. When 0, it disables all interrupts.

IE.6: Not implemented

ET2: Reserved for future use.

ES: Enable serial port interrupt. Set to 1 by program to enable serial port interrupt. Cleared to 0 to disable serial port interrupt.

ET1: Enable (=1)/disable (=0) timer 1 interrupt

EX1: Enable (=1)/disable (=0) external interrupt 1

ET0: Enable (=1)/disable (=0) timer 0 interrupt

ET1: Enable (=1)/disable (=0) external interrupt 0

Interrupt Priority (IP):

This a bit addressable register, with byte address B8H. The addresses are shown in table 3. The priority of the interrupts is determined by the bits of IP SFR. The bits which are set to 1, have a high priority and bits with 0 have low priority. Interrupts with high priority can interrupt another interrupt with low priority. The lower priority interrupt is serviced after higher priority interrupt is finished.

Table 3: Interrupt Priority (IP)

IP.7	IP.6	IP.5	IP.4	IP.3	IP.2	IP.1	IP.0
-	-	PT2	PS	PT1	PX1	PT0	PX0
-	-	-	BC	BB	BA	B9	B8

IP.7 & IP.6: Not implemented
IP.5: Reserved for future use
PS: Serial port priority interrupt
PT1: Priority of timer 1 interrupt
PX1: Priority of external interrupt 1
PT0: Priority of timer 0 interrupt
PX0: Priority of external interrupt 0

When two or more interrupts have the same priority the microcontroller has its own ranking of providing service which is a below:

1. External Interrupt 0 (P3.2)
2. Timer 0 Overflow
3. External Interrupt 0 (P3.3)
4. Timer 1 Overflow
5. Serial port

UNIT 6:

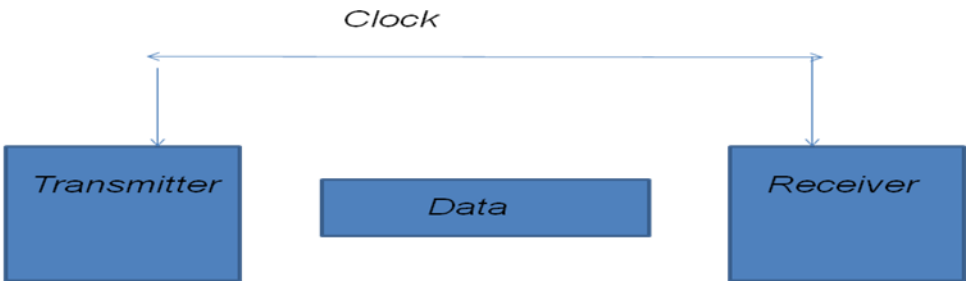
8051 Serial Communication: Data communication, Basics of Serial Data Communication, 8051 Serial Communication, connections to RS-232, Serial communication Programming in assembly and C.
8255A Programmable Peripheral Interface:, Architecture of 8255A, I/O addressing,, I/O devices interfacing with 8051 using 8255A.

8051 SERIAL COMMUNICATION

Types of Serial Communication

1. Synchronous serial Data Communication
2. Asynchronous Serial Data Communication

Pins TxD (P3.1) and RxD (P3.0) are used for transmitting and receiving the data serially. Figure below shows synchronous serial data communication which uses a common clock for synchronization of transmitter and receiver



Serial Interface

The serial port of 8051 is full duplex, i.e., it can transmit and receive simultaneously. The register SBUF is used to hold the data. The special function register SBUF is physically two registers. One is, write-only and is used to hold data to be transmitted out of the 8051 via TXD. The other is, read-only and holds the received data from external sources via RXD. Both mutually exclusive registers have the same address 099H.

Serial Port Control Register (SCON)

Register SCON controls serial data communication. Address: 098H (Bit addressable)

SM0	SM1	SM2	REN	TB8	RB8	TI	RI

Mode select bits

SM0	SM1	MODE
0	0	Mode0
0	1	Mode1
1	0	Mode2
1	1	M de3

SM2: used for multiprocessor communication.
REN: set or cleared by software to enable/disable reception.
TB8: Transmitted bit 8,not widely used.
RB8: Received bit 8.
TI: Transmit interrupt flag –set by the hardware at the beginning of the stop bit in mode 1, must be cleared by software.
RI: Receive interrupt flag –set by the hardware halfway through the stop bit time in mode1, must be cleared by software.

SCON Register
Serial control register: SCON
SM0, SM1 : Serial port mode specifier
REN : (Receive enable) set/cleared by software to enable/disable reception.
TI : Transmit interrupt flag.
RI : Receive interrupt flag.
SM2 = RB8 = TB8 =0 (not widely used)
REN (Receive Enable) -SCON.4
Set/cleared by software to enable/disable reception.
REN=1

- It enable the 8051 to receive data on the RxD pin of the 8051.
- If we want the 8051 to both transfer and receive data, REN must be set to 1.

SETB SCON.4
REN=0
The receiver is disabled.
The 8051 cannot receive data.
CLR SCON.4
SM0, SM1
SM1 and SM0 determine the framing of data.
SCON.6 (SM1) and SCON.7 (SM0)
Only mode 1 is compatible with COM port of PC.

SM1	SM0	Mode	Operating Mode	Baud Rate
0	0	0	Shift register	Fosc./12
0	1	1	8-bit UART	Variable by Tmer1
1	0	2	9-bit UART	Fosc./64 or Fosc./32
1	1	3	9-bit UART	Variable

TB8 (Transfer Bit 8) -SCON.3

- TB8 is used for serial modes 2 and 3.
- The 9th bit that will be transmitted in mode 2 & 3.
- Set/Cleared by software

RB8 (Receive Bit 8) -SCON.2

In serial mode 1, RB8 gets a copy of the stop bit when an 8-bit data is received.

TI (Transmit Interrupt Flag) -SCON.1
When the 8051 finishes the transfer of the 8-bit character, it raises the TI flag.
TI is raised by hardware at the beginning of the stop bit in mode 1.
Must be cleared by software

RI (Receive Interrupt) -SCON.0
Receive interrupt flag. Set by hardware halfway through the stop bit time in mode 1. Must be cleared by software.

When the 8051 receives data serially via RxD, it gets rid of the start and stop bits and place the byte in the SBUF register.
Then 8051 rises RI to indicate that a byte.
RI is raised at the beginning of the stop bit.

Power Mode control Register

Register PCON controls processor power down, sleep modes and serial data baud rate. Only one bit of PCON is used with respect to serial communication. The seventh bit (b7)(SMOD) is used to generate the baud rate of serial communication.

Address: 87H

b7							b0
SMOD	—	—	—	GF1	GF0	PD	IDL

SMOD: Serial baud rate modify bit
GF1: General purpose user flag bit 1
GF0: General purpose user flag bit 0
PD: Power down bit
IDL: Idle mode bit

Data Transmission

Transmission of serial data begins at any time when data is written to SBUF. Pin P3.1 (Alternate function bit TXD) is used to transmit data to the serial data network. TI is set to 1 when data has been transmitted. This signifies that SBUF is empty so that another byte can be sent.

Data Reception

Reception of serial data begins if the receive enable bit is set to 1 for all modes. Pin P3.0 (Alternate function bit RXD) is used to receive data from the serial data network. Receive interrupt flag, RI, is set after the data has been received in all modes. The data gets stored in SBUF register from where it can be read.

Serial Data Transmission Modes:

Mode-0**:** In this mode, the serial port works like a shift register and the data transmission works synchronously with a clock frequency of f_{osc} /12. Serial data is received and transmitted through RXD. 8 bits are transmitted/ received aty a time. Pin TXD outputs the shift clock pulses of frequency f_{osc} /12, which is connected to the external circuitry for synchronization. The shift frequency or baud rate is always 1/12 of the oscillator frequency.

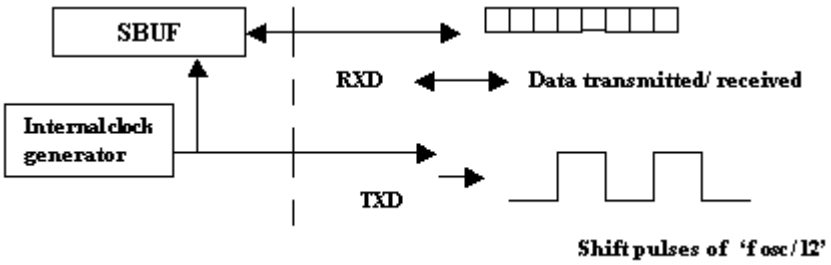


Fig : Data transmission/reception in Mode-0

In mode-1, the serial port functions as a standard Universal Asynchronous Receiver Transmitter (UART) mode. 10 bits are transmitted through TXD or received through RXD. The 10 bits consist of one start bit (which is usually '0'), 8 data bits (LSB is sent first/received first), and a stop bit (which is usually '1'). Once received, the stop bit goes into RB8 in the special function register SCON. The baud rate is variable.

The following figure shows the way the bits are transmitted/ received.

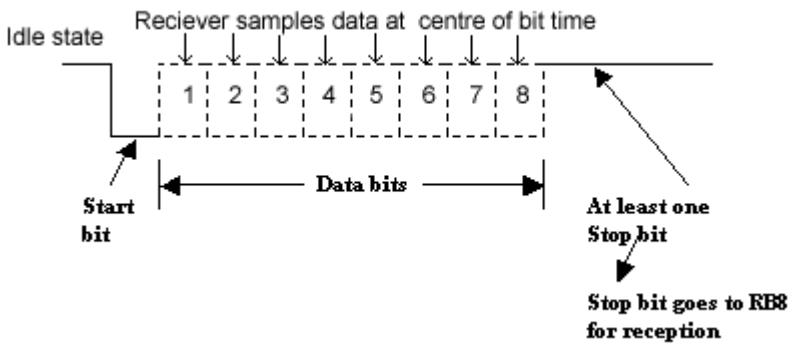


Fig : Data transmission format in UART mode

Bit time= $1/f_{baud}$

In receiving mode, data bits are shifted into the receiver at the programmed baud rate. The data word (8-bits) will be loaded to SBUF if the following conditions are true.

1. RI must be zero. (i.e., the previously received byte has been cleared from SBUF)
2. Mode bit SM2 = 0 or stop bit = 1.

After the data is received and the data byte has been loaded into SBUF, RI becomes one.

Mode-1 baud rate generation:

Timer-1 is used to generate baud rate for mode-1 serial communication by using overflow flag of the timer to determine the baud frequency. Timer-1 is used in timer mode-2 as an auto-reload 8-bit timer. The data rate is generated by timer-1 using the following formula.

$$f_{baud} = \frac{2^{SMOD}}{32} X \frac{fosc}{12 X [256-(TH1)]}$$

Where,

SMOD is the 7th bit of PCON register

f_{osc} is the crystal oscillator frequency of the microcontroller

It can be noted that f_{osc}/ (12 X [256- (TH1)]) is the timer overflow frequency in timer mode-2, which is the auto-reload mode.

If timer-1 is not run in mode-2, then the baud rate is,

$$f_{baud} = \frac{2^{SMOD}}{32} X \text{ (timer-1 overflow frequency)}$$

Timer-1 can be run using the internal clock, fosc/12 (timer mode) or from any external source via pin T1 (P3.5) (Counter mode).

Serial Data Mode-2 - Multiprocessor Mode :

In this mode 11 bits are transmitted through TXD or received through RXD. The various bits are as follows: a start bit (usually '0'), 8 data bits (LSB first), a programmable 9 th (TB8 or RB8)bit and a stop bit (usually '1').

While transmitting, the 9 th data bit (TB8 in SCON) can be assigned the value '0' or '1'. For example, if the information of parity is to be transmitted, the parity bit (P) in PSW could be moved into TB8. On reception of the data, the 9 th bit goes into RB8 in 'SCON', while the stop bit is ignored. The baud rate is programmable to either 1/32 or 1/64 of the oscillator frequency.

Mode-3 - Multi processor mode with variable baud rate :

In this mode 11 bits are transmitted through TXD or received through RXD. The various bits are: a start bit (usually '0'), 8 data bits (LSB first), a programmable 9 th bit and a stop bit (usually '1'). Mode-3 is same as mode-2, except the fact that the baud rate in mode-3 is variable (i.e., just as in mode-1).

$$f_{baud} = (2^{SMOD}/32) * (f_{osc} / 12 (256-TH1)) .$$

This baudrate holds when Timer-1 is programmed in Mode-2.

Programming the 8051 to transfer data serially

Write a program for the 8051 to transfer letter "A" serially at 4800baud, continuously.

```
        MOV  TMOD,#20H      ;timer 1, mode 2
        MOV  TH1,#-6        ;4800 baud rate
        MOV  SCON,#50H      ;8-bit,1 stop,REN enabled
        SETB TR1            ;start timer 1
AGAIN:  MOV  SBUF,#"A"      ;letter "A" to be transferred
HERE:   JNB  TI,HERE        ;wait for the last bit
        CLR  TI             ;clear TI for next char
        SJMP AGAIN          ;keep sending A
```

Write a program to transfer the message "YES" serially at 9600 baud, 8-bit data, 1 stop bit. Do this continuously.

```
        MOV  TMOD,#20H      ;timer 1, mode 2
        MOV  TH1,#-3        ;9600 baud
        MOV  SCON,#50H
        SETB TR1
AGAIN:  MOV  A,#"Y"         ;transfer "Y"
        ACALL TRANS
        MOV  A,#"E"         ;transfer "E"
        ACALL TRANS
        MOV  A,#"S"         ;transfer "S"
        ACALL TRANS
        SJMP AGAIN          ;keep doing it
```

;serial data transfer subroutine

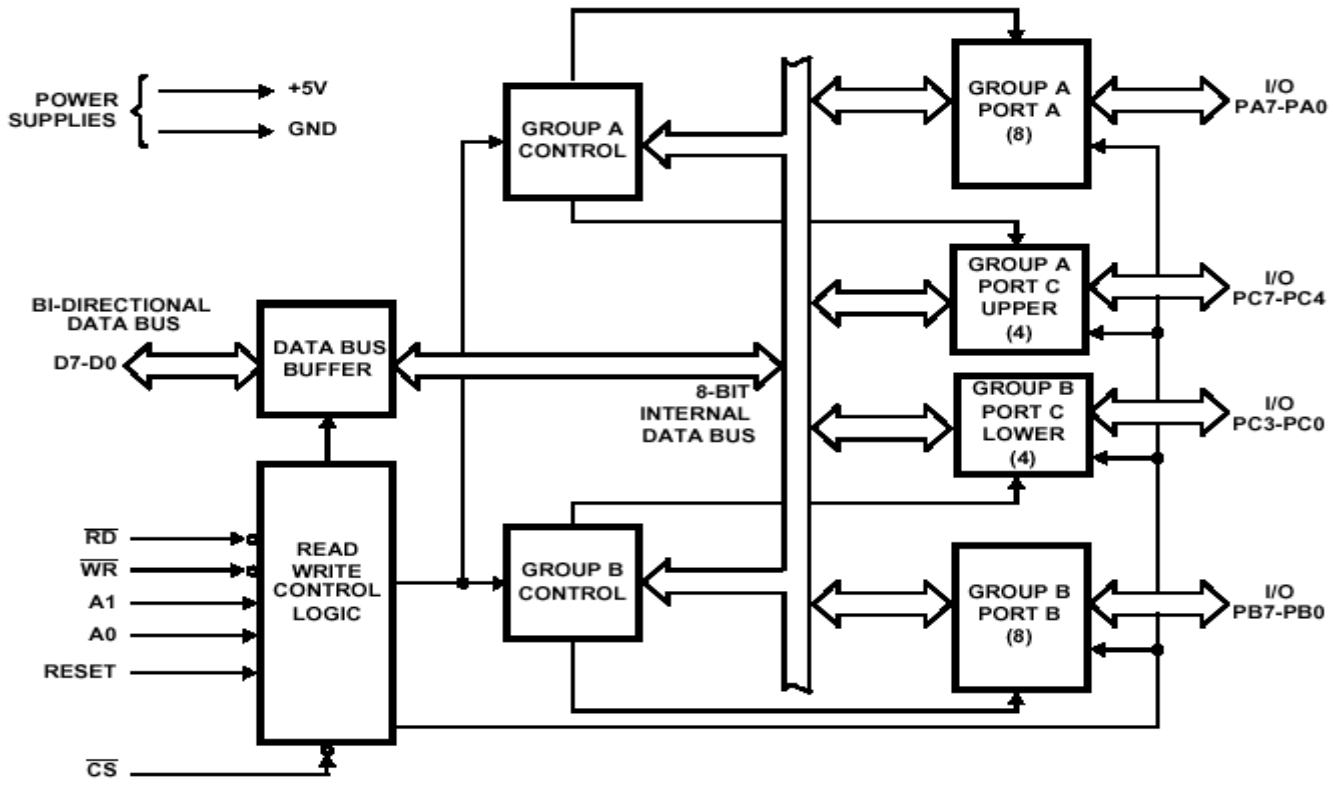
```
TRANS:  MOV   SBUF,A        ;load SBUF
HERE:   JNB   TI,HERE       ;wait for last bit to transfer
        CLR   TI            ;get ready for next byte
        RET
```

Baud Rates in the 8051

- Timer 1, mode 2 (8-bit, auto-reload)
- Define TH1 to set the baud rate.

XTAL = 11.0592 MHz
The system frequency = 11.0592 MHz / 12 = 921.6 kHz
Timer 1 has 921.6 kHz/ 32 = 28,800 Hz as source.
TH1=FDH means that UART sends a bit every 3 timer source.
Baud rate = 28,800/3= 9,600 Hz
Example
With XTAL = 11.0592 MHz, find the TH1 value needed to have the following baud rates. (a) 9600 (b) 2400 (c) 1200
Solution:
With XTAL = 11.0592 MHz, we have:
The frequency of system clock = 11.0592 MHz / 12 = 921.6 kHz
The frequency sent to timer 1 = 921.6 kHz/ 32 = 28,800 Hz
(a) 28,800 / 3 = 9600 where -3 = FD (hex) is loaded into TH1
(b) 28,800 / 12 = 2400 where -12 = F4 (hex) is loaded into TH1
(c) 28,800 / 24 = 1200 where -24 = E8 (hex) is loaded into TH1
Registers Used in Serial Transfer Circuit
SUBF (Serial data buffer)
SCON (Serial control register)
PCON (Power control register)
SBUF Register
Serial data register: **SBUF**
MOV SBUF,#'A' ;put char 'A' to transmit
MOV SBUF,A ;send data from A
MOV A,SUBF ;receive and copy to A
An 8-bit register
Set the usage mode for two timers
For a byte of data to be transferred via the TxD line, it must be placed in the SBUF.
SBUF holds the byte of data when it is received by the 8051's RxD line.

Block Diagram of the 8255 Programmable Peripheral Interface (PPI)



Data Bus Buffer

This three-state bi-directional 8-bit buffer is used to interface the 8255 to the system data bus. Data is transmitted or received by the buffer upon execution of input or output instructions by the CPU. Control words and status informa-tion are also transferred through the data bus buffer.

Read/Write and Control Logic

The function of this block is to manage all of the internal and external transfers of both Data and Control or Status words. It accepts inputs from the CPU Address and Control busses and in turn, issues commands to both of the Control Groups.

(CS) Chip Select. A "low" on this input pin enables the communcation between the 8255 and the CPU.

(RD) Read. A "low" on this input pin enables 8255 to send the data or status information to the CPU on the data bus. In essence, it allows the CPU to "read from" the 8255.

(WR) Write. A "low" on this input pin enables the CPU to write data or control words into the 8255.

(A0 and A1) Port Select 0 and Port Select 1. These input signals, in conjunction with the RD and WR inputs, control the selection of one of the three ports or the control word register. They are normally connected to the least significant bits of the address bus (A0 and A1).

(RESET) Reset. A "high" on this input initializes the control register to 9Bh and all ports (A, B, C) are set to the input mode.

A1	A0	SELECTION
0	0	PORT A
0	1	PORT B
1	0	PORT C
1	1	CONTROL

Group A and Group B Controls

The functional configuration of each port is programmed by the systems software. In essence, the CPU "outputs" a control word to the 8255. The control word contains information such as "mode", "bit set", "bit reset", etc., that initializes the functional configuration of the 8255. Each of the Control blocks (Group A and Group B) accepts "commands" from the Read/Write Control logic, receives "control words" from the internal data bus and issues the proper commands to its associated ports.

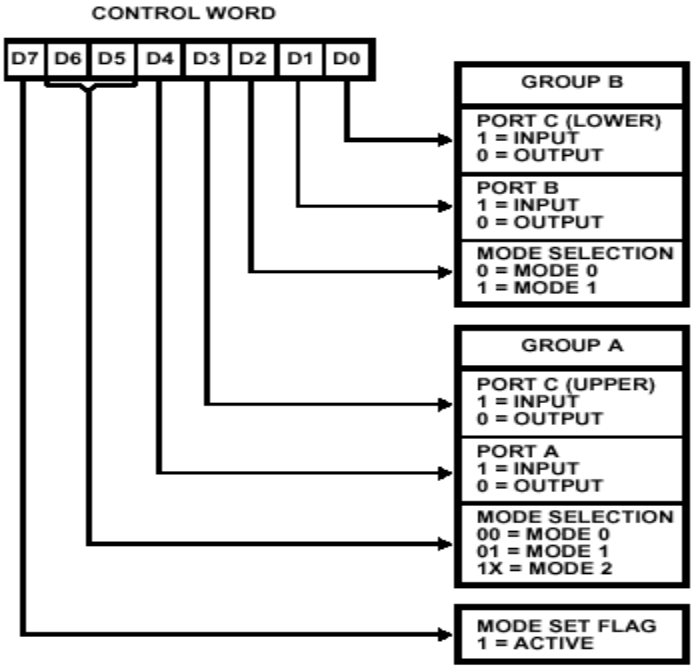
Ports A, B, and C

The 8255 contains three 8-bit ports (A, B, and C). All can be configured to a wide variety of functional characteristics by the system software but each has its own special features or "personality" to further enhances the power and flexibility of the 8255.

Port A One 8-bit data output latch/buffer and one 8-bit data input latch. Both "pull-up" and "pull-down" bus-hold devices are present on Port A.

Port B One 8-bit data input/output latch/buffer and one 8-bit data input buffer.

Port C One 8-bit data output latch/buffer and one 8-bit data input buffer (no latch for input). This port can be divided into two 4-bit ports under the mode control. Each 4-bit port contains a 4-bit latch and it can be used for the control signal output and status signal inputs in conjunction with ports A and B.



UNIT 7:

Motivation for MSP430microcontrollers – Low Power embedded systems, On-chip peripherals (analog and digital), low-power RF capabilities. Target applications (Single-chip, low cost, low power, high performance system design). 2 Hrs

MSP430 RISC CPU architecture, Compiler-friendly features, Instruction set, Clock system, Memory subsystem. Key differentiating factors between different MSP430 families. 2 Hrs

Introduction to Code Composer Studio (CCS v4). Understanding how to use CCS for Assembly, C, Assembly+C projects for MSP430 microcontrollers. Interrupt programming. 3 Hrs

Digital I/O – I/O ports programming using C and assembly, Understanding the muxing scheme of the MSP430 pins. 2 Hrs

Introduction

The MSP430 is a 16-bit microcontroller that has a number of special features not commonly available with other microcontrollers:

- Complete system on-a-chip — includes LCD control, ADC, I/O ports,ROM, RAM, basic timer, watchdog timer, UART, etc.
- Extremely low power consumption — only 4.2 nW per instruction, typical
- High speed — 300 ns per instruction @ 3.3 MHz clock, in register and register addressing mode
- RISC structure — 27 core instructions
- Orthogonal architecture (any instruction with any addressing mode)
- Seven addressing modes for the source operand
- Four addressing modes for the destination operand
- Constant generator for the most often used constants (–1, 0, 1, 2, 4, 8)
- Only one external crystal required — a frequency locked loop (FLL) oscillator derives all internal clocks
- Full real-time capability — stable, nominal system clock frequency is available after only six clocks when the MSP430 is restored from low-power mode (LPM) 3; — no waiting for the main crystal to begin oscillation and stabilize

The 27 core instructions combined with these special features make it easyto program the MSP430 in assembler or in C, and provide exceptional flexibility and functionality.

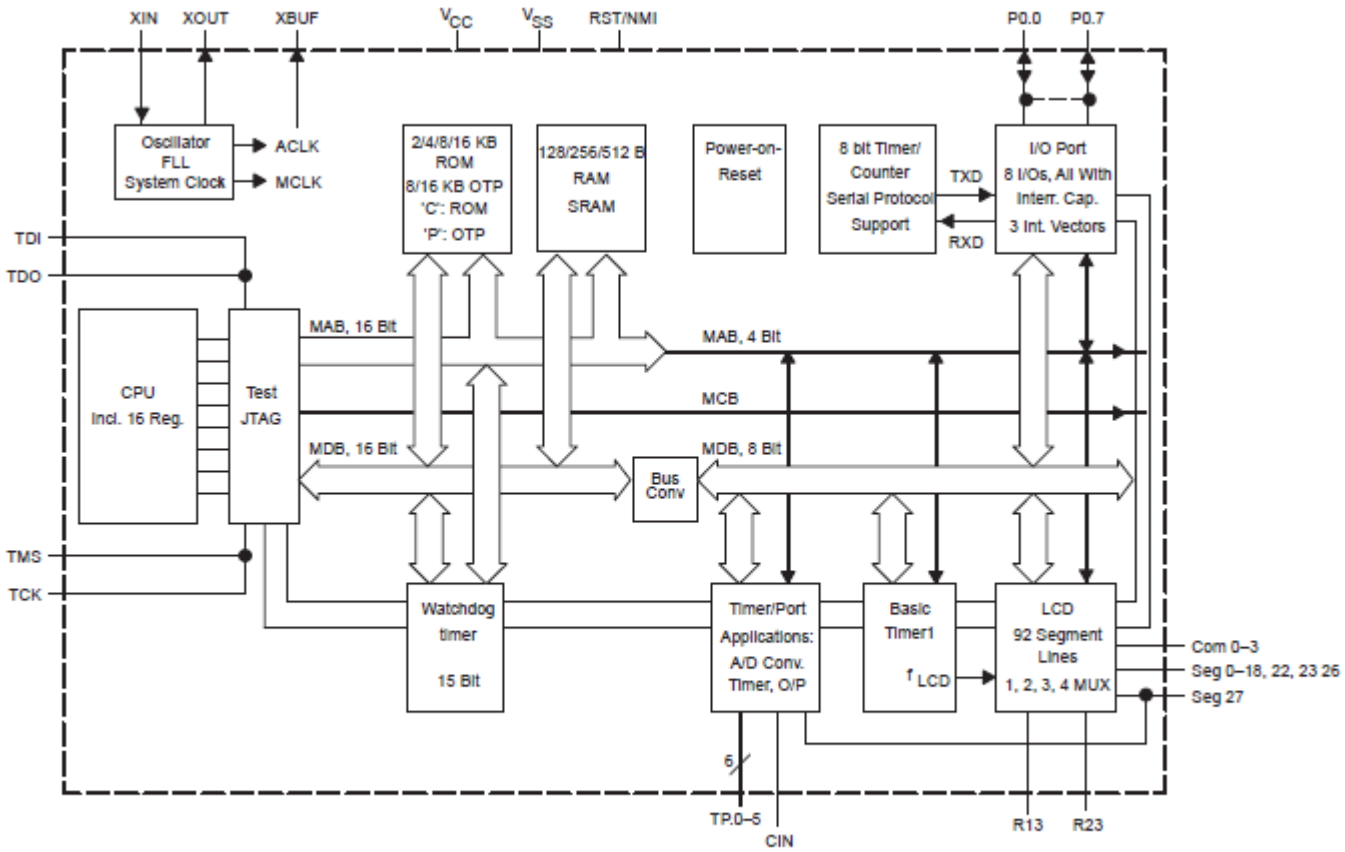


Figure 1–1. MSP430C31x Block Diagram

Advantages of the MSP430 Concept

The MSP430 concept differs considerably from other microcontrollers and offers some significant advantages over more traditional designs.

1.5.1 RISC Architecture without RISC Disadvantages

Typical RISC architectures show their highest performance in calculation- intensive applications in which several registers are loaded with input data, all calculations are made within the registers, and the results are stored back into RAM. Memory accesses (using addressing modes) are necessary only for the LOAD instructions at the beginning and the STORE instructions at the end of the calculations. The MSP430 can be programmed for such operation, for example, performing a pure calculation task in the floating point without any I/O accesses. Pure RISC architectures have some disadvantages when running real-time applications that require frequent I/O accesses, however. Time is lost whenever

an operand is fetched and loaded from RAM, modified, and then stored back into RAM. The MSP430 architecture was designed to include the best of both worlds, taking advantage of RISC features for fast and efficient calculations, and addressing modes for real-time requirements:

- The RISC architecture provides a limited number of powerful instructions, numerous registers, and single-cycle execution times.
- The more traditional microcomputer features provide addressing modes for all instructions. This functionality is further enhanced with 100% orthogonality, allowing any instruction to be used with any addressing mode.

1.5.2 Real-Time Capability with Ultra-Low Power Consumption

The design of the MSP430 was driven by the need to provide full real-time capability while still exhibiting extremely low power consumption. Average power consumption is reduced to the minimum by running the CPU and certain other functions of the MSP430 only when it is necessary. The rest of the time (the majority of the time), power is conserved by keeping only selected low-power peripheral functions active. But to have a true real-time capability, the device must be able to shift from a low-power mode with the CPU off to a fully active mode with the CPU and all other device functions operating nominally in a very short time. This was accomplished primarily with the design of the system clock:

No second high frequency crystal is used — inherent delays can range

from 20 ms to 200 ms until oscillator stability is reached

- Instead, a sophisticated FLL system clock generator is used — generator output frequency (MCLK) reaches the nominal frequency within 8 cycles after activation from low power mode 3 (LPM3) or sleep mode. This design provides real-time capability almost immediately after the device comes out of a LPM — as if the CPU is always active. Only two additional MCLK cycles (2 □ s @ fC = 1 MHz) are necessary to get the device from LPM3 to the first instruction of the interrupt handler.

1.5.3 Digitally Controlled Oscillator Stability

The digitally controlled oscillator (DCO) is voltage and temperature dependent, which does not mean that its frequency is not stable. During the active mode, the integral error is corrected to approximately zero every 30.5 □ s . Thisis accomplished by switching between two different DCO frequencies. One frequency is higher than the programmed MCLK frequency and the other is lower, causing the errors to essentially cancel-out. The two DCO frequencies are interlaced as much as possible to provide the smallest possible error at any given time. See System Clock Generator for more information.

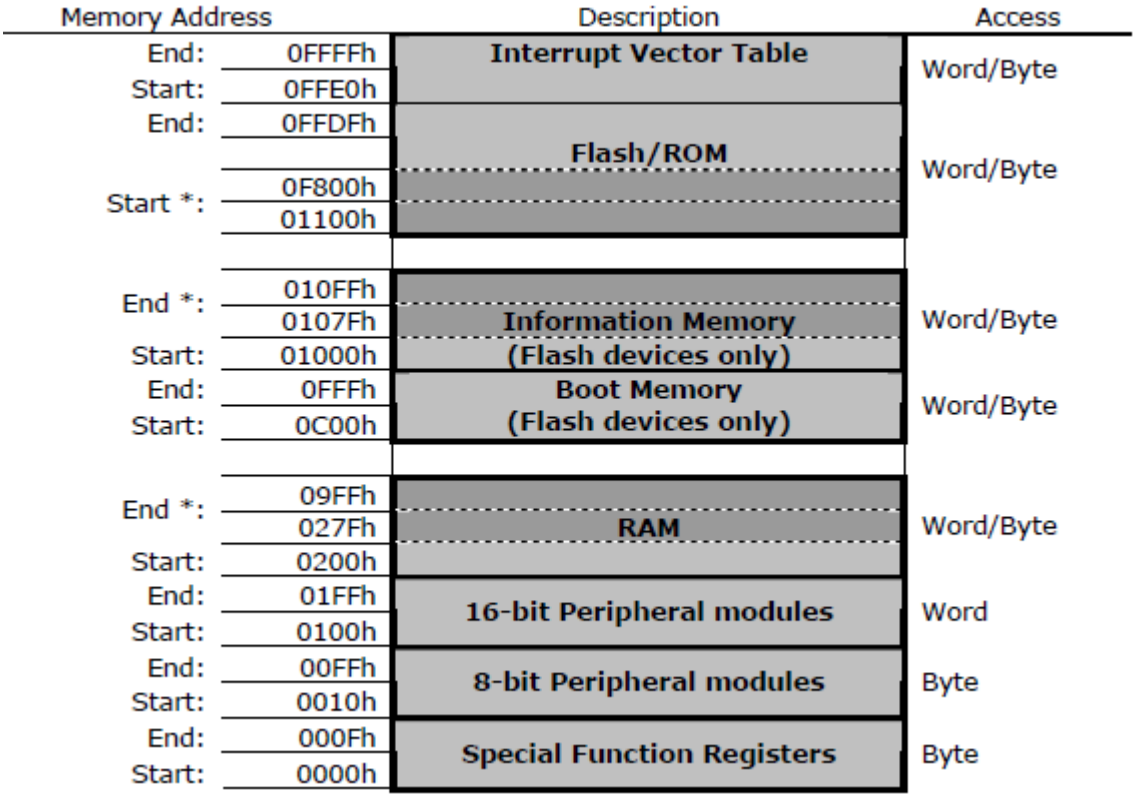
1.5.4 Stack Processing Capability

The MSP430 is a true stack processor, with most of the seven addressing modes implemented for the stack pointer (SP) as well as the other CPU registers (PC and R4 through R15). The capabilities of the stack include:

- Free access to all items on the stack — not only to the top of the stack (TOS)
- Ability to modify subroutine and interrupt return addresses located on the stack

- Ability to modify the stored status register of interrupt returns located on the stack
- No special stack instructions — all of the implemented instructions may be used for the stack and the stack pointer
- Byte and word capability for the stack
- Free mix of subroutine and interrupt handling — as long as no stack modification (PUSH, POP, etc.) is made, no errors can occur

All memory, including RAM, Flash/ROM, information memory, special function registers (SFRs), and peripheral registers are mapped into a single, contiguous address space as shown in *Figure 4−3*.
Note: See the device-specific datasheets for specific memory maps. Code access is always performed on even addresses. Data can be accessed as bytes or words.
The MSP430 is available with either Flash or ROM memory types. The memory type is identified by the letter immediately following "MSP430" in the part numbers.
Flash devices: Identified by the letter "F" in the part numbers, having the advantage that the code space can be erased and reprogrammed.
ROM devices: Identified by the letter "C" in the part numbers. They have the advantage of being very inexpensive because they are shipped pre-programmed, which is the best solution for high-volume designs.



The clock system in the MSP430x5xx family of devices is supported by the Unified Clock System (UCS) module that includes support for a 32-kHz watch crystal oscillator (XT1 LF mode), an internal very-low-power low frequency oscillator (VLO), an internal trimmed low-frequency oscillator (REFO), an integrated internal digitally controlled oscillator (DCO), and a high-frequency crystal oscillator (XT1 HF mode or XT2). The UCS module is designed to meet the requirements of both low system cost and low power consumption. The UCS module features digital frequency locked loop (FLL) hardware that, in conjunction with a digital modulator, stabilizes the DCO frequency to a programmable multiple of the selected FLL reference frequency. The internal DCO provides a fast turn-on clock source and stabilizes in less than 5 µs. The UCS module provides the following clock signals:

• Auxiliary clock (ACLK), sourced from a 32-kHz watch crystal, a high-frequency crystal, the internal lowfrequency oscillator (VLO), the trimmed low-frequency oscillator (REFO), or the internal digitally controlled oscillator DCO.

• Main clock (MCLK), the system clock used by the CPU. MCLK can be sourced by same sources made available to ACLK.

• Sub-Main clock (SMCLK), the subsystem clock used by the peripheral modules. SMCLK can be sourced by same sources made available to ACLK.

• ACLK/n, the buffered output of ACLK, ACLK/2, ACLK/4, ACLK/8, ACLK/16, ACLK/32.

The RTC_A module can be used as a general-purpose 32-bit counter (counter mode) or as an integrated realtime clock (RTC) (calendar mode). In counter mode, the RTC_A also includes two independent 8-bit timers that can be cascaded to form a 16-bit timer/counter. Both timers can be read and written by software. Calendar mode integrates an internal calendar which compensates for months with less than 31 days and includes leap year correction. The RTC_A also supports flexible alarm functions and offset-calibration hardware.

Watchdog Timer (WDT_A) (Link to User's Guide)

The primary function of the watchdog timer (WDT_A) module is to perform a controlled system restart after a software problem occurs. If the selected time interval expires, a system reset is generated. If the watchdog function is not needed in an application, the module can be configured as an interval timer and can generate interrupts at selected time intervals.

Flash/ROM

The start address of Flash/ROM depends on the amount of Flash/ROM present on the device. The start address varies between 01100h (60k devices) to 0F800h (2k devices) and always runs to the end of the address space at location 0FFFFh. Flash can be used for both code and data. Word or byte tables can also be stored and read by the program from Flash/ROM. All code, tables, and hard-coded constants reside in this memory space.

4.3.3 Information memory (Flash devices only)

The MSP430 flash devices contain an address space for information memory. It is like an onboard EEPROM, where variables needed for the next power up can be stored during power down. It can also be used as code memory. Flash memory may be written one byte or word at a time, but must be erased in segments. The information memory is divided into two 128-byte segments. The first of these segments is located at addresses 01000h through to 0107Fh (Segment B), and the second is at address 01080h through to 010FFh (Segment A). This is the case in 4xx devices. It is 256 bytes (4 segments of 64 bytes each) in 2xx devices.

Boot memory (Flash devices only)

The MSP430 flash devices contain an address space for boot memory, located between addresses 0C00h through to 0FFFh. The “bootstrap loader” is located in this memory space, which is an external interface that can be used to program the flash memory in addition to the JTAG. This memory region is not accessible by other applications, so it cannot be overwritten accidentally. The bootstrap loader performs some of the same functions as the JTAG interface (excepting the security fuse programming), using the TI data structure protocol for UART communication at a fixed data rate of 9600 baud.

RAM

RAM always starts at address 0200h. The end address of RAM depends on the amount of RAM present on the device. RAM is used for both code and data.

Peripheral Modules

Peripheral modules consist of all on-chip peripheral registers that are mapped into the address space. These modules can be accessed with byte or word instructions, depending if the peripheral module is 8-bit or 16-bit respectively. The 16-bit peripheral modules are located in the address space from addresses 0100 through to 01FFh and the 8-bit peripheral modules are mapped into memory from addresses 0010h through to 00FFh.

Special Function Registers (SFRs)

Some peripheral functions are mapped into memory with special dedicated functions. The Special Function Registers (SFRs) are located at memory addresses from 0000h to 000Fh, and are the specific registers for:

- Interrupt enables (locations 0000h and 0001h);
- Interrupt flags (locations 0002h and 0003h);
- Enable flags (locations 0004h and 0005h);

SFRs must be accessed using byte instructions only. See the device specific data sheets for the applicable SFR bits.

Central Processing Unit (MSP430 CPU)

The RISC type architecture of the CPU is based on a short instruction set (27 instructions), interconnected by a 3-stage instruction pipeline for instruction decoding. The CPU has a 16-bit ALU, four dedicated registers and twelve working registers, which makes the MSP430 a high performance microcontroller suitable for low power applications. The addition of twelve working general purpose registers saves CPU cycles by allowing the storage of frequently used values and variables instead of using RAM. The orthogonal instruction set allows the use of any addressing mode for any instruction, which makes programming clear and consistent, with few exceptions, increasing the compiler efficiency for high-level languages such as C.

Arithmetic Logic Unit (ALU)

The MSP430 CPU includes an arithmetic logic unit (ALU) that handles addition, subtraction, comparison and logical (AND, OR, XOR) operations. ALU operations can affect the overflow, zero, negative, and carry flags in the status register.

4.4.2 MSP430 CPU registers

The CPU incorporates sixteen 16-bit registers:

- Four registers (R0, R1, R2 and R3) have dedicated functions;
- There are 12 working registers (R4 to R15) for general use

R0: Program Counter (PC)

The 16-bit Program Counter (PC/R0) points to the next instruction to be read from memory and executed by the CPU. The Program counter is implemented by the number of bytes used by the instruction (2, 4, or 6 bytes, always even). It is important to remember that the PC is aligned at even addresses, because the instructions are 16 bits, even though the individual memory addresses contain 8-bit values.

R1: Stack Pointer (SP)

The Stack Pointer (SP/R1) is located in R1.

1st: stack can be used by user to store data for later use (instructions: store by PUSH, retrieve by POP);

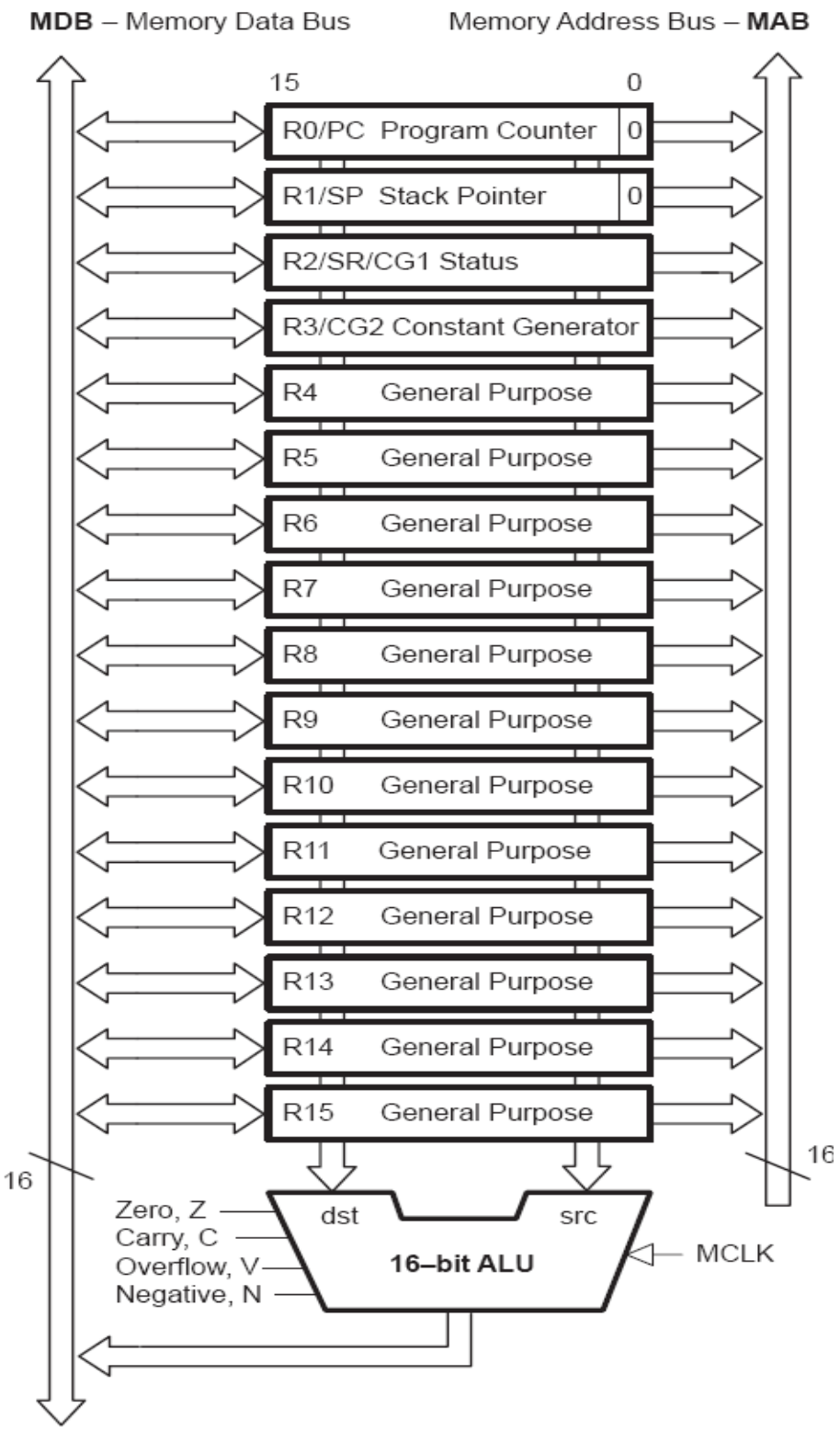
2nd: stack can be used by user or by compiler for subroutine parameters (PUSH, POP in calling routine; addressed via offset calculation on stack pointer (SP) in called subroutine);

3rd: used by subroutine calls to store the program counter value for return at subroutine's end (RET);

4th: used by interrupt - system stores the actual PC value first, then the actual status register content (on top of stack) on return from interrupt (RETI) the system get the same status as just before the interrupt happened (as long as none has changed the value on TOS) and the same program counter value from stack.

R2: Status Register (SR)

The Status Register (SR/R2) stores the state and control bits. The system flags are changed automatically by the CPU depending on the result of an operation in a register. The reserved bits of the SR are used to support the constants generator.



MSP430 CPU block diagram

SR

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved for CG1							V	SCG1	SCG0	OSCOFF	CPUOFF	GIE	N	Z	C

Bit		Description
8	V	Overflow bit. V = 1 ⇒ Result of an arithmetic operation overflows the signed-variable range.
7	SCG1	System clock generator 0. SCG1 = 1 ⇒ DCO generator is turned off – if not used for MCLK or SMCLK.
6	SCG0	System clock generator 1. SCG0 = 1 ⇒ FLL+ loop control is turned off.
5	OSCOFF	Oscillator Off. OSCOFF = 1 ⇒ turns off LFXT1 when it is not used for MCLK or SMCLK.
4	CPUOFF	CPU off. CPUOFF = 1 ⇒ disable CPU core.
3	GIE	General interrupt enable. GIE = 1 ⇒ enables maskable interrupts.
2	N	Negative flag. N = 1 ⇒ result of a byte or word operation is negative.
1	Z	Zero flag. Z = 1 ⇒ result of a byte or word operation is 0.
0	C	Carry flag. C = 1 ⇒ result of a byte or word operation produced a carry.

R2/R3: Constant Generator Registers (CG1/CG2)

Depending of the source-register addressing modes (As) value, six commonly used constants can be generated without a code word or code memory access to retrieve them. This is a very powerful feature, which allows the implementation of emulated instructions, for example, instead of implementing a core instruction for an increment, the constant generator is used.

R4 - R15: General–Purpose Registers

These general-purpose registers are used to store data values, address pointers, or index values and can be accessed with byte or word instructions.

UNIT 8:

On-chip peripherals. Watchdog Timer, Comparator, Op-Amp, Basic Timer, Real Time Clock (RTC), ADC, DAC, SD16, LCD, DMA. 2 Hrs

Using the Low-power features of MSP430. Clock system, low-power modes, Clock request feature, Low-power programming and Interrupt. 2 Hrs

Interfacing LED, LCD, External memory. Seven segment LED modules interfacing. Example – Real-time clock. 2 Hrs

Case Studies of applications of MSP430 - Data acquisition system, Wired Sensor network, Wireless sensor network with Chipcon RF interfaces. 3 Hrs

System reset

The MSP430 families make use of two independent reset signals:

 Hardware reset signal - POR (Power On Reset);

 Software reset signal – PUC (Power Up Clear).

Different events can generate each one of the reset signals. The following sources can generate a POR or a PUC:

 POR:

- Initial device power up;
- Low signal at the reset pin (RST/NMI), when this is configured in reset mode;
- Low signal at the Supervisory Voltage System (SVS), when the register bit PORON is high.

 PUC:

- Active POR signal;
- Expiry of watchdog timer, when it is configured in supervision mode (Further details in section 5.4);

 Flash memory control registers access security key violation. When the hardware reset signal (POR) is high, the Status Register is reset and the Program Counter is loaded with the address in program memory location 0FFFEh. Peripheral registers all enter their power-up state. When the reset signal is from software (PUC), the Status Register is reset, and the Program Counter is loaded with either the reset vector (0FFFEh), or the PUC source interrupt vector. Only some peripheral registers are reset by PUC. These conditions depend on the reset source and the specific MSP430 device.

All 2xx and 4xx MSP430 devices have a reset circuit to detect a power source disturbance, known as a Brown Out Reset (BOR). This circuitry is an elaborate POR system, which includes a hysteresis circuit to allow the device to stay in reset mode until the voltage is higher than the upper threshold (VB_IT+). When the voltage is higher than this value, the BOR takes 2 msec to become inactive and allow the program execution by CPU. Similarly, when the voltage decreases below the lower threshold (VB_IT-), either by power source interruption or battery discharge, the BOR circuit will generate a reset signal, which will remain active until the voltage rises above the lower threshold value.

System clocks

The MSP430 devices have a clock system that allows the CPU and the peripherals to operate from different clock sources. The system clocks depend on the particular device in the MSP430 family:

MSP430x2xx: The Basic Clock Module is composed of one or two oscillators (depending on the device) and is able to work with external crystals or resonators, in addition to the internal digitally controlled oscillator (DCO). It allows a working frequency up to 16 MHz, lower power consumption and lower internal oscillator start up time.

MSP430x4xx: The system clock is defined by the Frequency Locked Loop (FLL+). This system is composed of one or two oscillators (depending on the device), and is able to work with external crystals or resonators, as well as the internal Digitally Controlled Oscillator (DCO). The DCO is adjusted and controlled by hardware, providing multiple working frequencies from an external low frequency oscillator.

The clock sources from these oscillators can be selected to generate a range of different clock signals: Master clock (MCLK), Sub-system main clock (SMCLK) and auxiliary clock (ACLK). Each of these clock signals can be internally divided by 1, 2, 4 or 8, before being made available to the CPU and peripheral devices:

 MCLK: Can be generated by the DCO (but can also be fed by the crystal oscillator), which can be activated and reach stability in less than 6 msec. It can be used by the CPU and high-speed peripherals;

 SMCLK: Used as alternative clock source for peripherals;

 ACLK: Background real-time clock with self wake-up function for low power modes (32.768 kHz watch crystal). It is always fed by the crystal oscillator.

Low/High frequency oscillator (LFXT1)

The Low-frequency/high-frequency oscillator (LFXT1) is implemented in all MSP430 devices.

It can be used with low-frequency 32.768 kHz watch crystals, providing a Real Time Clock (RTC), or standard crystals, resonators, or external clock sources in the range 450 kHz to 8 MHz (16 MHz for the 2xx family). The operating mode selection is defined by a bit of a control register that is configured as a low signal (=0) to provide a low frequency clock, and otherwise to provide a high frequency clock.

Types of interrupts

The MSP430 offers various interrupt sources, both internal and external. There are three types of interrupts:
Reset;
(Non)-maskable interrupts (NMI) by GIE;
Maskable interrupts by GIE.

Each one of these interrupts has a priority, determining which interrupt is taken when more than one interrupt is pending at any one time. The nearer a module is to the CPU/NMIRS, the higher the priority.

The main difference between non-maskable interrupts and maskable interrupts is the fact that the non-maskable interrupt (NMI) cannot be disabled by the General Interrupt Enable (GIE) bit in the Status Register (SR). NMIs are used for high priority events such as emergency shutdown of a machine.

Because all maskable interrupts are recognized by the CPU interrupt control, the GIE bit must be set.

The system reset interrupts (Oscillator/Flash and the Hard Reset) are treated as non-maskable interrupts, with highest priority possessing and their own interrupt vectors.

Non Maskable Interrupts

NMI is not masked by GIE, but is enabled by individual interrupt enable bits, depending on the event source:

NMIIE: Non-Maskable Interrupts Interrupt Enable. When this bit is set, the RST/NMI is configured in NMI mode. A signal edge selected by the WDTNMIES bit generates a NMI interrupt, if the NMIIE bit is set. The RST/NMI flag NMIIFG is also set.

ACCVIE: ACCess Violation to the flash memory Interrupt Enable.

The flash ACCVIFG flag is set when a flash access violation occurs.

OFIE: Oscillator Fault Interrupt Enable. The oscillator fault signal warns of a possible error condition with the crystal oscillator. This kind of signal can be triggered by a PUC signal.

Maskable Interrupts

Peripherals with interrupt capability or the watchdog timer overflow in interval timer mode can cause maskable interrupts. Each maskable interrupt also has an individual enable/disable flag, located in peripheral registers or in the individual module.

Additionally, all maskable interrupts can be disabled by the general interrupt enable (GIE) bit in the status register (SR).

Watchdog timer (WDT and WDT+)

The 16-bit watchdog timer (WDT) module can be used as a:

Processor supervisor: In supervision mode, the main function of the WDT is to supervise the correct operation of the application software. If a problem occurs with the software application that causes the software to hang or enter an infinite loop, the selected time interval in the watchdog timer is exceeded and the WDT performs a system reset: Power Up Clear (PUC). The procedure in this mode consists of performing an interrupt request on counter overflows. Under normal operating conditions, the watchdog timer would be reset by program code before its timer expires and would therefore inhibit the PUC operation.

Interval timer: This module can be configured as an independent interval timer, to perform a “standard” periodic interrupt on counter overflow, for example, to drive an event scheduler (a low-cost operating system). The 16-bit upper counter (WDTCNT) is not directly accessible by software. Its control and the interval time are selected through Watchdog Timer Control Register (WDTCTL). This counter can use the clock signal from ACLK or SMCLK, by defining the appropriate WDTSSEL bit.

The WDT mode is selected by the WDTTMSEL bit in the WDTCTL register. After a PUC condition, the WDT module is configured in supervision mode with approximately 32 msec initial time interval, using DCOCLK. The user should define, stop or clear the WDT before the time interval expires, to prevent a new PUC.

The WDT control is performed through the 16-bit Watchdog Timer Control Register, WDTCTL:

Low power operating modes

This section presents one of the main features of the MSP430 families, that is their low power consumption (around 1 mW/MIPS or less). This is increasingly important with the growth of battery operated embedded systems devices.

Although the MSP430 families are designed for low power consumption, it should borne in mind that this goal can only be accomplished using a design utilizing low power operating modes.

The total power consumption depends on several factors: clock
frequency, ambient temperature, supply voltage, peripheral selection, input/output usage and memory type.

Low power modes

The MSP430 architecture allows six operating modes, five of these modes are suitable for low power consumption operation. These modes are configured by the Status Register bits: CPUOFF; OSCOFF;
SCG1 and SCG0 as follows:

Active mode (AM): Configured disabling the SR bits described above;CPU is active;
All enabled clocks are active; current consumption: 250microA.